



中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生

数学建模竞赛

学 校	景德镇陶瓷大学
--------	---------

参赛队号	26104080010
------	-------------

	1.夏伟东
--	-------

队员姓名	2.张天赐
------	-------

	3.解家磊
--	-------

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题 目： 基于跨核同步层次分层的通用 NPU 多核切图与调度

摘 要：

多核 NPU 调度需要联合决定三件事：把算子-张量计算图切成哪些子图、每个子图放到哪个核心、各核心按什么顺序执行子图，目标是在数据依赖、L1/UB 容量、四条指令流水线和共享 DDR 带宽的约束下缩短总执行时间 Makespan，并控制额外数据搬运。本文把官方评估程序作为唯一的事实基准，按“机理分析—统一建模—算法设计—检验评价”的路线求解三个问题，文中全部 Makespan、搬运量与命中率均由官方评估脚本在固定配置 config.txt 下给出。

机理分析与统一模型。 逐行解析评估器后发现一条决定模型形态的隐含约束：核内调度对“申请片上张量的算子”施加全局申请序，只有队首者能发射，因此排在核内序列前部、数据尚未送达的跨核 COPY_IN 会让整个核心的四条流水线一起停摆，真实代价远大于配置中 500 或 1000 cycle 的名义延迟。据此，本文定义跨核同步层次 $r(v)$ ，以“子图 = (核心, 层次) 等价类”作为切图的构造性定义，证明按 (r, c) 编号后赛题的全部硬约束自动成立，搜索中无需判环；官方 trace 的受控对比显示，仅按同步层次重组子图就使同一用例的整核停摆时间下降 40%。在此之上设计三个问题共用的四阶段算法 CAP-LS：S1 以张量扇出阈值做亲和原子化并按线性扩展连续分块；S2 以 Cube/Vector 负载、DDR 字节与同步深度的复合代价做分核与增量局部搜索；S3 做同步层次分层；S4 以 14~18 组多起点候选由官方评估器择优，并用保底池保证逐用例加速比随核数不降。

针对问题一（场景 A，子图即 Task），建立 Task 级时序模型，揭示“边界搬运随粒度增加、换入换出随粒度减少”的权衡，得出场景 A 偏粗粒度的结论，并加入代理驱动的模拟退火调整 Task 粗细。在全部 100 个用例上， $N = 2 \sim 5$ 的平均加速比为 1.81、2.55、3.19、3.74；单个（用例，核数）的端到端求解时间中位数为 45.8 s，351/400 个配置在 10 分钟内完成。

针对问题二（场景 B，核即 Task），证明跨核搬运量恰为以张量字节为权的 $(\lambda - 1)$

超图割，并把“驻留对顺序”的权衡建模为容量感知表调度，新增单算子子图候选。平均加速比为 2.06、2.88、3.63、4.24；冠军方案的换入换出总量下降 64%； $N = 4$ 时 29 个用例出现超线性加速，其来源是单核基准自身的大量换入换出在多核切分后消失。

针对问题三（场景 B+ 共享只读 L2），由评估器的 FIFO 命中规则推出命中只来自多核复读与换入复读，给出命中上界与时序条件，把预测命中字节按 250 B/cycle 折算进分配代价，并让问题二、三的冠军方案互投。只读 Cache 配置在 $N = 2 \sim 5$ 的平均加速比为 2.06、2.89、3.67、4.31；只读 Cache 相对无 L2 的平均加速比在 $N = 1 \sim 5$ 时为 1.009、1.001、1.003、1.013、1.020，按字节命中率 9.2%~29.0%。机理分析与灵敏度实验表明，题设的 1 MB、250 B/cycle 已接近饱和，收益上限由计算图可共享的复读量决定，而约 60% 的未命中来自并发首读与 FIFO 淘汰，是 Cache 感知调度可继续争取的空间。

模型检验。与随机、拓扑等分、负载均衡、通信聚类四类基线相比， $N = 4$ 时三个问题的平均加速比分别比最强基线高 +36%、+30%、+30%（Wilcoxon 符号秩检验，Holm 校正 $p \leq 3.8 \times 10^{-14}$ ），且没有一个用例劣于该基线；消融实验表明同步层次分层是贡献最大的单一机制，去掉后三个问题的平均加速比分别变化 -19.8%、-8.3%、-9.2%，与通信感知分块同时去掉则达 -58.0%、-39.7%、-39.9%。全部 1500 个最终方案经官方命令行脚本逐一复核，不一致数为 0。

本文的创新点在于：（1）从评估器中识别出“全局申请序导致整核停摆”的机制，并据此提出跨核同步层次分层；（2）把可行性写成构造性定理，使搜索只在分核这一维上进行；（3）用一套框架覆盖三个场景，场景差异只落在代价权重、候选集合与 L2 折算三处。

关键词：多核 NPU；计算图切分；无环划分；跨核同步层次；容量感知表调度；只读 L2 Cache；组合式局部搜索

目 录

第 1 章	问题背景与问题重述	6
1.1.	问题背景	6
1.2.	问题重述	7
1.3.	问题分析	8
1.3.1.	问题一的分析	8
1.3.2.	问题二的分析	8
1.3.3.	问题三的分析	9
1.4.	技术路线	9
1.4.1.	机理分析	9
1.4.2.	统一建模	9
1.4.3.	算法设计与问题求解	9
1.4.4.	检验评价	9
第 2 章	模型假设	11
第 3 章	符号说明	12
第 4 章	评估机制解析与统一优化模型	13
4.1.	问题分析	13
4.2.	评估器的五条实现事实	13
4.3.	测试数据的结构特征	15
4.4.	统一优化模型	16
4.4.1.	算子级 DAG	16
4.4.2.	决策变量	17
4.4.3.	约束条件	17
4.4.4.	目标函数与解析代理	18
4.5.	理论下界与超线性加速的来源	18
4.6.	可行性的构造性刻画	19
4.7.	本章小结	20
第 5 章	问题一模型的建立与求解	21
5.1.	问题分析	21
5.2.	求解思路	21
5.3.	场景 A 的 Task 级时序模型	22
5.4.	切图粒度的权衡	23
5.5.	CAP-LS 算法设计	23
5.5.1.	S1: 亲和原子化与无环切块	24
5.5.2.	S2: 负载、通信与同步深度联合分配	24
5.5.3.	S3: 同步层次分层	24
5.5.4.	S4: 多起点组合与保底池	25
5.6.	高效求解策略	26

5.6.1.	减少官方评估的四项措施.....	26
5.6.2.	代理驱动的模拟退火.....	27
5.7.	复杂度分析	27
5.8.	求解结果与分析	27
5.8.1.	平均加速比.....	27
5.8.2.	求解时间.....	29
5.9.	本章小结	30
第 6 章	问题二模型的建立与求解	31
6.1.	问题分析	31
6.2.	求解思路	31
6.3.	场景 B 的执行模型与搬运量公式.....	32
6.4.	算法改进	33
6.4.1.	粒度两端覆盖与同步权重.....	33
6.4.2.	单算子子图与容量感知表调度.....	33
6.4.3.	优先级采样与跨问题互投.....	34
6.5.	粒度、驻留与通信的耦合	34
6.6.	求解结果与分析	34
6.6.1.	平均加速比.....	34
6.6.2.	超线性加速的来源.....	35
6.6.3.	额外搬运的构成.....	36
6.6.4.	结构分层与难例.....	37
6.7.	本章小结	37
第 7 章	问题三模型的建立与求解	38
7.1.	问题分析	38
7.2.	求解思路	38
7.3.	只读 L2 的命中模型	39
7.4.	Cache 感知的分配代价	39
7.5.	L2 容量与带宽的影响.....	40
7.6.	求解结果与分析	40
7.6.1.	无 L2 与只读 Cache 两种配置的对比	40
7.6.2.	只读 Cache 配置的加速比.....	42
7.6.3.	命中与未命中的来源.....	43
7.6.4.	问题三与问题二的比较.....	43
7.7.	本章小结	44
第 8 章	模型检验与灵敏度分析	45
8.1.	实验设置与统计口径	45
8.2.	与基线算法的对比	45
8.3.	消融实验	46
8.4.	灵敏度分析	47

8.4.1. 切图粒度.....	47
8.4.2. 硬件参数.....	48
8.5. 与理论下界的距离	49
8.6. 代理模型的可靠性与候选组合	50
8.7. 图结构特征与难例	52
8.8. 赛题优化方向的对应	53
8.9. 本章小结	54
第 9 章 模型的评价与推广	55
9.1. 模型的优缺点	55
9.1.1. 模型的优点.....	55
9.1.2. 模型的缺点.....	55
9.2. 模型的改进与推广	56
9.3. 总结	56
参考文献.....	58
附 录.....	60
附录 A 逐用例结果	60
附录 B 程序说明与复现步骤	77
附录 C 核心程序（节选）	79

第1章 问题背景与问题重述

1.1. 问题背景

随着人工智能模型规模持续增长,单个 AI 核心受制造成本、功耗与散热的约束,难以独立满足算力需求,多核 NPU 因此成为提升并行能力与能效的主流架构^[1]。在多核 NPU 上运行一个神经网络,编译器需要把完整的计算图划分为若干子图、把子图分配到各个核心,并确定每个核心上子图的执行顺序。这一过程既要利用核内高效的计算单元与私有缓存复用数据,又要协调跨核依赖,本质上是在多核并行度、核内缓存容量与共享主存带宽三者之间做权衡:切得太碎,跨核数据搬运与同步增多;切得太粗,并行空间不足,单个核心还可能因缓存装不下而频繁换入换出。

赛题给出的多核 NPU 由 N 个同构 AI 核心和共享片外主存 DDR 组成,硬件结构与本题的固定参数如图 1-1 所示。每个核心有 Cube 矩阵单元 (PIPE_M)、Vector 向量单元 (PIPE_V) 和两类搬运单元 (PIPE_MTE2 负责 DDR→L1/UB 与 UB→L1, PIPE_MTE3 负责 L1/UB→DDR 与 L1→UB),以及私有缓存 L1 (512 KB) 和 UB (128 KB)。四条流水线可以并行,但每条流水线同一时刻只执行一条指令。所有核心读写 DDR 时公平共享 60 B/cycle 的总带宽;问题三另加一个所有核心共享、带宽独立于 DDR 的只读 L2 Cache (1 MB, 250 B/cycle)。

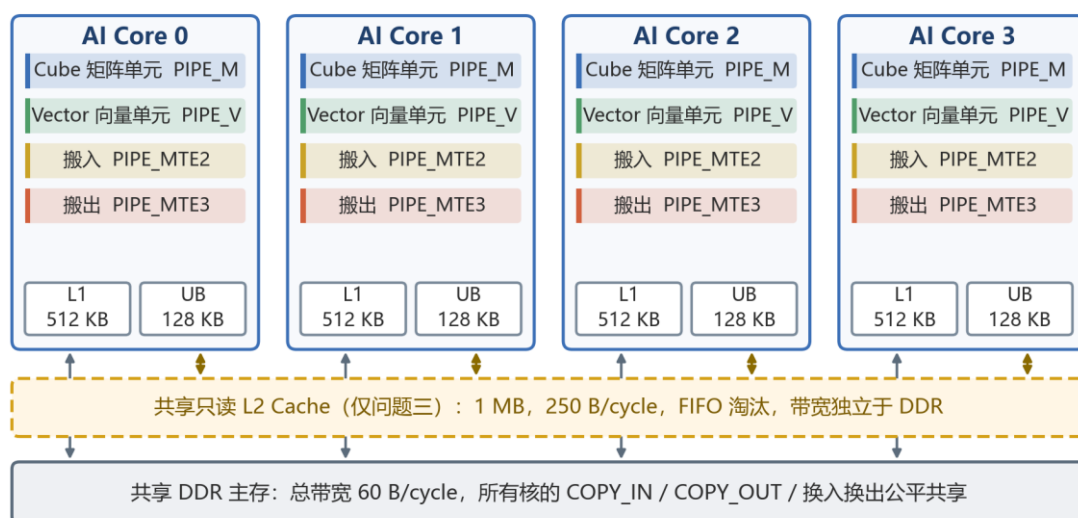


图 1-1 多核 NPU 硬件结构与本题固定配置

待调度对象是一张有向无环计算图 $G = (V, E)$, $V = T \cup O$ 由张量节点和操作节点组成,边只允许“张量→操作”或“操作→张量”。张量带有存储位置 pos (DDR/L1/UB) 与字节数 size; 操作带有类型 op、所在流水线 pipe 与周期数 cycles。图的输入、输出张量位于 DDR, 中间张量位于核内缓存。以分块矩阵乘为例, 每个数据块依次经历“COPY_IN 搬入—Cube 计算—COPY_OUT 写回”, 计算图正是由大量这样的分块计算链交织而成。

与本题相关的研究大致有三类。第一类是图与超图划分：多层划分方法^[5]以割边为目标，超图划分的连通度 $(\lambda-1)$ 目标能够准确刻画“一份数据被多方读取”的通信量^[6]，有向无环划分进一步要求划分后的商图无环^[7]。第二类是深度学习计算图在多设备上的切分与放置，例如用强化学习学习放置策略^[8]、在更大的并行策略空间中搜索^[9]、分层自动化算子间与算子内并行^[10]，以及面向多核加速器的粗粒度数据流调度^[11]。第三类是带通信代价的 DAG 调度与整体同步并行（BSP）模型^[3,13,14]。这些方法的代价模型多以通信量或实测时间为依据，而本题的成绩完全由一个确定的评估程序给出，其中核内启发式调度的“全局申请序”与只读 FIFO 共享 Cache 是通用模型通常不考虑的机制。本文的出发点正是先把这些机制从评估器中读出来，再据此确定模型的结构。

1.2. 问题重述

一个多核调度方案需要同时给出两部分内容：一是**切图方案** `node_to_subgraph`，为每个非 COPY_IN/COPY_OUT 的操作指定子图编号；二是**子图调度方案** `core_schedules`，为每个核心列出其上子图的执行顺序。方案须满足：映射覆盖全部核内操作，子图依赖图为有向无环图，同核子图顺序不违反依赖。评估程序据此按场景规则重建跨子图的数据搬运，用固定的核内调度算法（Step1 确定拓扑序、Step2 在容量不足时插入换入换出、Step3 在四条流水线上排布）确定每个 Task 内的算子顺序，再在共享带宽下模拟多核并行执行，输出 Makespan（主指标）、总额外数据搬运量和 Cache 命中率（次指标）。根据是否具备核间同步机制，赛题区分两种硬件场景：

场景 A（无核间同步）：一个 Task 只能包含一个子图，同一核上的 Task 串行执行且核内缓存被清空，所有跨子图数据都经 DDR 中转；同核 Task 切换等待 100 cycle，跨核前驱完成后等待 1000 cycle。

场景 B（有核间同步）：同一核心的全部子图合并为一个 Task，同核子图直接复用 L1/UB 中的数据；跨核依赖由源核 COPY_OUT、500 cycle 同步延迟和目标核 COPY_IN 完成。

在此基础上，赛题提出三个问题：

问题 1：在场景 A 下建立多核切图与调度模型并设计求解算法，以缩短 Makespan 为主要目标、兼顾总额外数据搬运量，要求在 2~5 核范围内稳定、高效地求解；正文给出 1~5 核的平均加速比折线图，附录逐用例列出 Makespan 与总额外搬运量。

问题 2：在场景 B 下，在严格满足 L1/UB 容量约束的前提下进一步缩短 Makespan、减少额外搬运，输出内容与问题 1 相同。

问题 3：在场景 B 的基础上引入共享只读 L2 Cache，研究其容量与带宽对切图与调度的影响并设计算法；给出 1~5 核下无 L2 与只读 Cache 两种配置的对比曲线，以及相同核数下只读 Cache 相对无 L2 的加速比（无 L2 的 Makespan 除以只读 Cache 的 Makespan），附录逐用例列出两种配置的 Makespan、额外搬运量与命中率。

加速比口径。单核基准 Makespan 由核内调度算法在整图上计算，单核加速比

定义为 1； N 核加速比为 $SU_N = MK_1/MK_N$ 。平均加速比取 100 个用例加速比的算术平均。由于核内调度是启发式的，单核基准本身并非最优，因此可能出现加速比大于核数的现象（第 4.5 节给出机理）。

1.3. 问题分析

三个问题共享同一组决策——切图、分核、同核排序——且三者相互牵制：切图改变子图规模、并行空间与通信边界；分核决定负载均衡与跨核依赖的数量；同核顺序又影响依赖等待、张量驻留时长与换入换出。即便去掉通信、容量与带宽，把独立任务分到同构机器上使 **makespan** 最小已是 NP-难问题（第 4.1 节），因此本文不追求精确解，而是在“单用例求解时间远小于官方评估时间”的预算内稳定逼近理论下界。三个问题的差异与联系见表 1-1。

表 1-1 三个问题的差异与联系

项目	问题一（场景 A）	问题二（场景 B）	问题三（场景 B + L2）
Task 的组成	一个子图	一个核心的全部子图	同问题二
同核子图边界	经 DDR 往返，产生搬运	零搬运，只决定执行顺序	同问题二
跨核依赖代价	1000 cycle，须等全部前驱 Task	500 cycle，另有申请序停摆	同问题二，复读可由 L2 吸收
子图粒度倾向	偏粗	中到细	中到细
首要优化量	同步层次数与 Task 串行级数	负载均衡与跨核依赖后置	负载均衡与 Cache 命中

1.3.1. 问题一的分析

场景 A 中子图就是 Task，同核 Task 严格串行、彼此之间不保留缓存数据，因此每一条子图边界都要经 DDR 往返一次，跨核依赖还要额外等待 1000 cycle。这使问题一的难点落在“粒度”上：子图切得越细，边界搬运和 Task 切换越多；切得越粗，单个 Task 越可能超出 L1/UB 容量而触发换入换出，同时核间并行也越少。另一方面，一个 Task 必须等其全部前驱 Task 完成才能激活，跨核依赖链的级数直接串行累加到 **Makespan** 上。因此问题一需要：（1）一个能刻画 Task 激活与边界搬运的时序模型，用来比较不同粒度；（2）一种让跨核依赖级数尽量少的子图构造；（3）在上述构造之外调整 Task 粗细的局部改进手段。

1.3.2. 问题二的分析

场景 B 把同一核心的子图合成一个 Task，同核边界不再产生搬运，切图在核内只剩“决定执行顺序”这一个作用，跨核搬运量则只取决于分核。但场景 B 引入了两个新问题：其一，跨核依赖虽然只有 500 cycle 的名义延迟，核内调度的全局申请序却可能让一个未就绪的跨核 **COPY_IN** 堵住整个核心（第 4.2 节事实 3），必须把依赖远端数据的算子在时间上往后放；其二，同一 Task 内所有子图共享 L1/UB，核内

顺序决定了同一时刻有哪些张量驻留在片上，顺序不当会引发大量换入换出。问题二因此是“分核决定通信、顺序决定驻留”的两层问题，需要在问题一框架上增加直接控制核内驻留的排序手段。

1.3.3. 问题三的分析

问题三在场景 B 上增加共享只读 L2。要回答“容量和带宽如何影响切图与调度”，首先要弄清命中从哪里来：评估器的 Cache 只服务 COPY_IN、按 FIFO 淘汰、在发射时判定命中、在搬运完成时写入。由此可以推出，命中只可能来自同一张量被多个核心各自读入，或同一核心对同一张量的再次读入；命中字节的上界由计算图中可共享的复读量决定，能否达到则取决于读者之间的时间间隔。问题三需要把命中折算进分配代价，使“把被多核共享的张量复制到多个核心”从纯粹的损失变为可以承受的代价，并通过对比实验与灵敏度分析说明 L2 容量、带宽的作用边界。

1.4. 技术路线

本文的技术路线如图 1-2 所示，共五个环节，各环节的内容与对应章节如下。

1.4.1. 机理分析

逐行解析官方评估器，归纳出决定模型形态的五条实现事实（第 4.2 节），其中“全局申请序导致整核停摆”是本文算法设计的出发点；同时提取 100 个测试用例的结构特征，判断并行潜力、带宽压力与容量压力的量级（第 4.3 节）。

1.4.2. 统一建模

把二部图收缩为算子级 DAG，建立以切图、分核、排序为决策变量、以九条硬约束为约束、以官方 Makespan 为目标的统一优化模型，给出理论下界与超线性加速的来源；进而定义跨核同步层次，把可行性写成构造性定理（第 4.6 节）。

1.4.3. 算法设计与问题求解

设计三个问题共用的四阶段算法 CAP-LS (Communication-Aware Partitioning with Level Stratification)，三个问题只在代价权重、候选集合与 L2 折算三处不同：问题一加入 Task 级时序模型与代理驱动退火（第 5 章），问题二加入单算子子图与容量感知表调度（第 6 章），问题三加入命中模型与 Cache 感知代价（第 7 章）。

1.4.4. 检验评价

在全部 100 个用例上做基线对比、消融实验、粒度与硬件参数灵敏度分析、理论下界差距分析和代理模型可靠性检验（第 8 章），并用官方命令行脚本逐一复核最终方案。

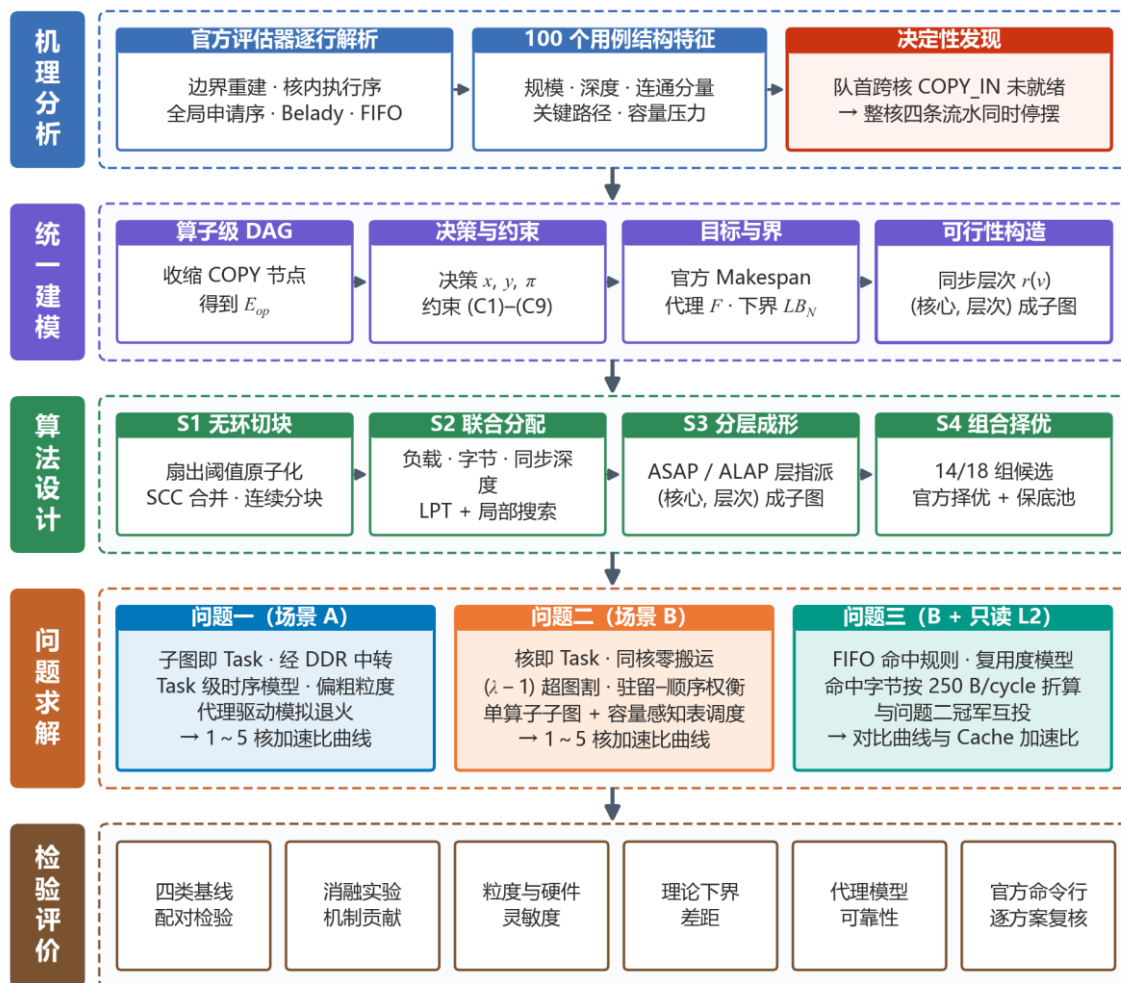


图 1-2 全文技术路线

第2章 模型假设

(1) **评估器即真值假设。** 模型推断与官方评估器行为冲突时，一律以评估器为准；文中上报的 Makespan、额外搬运量与命中率全部由官方脚本在 `data/config.txt` 固定配置下给出，解析模型只用于搜索内部排序。这一假设保证了结论与赛题评分口径一致。

(2) **核内调度固定假设。** 核内 Step1~Step3 是给定的确定性过程，方案只能通过子图划分与子图顺序间接影响它。赛题不允许修改核内调度，因此模型把它视为“子图序列 $\rightarrow$ 算子顺序与换入换出”的确定映射。

(3) **核心同构假设。** N 个核心的计算能力、流水线结构与私有缓存容量完全相同，因此分核只需考虑负载与通信，不必考虑核心差异。

(4) **DDR 公平共享假设。** n 个在途 DDR 搬运各得 $1/n$ 的带宽，搬运加入或完成时重新分配，与多核评估器的带宽池一致。

(5) **流水线独占假设。** 每核每条流水线同一时刻只有一条在飞指令，四条流水线之间可以并行，Cube 与 Vector 计算可以相互重叠。

(6) **只读 L2 语义假设。** 仅 COPY_IN 可以命中，命中键为写入片上的逻辑张量编号；未命中的读取在完成时写入 Cache，按 FIFO 淘汰且命中不刷新顺序；大于容量的张量不缓存；L2 与 DDR 的带宽池相互独立。

(7) **搬运时长假设。** 一次搬运的参考时长为字节数除以有效带宽（命中 250 B/cycle，未命中 60 B/cycle），实际时长由带宽池公平共享后的剩余工作量决定。

(8) **求解预算假设。** 单用例单候选的构造时间应远小于一次官方评估，算法不对“切图 $\times$ 分核 $\times$ 排序”做穷举；大图的官方评估较慢，允许以解析代价在候选之间预筛。

第3章 符号说明

表 3-1 符号说明

符号	说明
$G = (V, E), V = T \cup O$	原始计算图，张量节点集与操作节点集
V_{op}, E_{op}	可切分算子集（非 COPY 操作）与收缩 COPY 后的算子级依赖边集
s_t	张量 t 的字节数
c_v, p_v	算子 v 的周期数与所在流水线
P_t, C_t	张量 t 的生产者集合与消费者集合
N	核心数
x_v, y_s, π_k	决策：算子 v 的子图、子图 s 的核心、核 k 上的子图顺序
$c(v) = y_{x_v}$	算子 v 所在的核心
$r(v), l(v)$	跨核同步层次（ASAP）与一般的单调层指派
σ	算子级 DAG 的一个线性扩展（拓扑序）
θ, β	张量扇出阈值；粒度参数（单块计算量上限 $\beta W/N$ ）
C_{L1}, C_{UB}, C_{L2}	L1、UB、L2 的容量
bw, bw_{L2}	DDR 总带宽 60 B/cycle, L2 带宽 250 B/cycle
W_{same}, W_{cross}	场景 A 同核切换等待 100 cycle, 跨核等待 1000 cycle
δ	场景 B 跨核同步延迟 500 cycle
L_k^M, L_k^V	核 k 上 Cube、Vector 计算的总周期
$B_k^{in}, B_k^{out}, B_{DDR}$	核 k 的搬入、搬出字节数，方案的 DDR 搬运总字节
F	搜索内部使用的解析代理目标
MK_N, SU_N	N 核 Makespan; 加速比 MK_1/MK_N
LB_N	N 核 Makespan 的理论下界

第4章 评估机制解析与统一优化模型

三个问题使用同一套决策、同一个评估程序，只在 Task 的组成与 L2 上不同。本章先说明问题的计算复杂性和“先读评估器”的必要性，再给出评估器的五条实现事实与测试数据的结构特征，最后建立三个问题共用的统一优化模型，并证明一种使全部硬约束自动成立的子图构造。第 5~7 章在此基础上分别求解三个问题。

4.1. 问题分析

切图、分核、排序三项决策属于带通信代价的 DAG 调度问题。即使不考虑通信，把独立任务分到 N 台同构机器上使 makespan 最小就已是 NP-难问题；加入优先约束后，即使任务等长也是 NP-完全的^[2]；带通信代价的 DAG 调度更是一个以启发式方法为主的领域^[3]。本题还叠加了片上容量、流水线发射顺序和共享带宽争用。下面的命题说明，哪怕只保留最简单的特例，精确求解也不现实。

命题 4-1 把问题限制为“ $N = 2$ 、算子级 DAG 无边、所有算子在同一条流水线、无容量与带宽约束、通信代价为零”，其判定版本（是否存在 Makespan 不超过给定值的方案）仍是 NP-完全的。

证明 任给 PARTITION 实例 $\{a_1, \dots, a_n\}$ (PARTITION 是 NP-完全的^[4])，构造含 n 个孤立算子的计算图，令算子 v_i 的周期 $c_{v_i} = a_i$ 。任一方案的 Makespan 为 $\max(L_0, L_1)$ ，其中 L_k 为核 k 上的周期之和，且 $L_0 + L_1 = \sum_i a_i$ 。于是存在 Makespan $\leq \frac{1}{2} \sum_i a_i$ 的方案当且仅当原实例可二等分。归约是多项式的，方案的可行性与 Makespan 也能在多项式时间内验证，故判定版本是 NP-完全的。 $\square$

命题 4-1 决定了本文的求解策略：不追求精确解，而是用构造性的方法缩小搜索空间，在“单用例求解时间远小于官方评估时间”的预算内逼近理论下界。另一方面，本题的成绩完全由官方评估程序给出，程序中的核内调度与多核模拟包含许多通用模型不会实现的实现细节，模型若与评估器的行为不一致，优化得再好也无法兑现为成绩。因此本文的第一步是逐行解析 code/ 下的评估脚本，把决定成绩的机制显式地写进模型。

4.2. 评估器的五条实现事实

事实 1（边界重建规则）。 可切分节点恰为非 COPY 算子。评估器先丢弃原图中的 COPY 节点，再按场景规则重建边界搬运：场景 A 对每个子图的输入、输出边界张量各插入一次 COPY；场景 B 只对图输入、图输出和跨核边插入 COPY。由此可得推论：场景 B 中同核子图之间的边界不产生任何搬运，切图在核内只起决定执行顺序的作用。

事实 2（核内执行序由子图顺序决定）。 场景 B 中核内算子序等于“按子图名次分桶、桶内保持 Step1 顺序”的稳定排序。因此同核子图的顺序不仅决定依赖等待，

也直接决定核内算子的物理执行顺序。

事实 3（全局申请序）。 Step3 把所有“会申请片上张量”的算子排进一个全局申请序，只有排在队首的算子才能发射（以下摘自 `code/schedule_step3.py`，有删节）：

```
candidate = allocation_order[next_allocation_rank]
if (candidate in allocation_ready and candidate_pipe == pipe
    and can_allocate_outputs(candidate)):
    op_id = candidate          # 只有申请序队首可以发射
```

绝大多数算子都要申请片上输出张量，于是排在核内序列前部、数据尚未送达的跨核 COPY_IN 会堵住申请序的队首，使整个核心的四条流水线几乎同时停下，如图 4-1 所示。配置中的跨核延迟只有 500 cycle，但这种“整核停摆”的持续时间取决于远端数据何时到达，可以长得多。这说明只把跨核通信量降下来并不够，还必须把依赖远端数据的算子在时间上往后放。

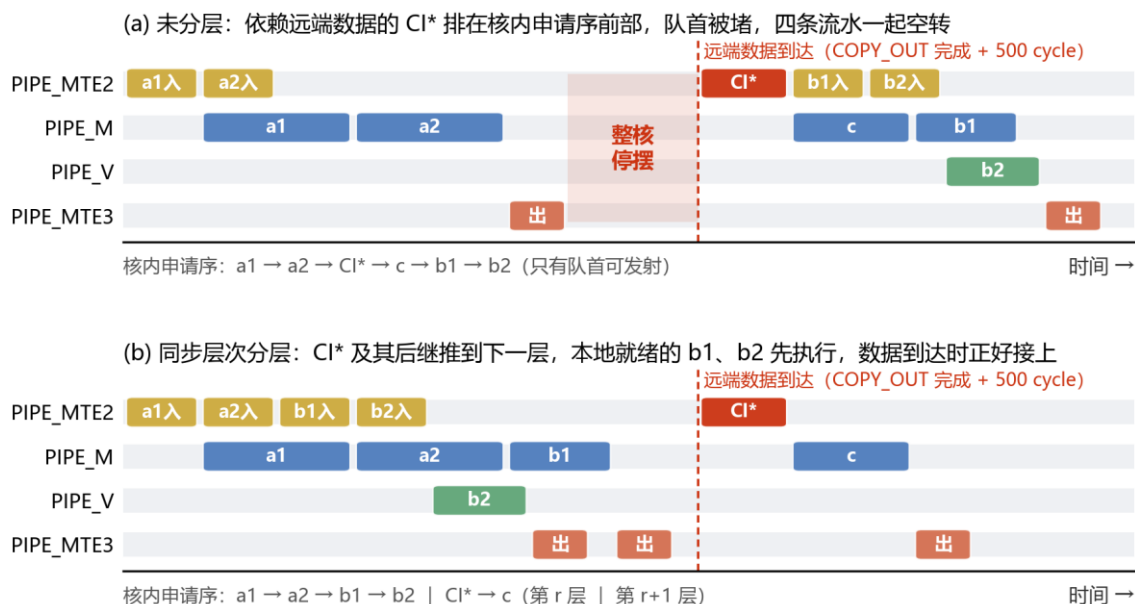


图 4-1 “全局申请序”造成整核停摆的机理示意

为验证这一机制，用官方命令行评估器的 `trace` 输出对 `case_003`（问题二， $N = 4$ ）做了受控对比（图 4-2）。(a)、(b) 两个方案的分核完全相同，只差“是否按第 4.6 节的同步层次成子图”一项：不分层时 `Makespan` 为 237,543 cycle，四个核合计出现 254,527 cycle 的整核停摆（四条流水同时空闲而核上仍有未完成算子），约占总核时的 26.8%；分层后整核停摆降到 151,909 cycle（下降 40%），`Makespan` 缩短 14.7%。最终方案 (c) 在分层的基础上再用表调度控制核内顺序，整核停摆进一步降到 92,107 cycle，`Makespan` 为 182,260 cycle。

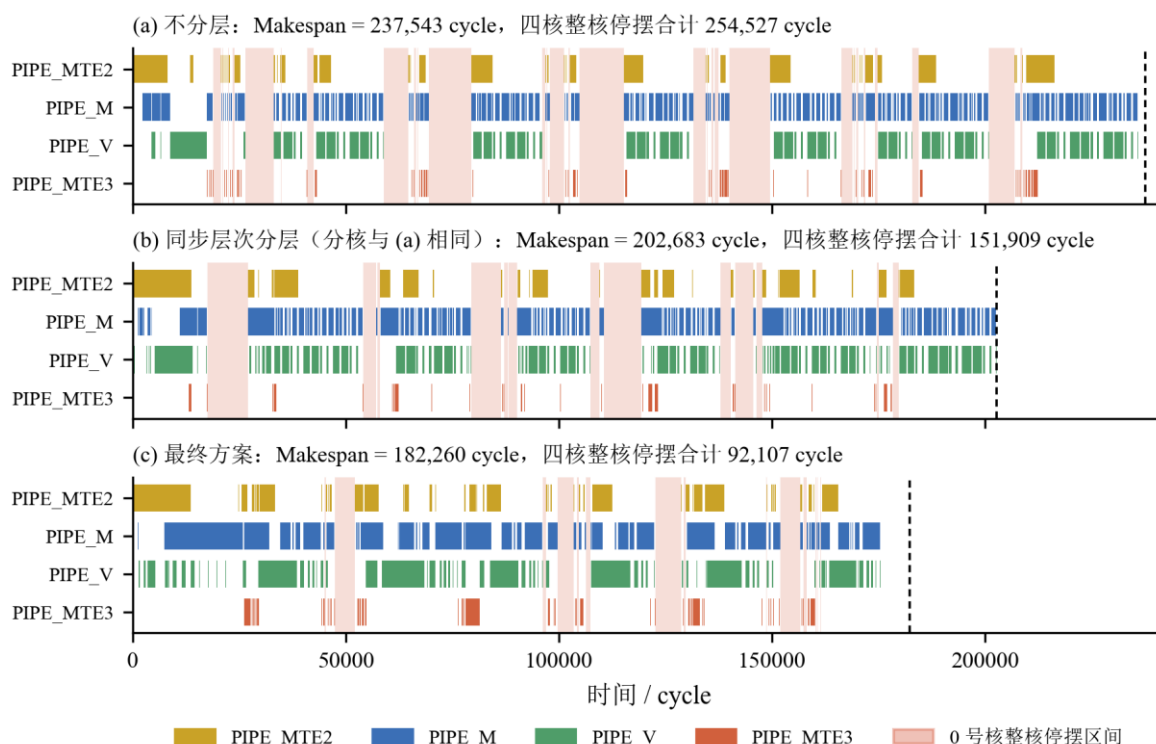


图 4-2 官方 trace 中 0 号核四条流水线的甘特图 (case_003, 问题二, $N = 4$): (a) 不分层; (b) 同步层次分层, 分核与 (a) 相同; (c) 最终方案

事实 4 (换出规则)。 Step2 在容量超限时换出“下一次使用最远”的驻留张量 (schedule_step2.py 中 trigger_one_spill 按 next_use 降序取首个), 即 Belady 规则^[12]。给定核内顺序后, 换出对象的选择已近似最优, 换入换出量主要由核内顺序与子图边界决定。

事实 5 (Cache 在发射时判定命中、在完成时写入)。 问题三中 COPY_IN 是否命中, 在它发射的时刻按当时的 FIFO 内容判定; 条目要到搬运完成后才写入, 命中也不刷新 FIFO 顺序。因此多个核心并发首读同一张量时会全部未命中。命中比未命中快且不占 DDR 带宽池, 会改变各 COPY_IN 的完成先后, 进而改变后续的淘汰顺序, 所以同一方案在问题三下的 Makespan 并不保证不高于它在问题二下的值 (第 7.6 节报告了这种情形的出现次数)。

4.3. 测试数据的结构特征

对 100 个官方用例提取结构特征, 统计结果见表 4-1, 其中三个关键特征的分布见图 4-3。

表 4-1 100 个测试用例的结构特征

特征	含义	最小值	中位数	最大值
$ V_{op} $	可切分算子数	552	3720.5	35705
D	算子级 DAG 深度	3	47.5	2440
$ V_{op} /D$	平均宽度	4.7	44.3	3110.3
K	弱连通分量数	1	39.5	2666
$\rho_{\max}$	最大连通分量的算子占比	0.000	0.122	1.000
$CP/\sum c_v$	关键路径占总计算量之比	0.0002	0.0232	0.2926
$\max(L^M, L^V)bw / B_{DDR}$	计算量与 DDR 时间下界之比	3.8	19.9	1177.2
$\sum_{t \in UB} s_t / C_{UB}$	UB 张量总量与容量之比	0.0	14.6	2745.1

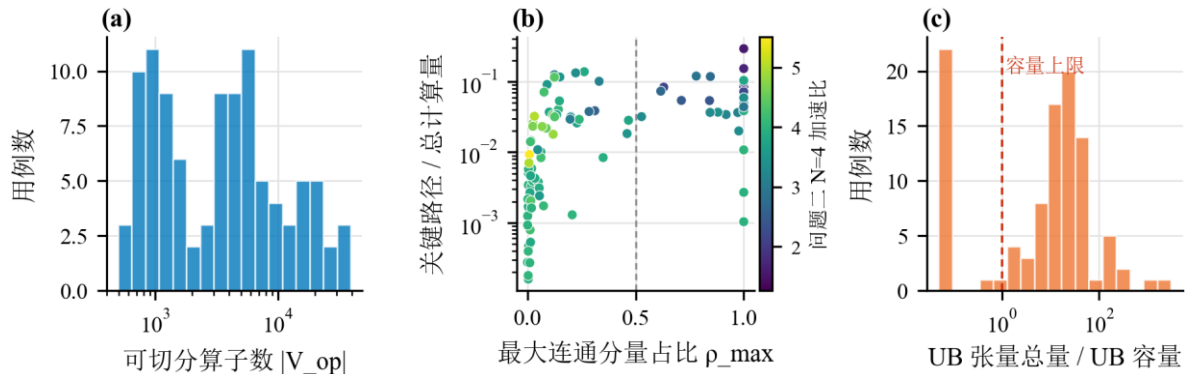


图 4-3 测试用例的结构特征：(a) 规模分布；(b) 最大连通分量占比与关键路径占比（颜色为问题二 $N=4$ 的加速比）；(c) UB 容量压力

由表 4-1 与图 4-3 可以得到三条指导算法设计的判断：

(1) **并行潜力普遍充足。** 关键路径中位数只占总计算量的 0.0232，有 50 个用例的弱连通分量不少于 40 个。先按连通结构把计算分到各核，再处理跨核通信，是合理的第一步。

(2) **多数用例受计算约束。** 计算量与 DDR 时间下界之比的中位数为 19.9，适度增加重复读取来换取并行度通常是划算的；但最小值只有 3.8，目标中必须保留 DDR 项。

(3) **整图单 Task 本身受容量之苦。** UB 压力的中位数为 14.6 倍，官方单核基准在 40 个用例上产生了换入换出，最多 2.77×10^8 字节。多核切分把 Task 缩小后，这部分搬运会减少甚至消失，这是超线性加速的来源（第 4.5 节）。

此外，有 16 个用例只有一个连通分量、29 个用例的最大分量占比超过 0.5（图 4-3b 中竖线右侧），它们必须横切主分量，是主要的难例。

4.4. 统一优化模型

4.4.1. 算子级 DAG

首先把二部图收缩成算子之间的依赖关系：

$$E_{op} = \{(u, v) \in V_{op}^2: G \text{ 中存在 } u \rightsquigarrow v \text{ 且路径上的中间操作全为 COPY}\} \quad (4-1)$$

这与评估器收缩 COPY 节点的做法一致。需要注意，形如 $u \rightarrow \text{COPY_OUT} \rightarrow t_{DDR} \rightarrow \text{COPY_IN} \rightarrow v$ 的依赖在“张量的生产者/消费者”关系上是断开的，但在 E_{op} 中仍是一条边；漏掉它会使自查结论为无环，而评估器却报告子图商图有环。

4.4.2. 决策变量

$$x: V_{op} \rightarrow \{1, \dots, S\}, y: \{1, \dots, S\} \rightarrow \{0, \dots, N-1\}, \pi_k (k = 0, \dots, N-1) \quad (4-2)$$

其中 x 为切图（算子到子图）， y 为分核（子图到核心）， π_k 为核 k 上子图的执行顺序；子图数 S 由算法决定（赛题不限制）。记 $c(v) = y_{x_v}$ 为算子 v 所在的核心。

4.4.3. 约束条件

（C1）覆盖与唯一： x 是 V_{op} 的一个划分，每个子图恰好在某个 π_k 中出现一次。

（C2）子图商图无环：令 $E_S = \{(x_u, x_v): (u, v) \in E_{op}, x_u \neq x_v\}$ ，要求图 $(\{1, \dots, S\}, E_S)$ 无环。

（C3）同核顺序相容：对任意 $(s, s') \in E_S$ ，若 $y_s = y_{s'} = k$ ，则 s 在 π_k 中位于 s' 之前。

（C4）场景 A 的 Task 图无环： E_S 与每个核上相邻子图的顺序边之并无环。

（C5）场景 B 的全局执行图无环：核内数据/内存依赖、流水线发射顺序与跨核 COPY 边之并无环。

（C6）核内容量：任一 Task 执行时 Step2 总能找到换出对象，且任一时刻的驻留量满足

$$\sum_{t \text{ 驻留于 } pos} s_t \leq C_{pos}, pos \in \{L1, UB\} \quad (4-3)$$

（C7）流水线资源：每核每条流水线同时至多一条在飞指令，发射顺序为 Step3 给出的固定序。

（C8）共享 DDR 带宽：任意时刻在途 DDR 搬运的瞬时带宽之和不超过 bw ，按公平共享分配。

（C9）L2 语义（仅问题三）：只读、FIFO、容量 C_{L2} 、命中带宽 bw_{L2} ，与 DDR 带宽池独立。

其中（C6）～（C9）由评估器内部的核内调度与多核模拟保证，方案只能通过子

图规模与顺序间接影响它们；（C1）～（C5）必须由方案本身满足，第 4.6 节给出一种使它们自动成立的构造。

4.4.4. 目标函数与解析代理

上报目标严格取官方评估器给出的 Makespan , Makespan 相同时以总额外搬运量 ΔB 为次序：

$$\min_{x,y,\pi} (\text{MK}(x,y,\pi), \Delta B(x,y,\pi)) \quad (\text{字典序, 均由官方评估器计算}) \quad (4-4)$$

官方评估对大图很慢（单次最长 1747 s），不能放进搜索内层。搜索内部使用一个可增量更新的解析代理：

$$F = \max(\max_k(L_k^M, L_k^V, \frac{B_k^{\text{in}}}{bw}, \frac{B_k^{\text{out}}}{bw}), \frac{B_{\text{DDR}}}{bw}) + \omega R \quad (4-5)$$

式中第一项是每核四条流水线中最忙者的负载与全局带宽下界， R 为块级跨核同步深度，同步权重 ω 在问题一取 $W_{\text{cross}} = 1000$ 、问题二和三取 $\delta = 500$ 。Cube 与 Vector 分开计负载，对应赛题“矩阵单元与向量单元自动并行”的优化方向：两类计算量不相加，而是各自与对方的空闲时间重叠。 B_k^{in} 、 B_k^{out} 按评估器的插入规则逐项计算（式（5-4）与式（6-2）、式（6-3）），移动一个块只需重算该块触及的张量。

场景 B 的切分搬运恰为超图的 $(\lambda-1)$ 割。记张量 t 触及的核集合（生产核与消费核）为 $\mathcal{P}_t \cup \mathcal{C}_t$, $\lambda_t = |\mathcal{P}_t \cup \mathcal{C}_t|$ 。按评估器规则，单生产核的中间张量每多触及一个核就多一次 COPY_OUT 和一次 COPY_IN，图输入张量每多一个消费核就多一次 COPY_IN，于是切分新增的搬运量（不含换入换出）为

$$\Delta B^{\text{cut}} = \sum_t w_t s_t (\lambda_t - 1), w_t = \begin{cases} 2, & t \text{ 为单生产核的中间张量} \\ 1, & t \text{ 为图输入张量} \end{cases} \quad (4-6)$$

图输出张量与多生产核张量按第 6.3 节的逐项规则另计。式（4-6）与超图划分的连通度目标一致，边权取张量字节数^[6]：把强连接张量的读者放在同一核心，就是在最小化这一割值。

4.5. 理论下界与超线性加速的来源

任何可行解都满足

$$\text{MK}_N \geq \text{LB}_N = \max(\frac{\sum_{p_v=M} c_v}{N}, \frac{\sum_{p_v=V} c_v}{N}, \frac{B_{\text{DDR}}^{\text{min}}}{bw}, \text{CP}) \quad (4-7)$$

四项分别来自 Cube 单元、Vector 单元、共享 DDR 带宽（ $B_{\text{DDR}}^{\text{min}}$ 为原图自带的搬运量）和依赖链（关键路径 CP）。第 8.5 节用 MK_N/LB_N 度量最终方案与这一不可达下界的距离。

这个下界只约束分子 MK_N ，并不约束加速比的分母 MK_1 。单核基准把整图作为一个 Task，驻留量远超容量（表 4-1 中 UB 压力的中位数为 14.6 倍），Step2 因而插入大量换入换出；多核切分把 Task 缩小到容量量级后，这部分搬运可以整体消失。于是加速比可以分解为

$$SU_N = \frac{MK_1}{MK_N} = \frac{MK_1}{MK_1^*} \times \frac{MK_1^*}{MK_N} \quad (4-8)$$

其中 MK_1^* 表示同一张图在单核上“不发生换入换出”时的理想时长。前一个因子是单核基准自身的调度损失，可以显著大于 1；后一个因子是真正的并行收益，近似不超过 N 。因此 $SU_N > N$ 与“并行收益不超过核数”并不矛盾。实验中可达加速比上界 MK_1/LB_N 的均值在 $N = 4$ 时为 4.93，高于核数本身。

4.6. 可行性的构造性刻画

直接对 (C2) ~ (C5) 做组合搜索代价极高，每一步移动都要重新判环。本节给出一种构造，使这些约束自动成立。它对应整体同步并行 (BSP) 模型^[13]中的超步：同一层内各核的计算相当于超步内的本地计算，层与层之间的跨核数据交换相当于超步边界上的通信与同步。近年的研究在 BSP 框架下讨论 DAG 调度^[14]，并把通信、同步与内存容量一起纳入调度模型^[15,16]。与经典 BSP 不同，本题的“栅栏”不是全局的：评估器只在某个跨核依赖满足时放行对应的 COPY_IN，所以层数只是同步深度的上界代理。

定义 4-1 (跨核同步层次) 给定核分配 $c: V_{op} \rightarrow \{0, \dots, N-1\}$ ，定义算子 v 的跨核同步层次为沿依赖路径累计的换核次数的最大值：

$$r(v) = \begin{cases} 0, & v \text{ 无前驱} \\ \max_{(u,v) \in E_{op}} (r(u) + \mathbb{1}[c(u) \neq c(v)]), & \text{否则} \end{cases} \quad (4-9)$$

引理 4-1 (可行性) 取子图为等价类 $S(k, \rho) = \{v: c(v) = k, r(v) = \rho\}$ ，按字典序 (ρ, k) 递增编号，每个核的 π_k 取其子图编号的升序，则约束 (C1) ~ (C5) 全部成立。

证明 对任意 $(u, v) \in E_{op}$ ：若 $c(u) = c(v)$ ，则 $r(v) \geq r(u)$ ，两者同属一个等价类，或 v 所在子图的编号更大；若 $c(u) \neq c(v)$ ，则 $r(v) \geq r(u) + 1$ ， v 所在子图的编号严格更大。所以 E_S 的每条边都指向编号更大的子图，商图无环，(C2) 成立；同核情形给出 (C3)；(C4) 新增的同核顺序边同样沿编号递增，仍然无环；(C5) 中的核内依赖与流水线发射顺序都服从核内算子序，而核内算子序沿子图编号递增（事实 2），跨核 COPY 边又沿层次严格递增，并起来仍无回边。(C1) 由构造直接成立。 $\square$

引理 4-2 (同层独立) 同一层次内不存在跨核边。

证明 若 (u, v) 跨核且 $r(u) = r(v) = \rho$ ，由式 (4-9) 得 $r(v) \geq \rho + 1$ ，矛盾。 $\square$

推论 4-3（单调层指派） 若 $l: V_{op} \rightarrow \mathbb{N}$ 对每条边满足 $l(v) \geq l(u) + \mathbb{1}[c(u) \neq c(v)]$ (称 l 单调), 把引理 4-1 中的 r 换成 l , 引理 4-1 与引理 4-2 的结论不变; 并且 r 是所有单调层指派中逐点最小的一个。

证明 两条引理的证明只用到了这一不等式。逐点最小性沿拓扑序归纳：无前驱时 $r(v) = 0 \leq l(v)$ ；否则 $l(v) \geq \max_u(l(u) + \mathbb{1}[c(u) \neq c(v)]) \geq \max_u(r(u) + \mathbb{1}[c(u) \neq c(v)]) = r(v)$ 。□

图 4-4 用一个 8 节点的例子说明这一构造：(a) 原始算子级 DAG；(b) 亲和原子化后按线性扩展 σ 的连续区间成块，块级商图必为 DAG；(c) 分核后按同步层次分层，子图恰是“列 = 层次、行 = 核心”网格中的一格，所有边都指向同一格或右侧的格子，红色虚线的跨核边必然跨到下一层。

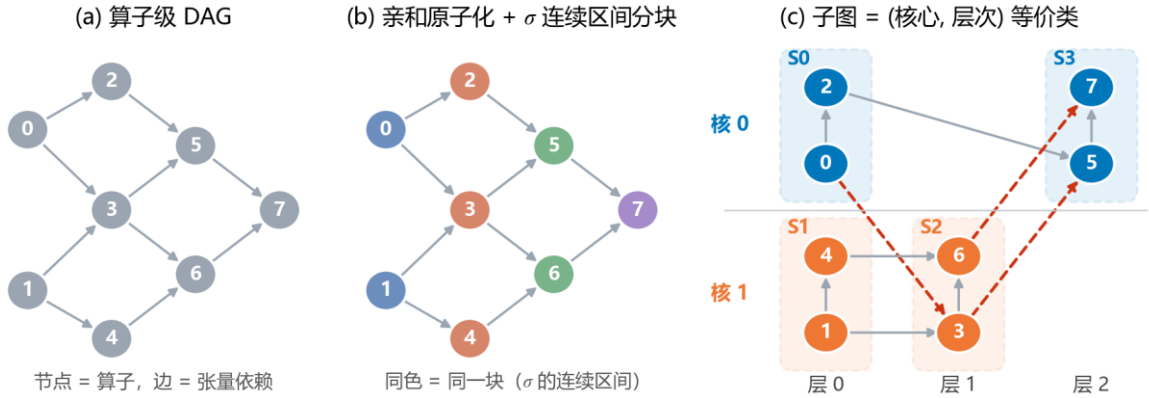


图 4-4 切图、分核与同步层次分层示意

引理 4-2 的调度含义是：同一层的各核子图互不依赖，可以不经同步地并行；跨核同步只发生在层与层之间，至多 $R = \max_v r(v)$ 次。结合事实 3，这一构造还有一个直接的好处：依赖远端数据的算子被自动推到更高的层，核心先跑完本层全部本地就绪的工作，再去等远端数据，从而避开“前部跨核 COPY_IN 让整核停摆”的情形（图 4-1b）。第 8.3 节的消融实验显示，这是贡献最大的单一机制。

推论 4-3 给出了层指派的自由度： r 把每个算子放在最早的合法层；把算子在合法区间内往后移到本核负载最轻的层（ALAP 填充），或把总周期过小的层并入前一层（层压缩），得到的仍是单调指派，可行性无需重新证明。实现中，每次按显式层指派成形子图前都用 `levels.check_monotone` 复核一次单调性。

4.7. 本章小结

本章从评估器的实现中读出了五条决定模型形态的事实，其中“全局申请序导致整核停摆”经官方 `trace` 证实；建立了切图、分核、排序三项决策的统一优化模型，给出了理论下界并解释了超线性加速的来源；最后以跨核同步层次构造子图，证明全部硬约束自动成立（引理 4-1），并且同层子图互不依赖（引理 4-2）。这使后续算法的搜索只需在“分核”这一维上进行，可行性不必在搜索中反复检查。

第5章 问题一模型的建立与求解

5.1. 问题分析

问题一处在场景 A：每个子图单独构成一个 Task，同核 Task 串行执行且不保留缓存数据，所有跨子图数据都经 DDR 中转。与场景 B 相比，场景 A 有三个特点决定了模型的侧重点。第一，**每条子图边界都有真实的搬运代价**，切图越细，边界上的 COPY_OUT/COPY_IN 越多，它们还要与其他核争用 DDR 带宽。第二，**Task 的激活是“与”关系**：一个 Task 必须等其全部前驱 Task 完成且等待期满后才能开始，跨核前驱还要多等 1000 cycle，因此跨核依赖链的级数会串行累加到 Makespan 上。第三，**Task 太大同样有害**：单个 Task 的驻留量超过 L1/UB 容量时，核内 Step2 会插入换入换出。于是问题一的核心是“粒度”与“跨核依赖级数”两件事，赛题还要求算法在 2~5 核范围内稳定、高效（以单个用例 5~10 分钟内给出结果为佳）。

5.2. 求解思路

问题一沿用第 4 章的统一模型与可行性构造，求解流程如图 5-1 所示：先建立 Task 级时序模型，用它分析粒度权衡并作为退火的适应度；再用 CAP-LS 生成 14 组候选，由官方评估器择优；最后以代理驱动的模拟退火进一步调整 Task 的粗细，并由保底池汇总。

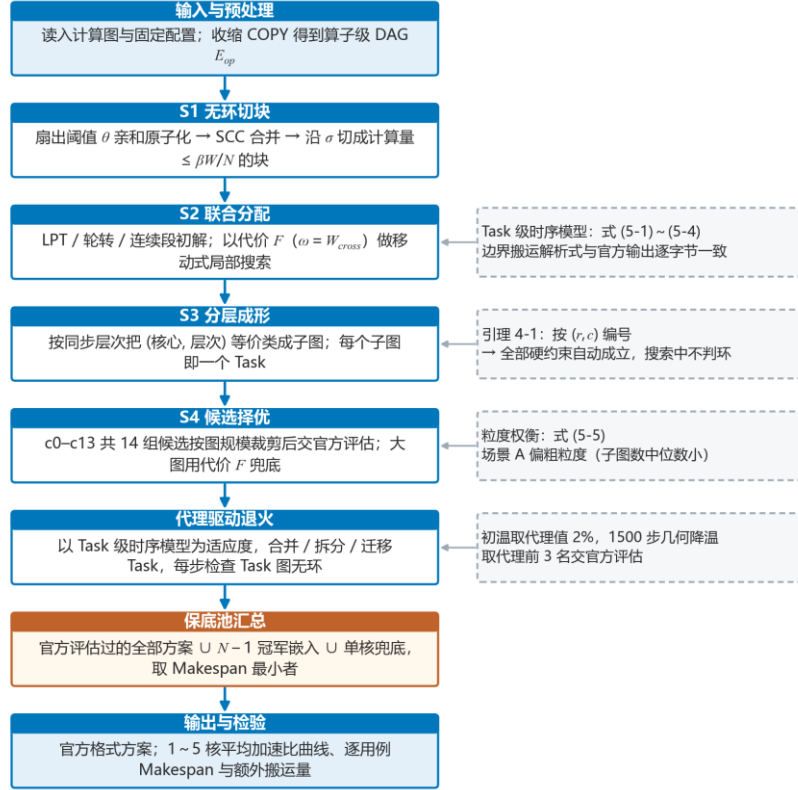


图 5-1 问题一求解流程图

5.3. 场景 A 的 Task 级时序模型

场景 A 中一个子图就是一个 Task。评估器 `evaluate_scene_a` 中 Task 的释放规则是：同核上一个 Task 结束后再等 $W_{same} = 100$ cycle，每个跨核前驱结束后再等 $W_{cross} = 1000$ cycle，实际激活时刻取全部约束的最大值（图 5-2）。记子图 s 的开始、结束时刻为 $start(s)$ 、 $end(s)$ ， $prevEnd(y_s)$ 为核心 y_s 上前一个 Task 的结束时刻，则

$$start(s) = \max(prevEnd(y_s) + W_{same}, \max_{s' \in pred(s), y_{s'} \neq y_s} (end(s') + W_{cross})) \quad (5-1)$$

$$end(s) = start(s) + dur(s), MK_A = \max_s end(s) \quad (5-2)$$

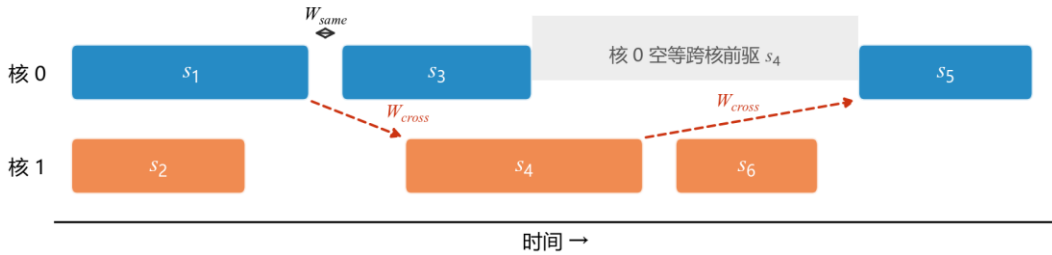


图 5-2 场景 A 的 Task 时序：同核切换等待 W_{same} ，跨核前驱等待 W_{cross}

同核前驱由核内顺序（C3）保证先完成，所以式（5-1）中不再单列。Task 内部由核内 Step1~Step3 调度，时长没有闭式表达；搜索中用四条流水线的并行上界近似，两类 COPY 共用 DDR 带宽：

$$\text{dur}(s) \approx \max(M_s, V_s, \frac{B_s^{\text{in}} + B_s^{\text{out}}}{bw}), M_s = \sum_{v \in s, p_v = M} c_v \quad (5-3)$$

V_s 同理。场景 A 在每条子图边界都重建 COPY（事实 1），子图 s 的边界搬入、搬出字节为

$$B_s^{\text{in}} = \sum_{t: C_t \cap s \neq \emptyset, P_t \cap s = \emptyset} s_t, B_s^{\text{out}} = \sum_{t: P_t \cap s \neq \emptyset, (t \in T_{\text{out}} \vee C_t \not\subseteq s)} s_t \quad (5-4)$$

式（5-4）逐条对应评估器的插入规则，没有任何拟合参数。我们在 30 个用例、三个问题共 90 组方案上，把式（5-4）与第 6 章式（6-2）、式（6-3）的预测值和官方输出的边界搬运量逐一比较，最大相对误差为 0.0000%，即搬运量模型与评估器逐字节一致。式（5-1）~式（5-3）则是时序代理：它忽略了 Task 内部的换入换出与 DDR 争用，只用于搜索内部排序，所有上报数字仍取官方评估。

5.4. 切图粒度的权衡

把某核心上的算子切成 q 个子图，该核的完成时间近似为

$$T_k(q) \approx \max(M_k, V_k) + (q - 1)W_{\text{same}} + \frac{\Delta B(q)}{bw} - \Phi(q) \quad (5-5)$$

第一项与 q 无关；第二项是 Task 切换开销；第三项是新增的边界搬运， $\Delta B(q)$ 随 q 单调不减；第四项 $\Phi(q)$ 是子图变小后 Step2 换入换出的减少量，同样随 q 单调不减。 q 太小时换入换出多， q 太大时边界搬运和切换开销多，中间存在最优粒度 q^* 。同核 Task 之间没有流水线重叠，把若干独立子图合成一个 Task，既能让头部 COPY_IN 与尾部 COPY_OUT 被计算掩盖，也免去了子图之间经 DDR 的往返，所以场景 A 的最优粒度偏粗。

实验结果与这一分析一致：问题一最终方案的子图数中位数为 6（ $N = 4$ ），远少于问题二的 672.5；在 29 个用例的粒度扫描中，问题一在粒度参数 $\beta = 2$ 处最好，最细的 $\beta = 0.03$ 时损失 14.2%（第 8.4 节）。 q^* 依赖图的结构，本文不解析求解，而是在 S4 用候选网格覆盖不同粒度，由官方评估器择优。

5.5. CAP-LS 算法设计

CAP-LS（Communication-Aware Partitioning with Level Stratification）由四个阶段组成，整体框架见图 5-3。三个问题共用这一流程，场景差异只落在代价权重、候选集合与 L2 折算三处；本节以问题一为例说明各阶段，问题二、三的调整见第 6、7 章。

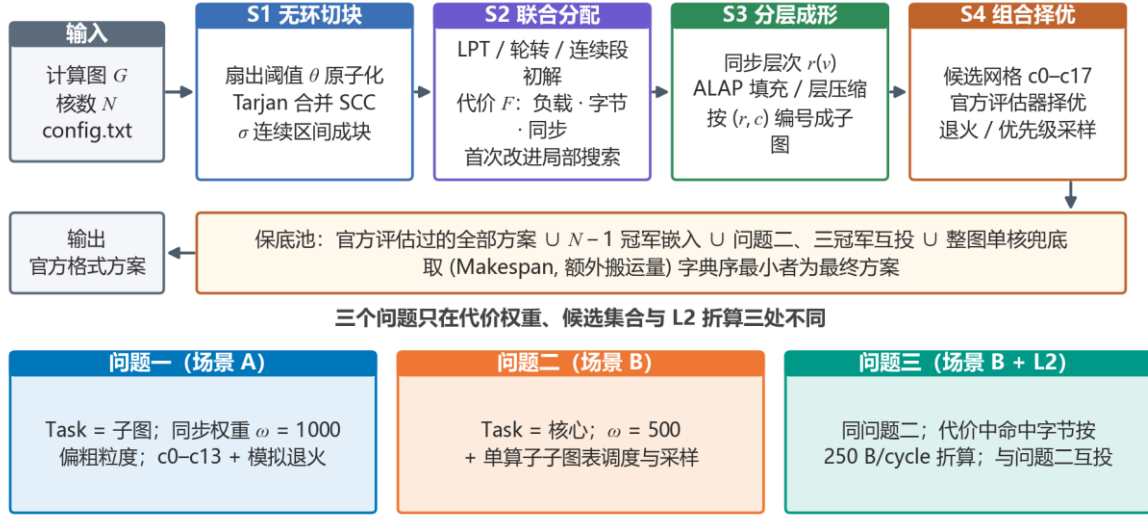


图 5-3 CAP-LS 求解框架

5.5.1. S1: 亲和原子化与无环切块

S1 的目标是在不破坏无环性的前提下，把强通信的算子绑在一起。以张量扇出区分两类连接：若 $|C_t| \leq \theta$ ，称张量 t 为强连接，把 $P_t \cup C_t$ 并入同一原子；若 $|C_t| > \theta$ ，称其为广播连接，允许被多个原子重复读取，不做绑定。阈值 θ 在 $\{\infty, 16, 8, 4, 2, 1\}$ 中从大到小取第一个使“最大原子的计算量不超过 $\beta W/N$ ”的值，其中 $W = \sum_v c_v$ ， β 为粒度参数。收缩后的原子图可能有环，用 Tarjan 算法^[17]求强连通分量并合并，得到原子级 DAG。按原子的拓扑序拼接、原子内部采用与官方 Step1 同类的逆向 DFS 序，得到全局线性扩展 σ ；计算量仍超过上限的原子沿 σ 切成等负载的连续段。每个块都是 σ 的连续区间，所以块级商图必然无环。

5.5.2. S2: 负载、通信与同步深度联合分配

初始分配有三种：最大优先（LPT）装箱^[18]、沿 σ 轮转、沿 σ 连续等分。LPT 装入块 b 时核心 k 的代价为

$$\Delta(k) = \max(L_k^M + M_b, L_k^V + V_b) + w_{\text{comm}} \sum_{j: c_j \neq k} \text{bytes}(b, j) \quad (5-6)$$

$\max(\cdot)$ 项分别累计 Cube 与 Vector 负载，让两类计算在同一核上互补。随后做移动式局部搜索：每次把一个块移到另一个核心，若代理目标 F （式（4-5））下降则接受（首次改进）。问题一中 F 的同步项权重取 $W_{\text{cross}} = 1000$ 。为控制规模，只考察计算量最大的 260 个块，移动次数上限为 $\min(\max(300, 8m'), 2200)$ （ m' 为考察块数），至多 6 轮。一次移动只重算该块触及张量的搬运贡献和块级同步深度，代价为 $O(T_b + m + e_B)$ ，其中 T_b 为块 b 触及的张量数， m 、 e_B 为块数与块间边数。

5.5.3. S3: 同步层次分层

由块的分配得到 $c(v)$ ，按式 (4-9) 一次拓扑扫描算出 $r(v)$ ，取子图为 (核心, 层次) 等价类并按 (r, c) 编号。候选中还可以换用推论 4-3 允许的两种单调层指派：ALAP 填充与层压缩；也可以限定每个子图至多 $\max_ops$ 个算子，沿 σ 再切片，对应式 (5-5) 中的粒度 q 。由引理 4-1 与推论 4-3，所得方案自动满足 (C1) ~ (C5)。

5.5.4. S4: 多起点组合与保底池

S1~S3 的结果取决于粒度 β 、初始分配、子图成形方式与层指派。问题一使用 14 组候选 (表 5-1 的 c0~c13)，问题二、三再加 4 组单算子子图候选 (c14~c17，第 6.4 节)，共 18 组，候选全部交官方评估器打分。候选之间常产生相同的方案，配合按方案哈希的磁盘缓存，同一方案只评估一次。

表 5-1 S4 的候选参数网格

编号	候选设置	设计意图
c0	$\beta = 0.35$, LPT, 按层次成子图	默认配置
c1 / c7 / c8	$\beta = 0.15 / 0.08 / 0.005$	逐级变细, 覆盖“深链 + 层内并行”结构
c2	$\beta = 0.35$, 轮转分配	沿 σ 均匀铺开
c3	$\beta = 0.35$, 连续段分配	链状图: 层数降为 $O(N)$
c4 / c5	以块为子图 (LPT / 轮转)	保留 σ 顺序的细粒度
c6	$\beta = 0.60$, 通信加权 LPT	粗块、少通信
c9	子图算子数上限 (问题一 2000, 问题二、三 500)	限制单个子图规模
c10	只用 $N - 1$ 个核心	通信受限时降低并行度
c11	问题一只用 $\lfloor N/2 \rfloor$ 个核; 问题二、三同步权重取 0	降低并行度 / 去掉同步项
c12 / c13	轮转 + ALAP 填充 / 再做层压缩	推论 4-3 的其他单调层指派
c14~c17	单算子子图 + 容量感知表调度 (仅问题二、三)	直接控制核内驻留

网格之外, S4 还有两条补充: 问题一从网格冠军出发做代理驱动的模拟退火 (第 5.6 节), 问题二、三对单算子子图候选做优先级扰动采样 (第 6.4 节)。最后由保底池汇总: 对每个 (用例, 问题, N), 把各实验阶段中经官方评估的全部方案、同问题 $N - 1$ 冠军追加一个空核后的嵌入方案, 以及问题二、三冠军方案的互投放在一起, 取 (Makespan, 额外搬运量) 字典序最小者作为最终方案; 若最优加速比仍小于 1, 再评估整图单核方案兜底。嵌入方案保证逐用例满足 $MK_N \leq MK_{N-1}$, 最终 1200 个方案中违反这一单调性的次数为 0。CAP-LS 的主流程见算法 5-1。

算法 5-1 CAP-LS (三个问题共用)

输入: 计算图 G , 核心数 N , 问题编号 $P \in \{1, 2, 3\}$, 候选网格 θ_P ; 输出: node_to_subgraph, core_schedules

1. 收缩 COPY 节点, 得到算子级依赖 E_{op} (式 (4-1)) ;
 2. **for** 每组候选参数 $(\beta, \text{init}, \text{mode}, \text{level}, \dots) \in \Theta_p$ **do**
 3. **for** $\theta \in (\infty, 16, 8, 4, 2, 1)$: 亲和原子化; **if** 最大原子计算量 $\leq \beta W / N$ **then break**;
 4. 合并原子图的强连通分量, 按拓扑序拼接各原子的逆向 DFS 序, 得到线性扩展 σ ;
 5. 沿 σ 切开计算量超限的原子, 得到块 $b_1, \dots, b_m$;
 6. 初始分配 $c^{(0)} \leftarrow \text{LPT} / \text{轮转} / \text{连续段}$;
 7. **repeat** 在计算量最大的 260 个块中逐个尝试换核, F 下降即接受 **until** 一轮无改进、满 6 轮或达到移动预算;
 8. 计算单调层指派 l (默认 $l = r$), 子图 $\leftarrow$ (核心, 层) 等价类, 按 (l, c) 递增编号;
 9. 调用官方评估器得到 Makespan (方案哈希命中缓存则跳过);
 10. **end for**; 从网格冠军出发做补充搜索 (问题一: 模拟退火; 问题二、三: 优先级采样);
 11. 保底池汇总全部官方评估过的方案、 $N - 1$ 嵌入与跨问题互投, 返回 (Makespan, 额外搬运量) 最小者。
-

步骤 3~5 为 S1, 步骤 6~7 为 S2, 步骤 8 为 S3, 步骤 2、9~11 为 S4。

5.6. 高效求解策略

5.6.1. 减少官方评估的四项措施

赛题要求问题一的算法“高效”。本题中算法本身很快, 耗时主要花在官方评估上: 单候选 S1~S3 的算法时间中位数只有 0.23 s (共 22,418 次构造), 而官方问题一评估器单次最长需要 1747 s。因此“高效”的关键是少调用评估器, 本文采取了四项措施:

(1) **搜索内层只用解析代价。** 局部搜索中的 F 可增量更新, 不调用评估器; 可行性由引理 4-1 保证, 搜索中不判环。

(2) **方案哈希缓存。** 评估结果按方案的规范化哈希写入磁盘缓存, 不同候选、不同实验阶段产生的相同方案只评估一次, 整个项目在固定配置下去重后共 31,833 次官方评估。

(3) **按规模裁剪候选。** 问题一中 $|V_{op}| \leq 6000$ 的 71 个用例评估全部 14 组候选; $6000 < |V_{op}| \leq 12000$ 的 12 个用例只评估前 8 组; $|V_{op}| > 12000$ 的 17 个用例只评估前 4 组。

(4) **大图代理兜底。** 官方问题一评估中位耗时超过 600 s 的 4 个用例 (case_014、case_072、case_076、case_091) 即使只评估 4 组候选也会远超 10 分钟。这 4 个用例改用局部搜索结束时的代价 F 在候选之间排序, 只把第一名交官方评估。其上报成绩仍来自官方评估, 但候选之间的选择依据变成了代理, 可能选不到官方意义上最好的候选。这一代价的量级可以参考第 8.6 节的实验: 在全部 100 个用例上改用代理在主实验候选 (网格与退火) 中选择, 问题一 $N = 4$ 的平均加速比由官方择优的 3.14

降到 3.07 (−2.4%)。就这 4 个用例本身而言，最终方案在 $N = 4$ 的加速比为 3.90~4.03，已接近线性，兜底造成的损失有限。

5.6.2. 代理驱动的模拟退火

同步层次构造偏保守：它把 Task 边界固定在层与层之间。为了在问题一中进一步调整 Task 的粗细，我们从网格冠军出发做模拟退火^[19]。适应度取式 (5-1)~式 (5-3) 的 Task 级时序模型，用增量状态维护，单步代价与图规模无关。邻域有三种：合并同核相邻的两个 Task（二者之间不存在经其他 Task 的路径时才允许）、把一个 Task 按拓扑序对半拆开、把一个 Task 移到另一个核心并按拓扑位置插入。退火会打破层次结构，所以每一步都检查 Task 图（依赖边与同核顺序边之并）是否无环，不可行的移动直接撤销。初温取初始代理值的 2%，按几何速率在 1500 步内降到初温的 10^{-3} ，记录代理最好的 3 个不同方案交官方评估。全部 400 个问题一配置中，退火耗时中位数为 4.9 s；退火方案在主实验中的 137 个配置上胜过全部网格候选，经保底池汇总后仍有 81 个最终方案来自退火。

5.7. 复杂度分析

设 $n = |V_{op}|$ ， $e = |E_{op}|$ ， τ 为张量数， m 为块数， e_B 为块间边数， B 为移动次数上限，各阶段的复杂度见表 5-2。

表 5-2 CAP-LS 各阶段的时间复杂度（单个候选）

阶段	复杂度	说明
读图与 COPY 收缩	$O(n + e + \tau)$	一次性，结果缓存
S1 亲和原子化	$O((n + \tau)\alpha(n))$	并查集，至多 6 个 θ 值
S1 SCC 合并与线性扩展	$O(n + e)$	迭代 Tarjan、迭代逆向 DFS
S2 LPT 装箱	$O(m \log m + mN)$	块按计算量排序
S2 局部搜索	$O(B(\bar{T} + m + e_B))$	$B \leq 2200$ ， $\bar{T}$ 为块平均触及张量数
S3 分层与子图成形	$O(n + e)$	一次拓扑扫描
合计	$O(n + e + \tau + B(\bar{T} + m + e_B))$	对 n 近似线性

整个流程不对“切图 × 分核 × 排序”做枚举：切图由 σ 的连续区间构造，排序由引理 4-1 直接给出，只有分核进入搜索，而且搜索是增量的。作为对照，朴素枚举的规模为 $\mathcal{B}_n \cdot N^S \cdot \prod_k |\pi_k|!$ ($\mathcal{B}_n$ 为 Bell 数)，在最小的 552 个算子的用例上已无法计算。

5.8. 求解结果与分析

5.8.1. 平均加速比

问题一在 1~5 核下的平均加速比如图 5-4 所示，统计量见表 5-3，逐用例的 Makespan 与总额外搬运量见附录 A。 $N = 1$ 时方案为整图一个子图，100 个用例在问题一评估器下的 Makespan 都与官方单核基准相等，因此 $N = 1$ 的加速比恒为 1。

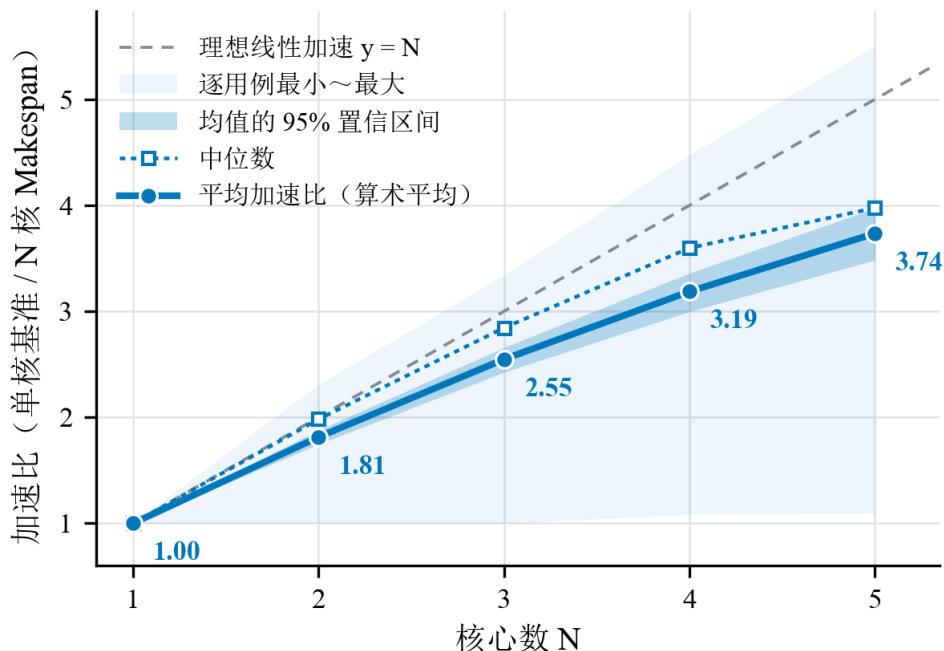


图 5-4 问题一在 1~5 核下的平均加速比（阴影为 bootstrap 95% 置信区间与逐用例范围）

表 5-3 问题一的加速比统计（100 个用例）

核心数	$N = 2$	$N = 3$	$N = 4$	$N = 5$
平均加速比（算术平均）	1.81	2.55	3.19	3.74
95% 置信区间	[1.75, 1.88]	[2.43, 2.66]	[3.00, 3.36]	[3.48, 3.97]
几何平均（补充）	1.78	2.46	3.02	3.48
中位数	1.98	2.84	3.60	3.98
最小值 / 最大值	1.00 / 2.31	1.00 / 3.34	1.08 / 4.48	1.10 / 5.50
超线性用例数（加速比 > N ）	31	13	11	7
并行效率（均值 / N ）	91%	85%	80%	75%

从图 5-4 与表 5-3 可以得到四点结论。第一，平均加速比随核数稳定上升， $N = 4$ 时为 3.19，中位数 3.60 明显高于均值，说明少数结构受限的难例拉低了平均值； $N = 4$ 时加速比最低的用例为 case_024（1.08）、case_064（1.08）、case_048（1.10）、case_016（1.12）、case_051（1.14）、case_069（1.34），它们都属于“单一主分量”或长同步链结构（第 8.7 节）。第二，每个核数下的最小加速比都不低于 1，且逐用例随核数单调不降，这来自保底池的 $N - 1$ 嵌入与单核兜底。第三，出现了超线性加速： $N = 2$ 时 31 个用例超过 2。按式 (4-8)，超线性来自单核基准自身的换入换出，而非多核“做得比理想还好”。第四， $N = 4$ 时 100 个用例共新增 DDR 搬运 $1.70 \times$

10⁹ 字节，其中切分边界占 58.7%，其余来自核内换入换出；场景 A 的每条子图边界都要经 DDR 往返，边界搬运的占比高于问题二（53.9%）。

图 5-5 进一步给出三个问题逐用例加速比的分布。问题一的箱体明显低于问题二、三，且下须更长，说明场景 A 的难例更难；三个问题的超线性用例（红点）都集中在分量碎片化、单核基准有换入换出的用例上。

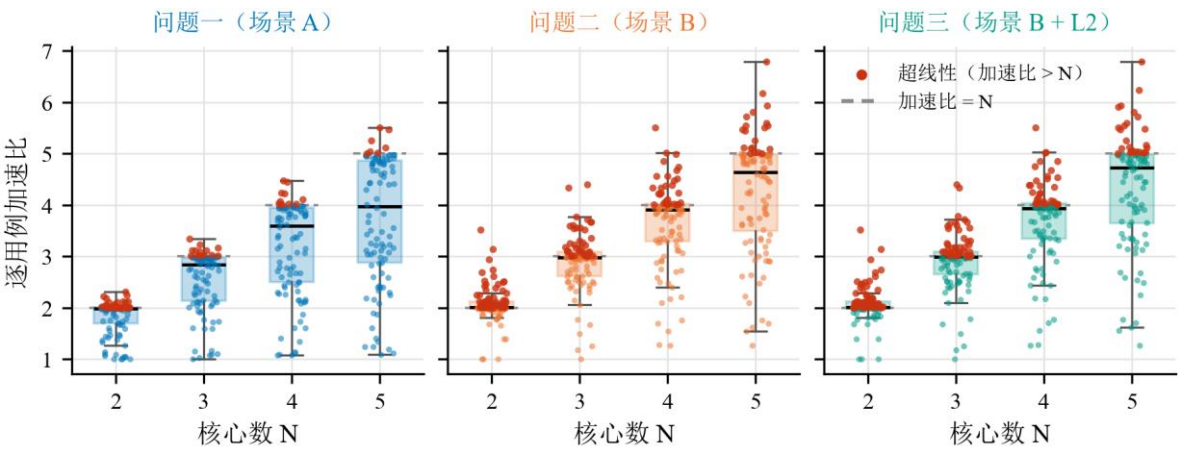


图 5-5 三个问题逐用例加速比的分布（红点为超线性用例，虚线为加速比 = N）

5.8.2. 求解时间

表 5-4 统计每个 (用例, N) 配置从构造候选到选出冠军的耗时，“端到端”按串行累加全部候选的构造时间、官方评估时间与退火时间，“不含评估”只计构造与退火；保底池汇总与其他实验阶段属于实验而非求解流程，不计入。

表 5-4 单个 (用例, N) 配置的求解时间（每个问题 400 个配置）

问题	端到端中位数 / s	端到端 90% 分位 / s	≤ 5 min 的配置	≤ 10 min 的配置	不含评估的中位数 / s	不含评估 ≤ 10 min
问题一	45.8	756	339	351	15.4	385
问题二	30.9	237	378	400	7.1	400
问题三	31.2	244	376	400	7.3	400

对照 5~10 分钟的目标，问题一 400 个配置中 339 个在 5 分钟内、351 个在 10 分钟内完成，超过 10 分钟的配置分布在 14 个用例上（case_014、025、028、030、041、058、062、072、076、079、085、087、091、092），主要是官方评估本身耗时很长的大图；问题二、三的全部配置都在 10 分钟内完成，最长分别为 510s 与 528s。图 5-6b 显示单候选的算法时间随图规模近似线性增长，与表 5-2 的分析一致。可以看出，超出 10 分钟的时间主要花在官方评估器上：不计评估时，问题一有 385/400 个配置在 10 分钟内完成。若比赛现场对大图有硬性时限，可以关闭大图的退火，只保留代理兜底。

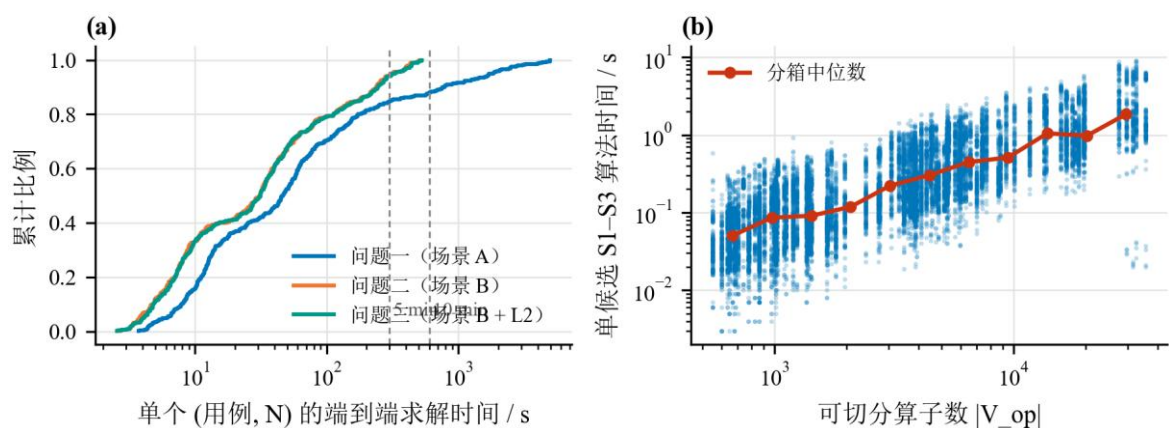


图 5-6 求解时间: (a) 单个 (用例, N) 的端到端耗时累计分布; (b) 单候选 S1~S3 算法时间随图规模的变化

5.9. 本章小结

针对问题一, 本章建立了场景 A 的 Task 级时序模型与边界搬运模型, 后者与官方输出逐字节一致; 由粒度权衡式 (5-5) 得出场景 A 偏粗粒度的结论, 并被最终方案的子图数与粒度扫描所证实。CAP-LS 以亲和原子化、联合分配与同步层次分层生成可行候选, 以官方评估择优、代理驱动退火和保底池得到最终方案。在全部 100 个用例上, $N = 2 \sim 5$ 的平均加速比为 1.81、2.55、3.19、3.74, 逐用例加速比随核数单调不降, 单个配置的端到端求解时间中位数为 45.8 s。

第6章 问题二模型的建立与求解

6.1. 问题分析

问题二处在场景 B：同一核心的全部子图合并为一个 Task，同核子图之间直接复用 L1/UB 中的数据；跨核依赖由源核 COPY_OUT、500 cycle 同步延迟与目标核 COPY_IN 完成。与问题一相比，场景 B 带来三处本质变化。

第一，**切图的作用变了**。同核子图边界不再重建搬运（事实 1），切图在核内退化为排序决策：子图的名次就是核内算子的执行顺序（事实 2）。粒度可以细到上千个子图而不增加 DDR 流量，跨核搬运量只取决于分核。

第二，**跨核依赖的真实代价变了**。名义延迟从 1000 cycle 降到 500 cycle，但全局申请序使未就绪的跨核 COPY_IN 可能堵住整个核心（事实 3，图 4-2），因此“把依赖远端数据的算子往后放”比问题一更关键。

第三，**出现了驻留与顺序的权衡**。同一 Task 内所有子图共享 L1/UB，核内顺序决定了同一时刻有哪些张量驻留在片上；顺序不当会引发大量换入换出，赛题还要求“严格满足 L1/UB 容量约束”。

因此问题二是一个两层问题：分核决定跨核搬运与同步深度，核内顺序决定驻留与流水线重叠。本章在 CAP-LS 框架上做五处调整，重点是新增一类直接控制核内驻留的“单算子子图 + 容量感知表调度”候选。

6.2. 求解思路

问题二的求解流程如图 6-1 所示。分核仍由 CAP-LS 的 S1~S2 给出（同步权重取 $\delta = 500$ ）；子图成形有两种方式：按（核心，层次）等价类成子图，或每个算子单独成子图、核内顺序由容量感知表调度决定；两类候选共 18 组全部交官方评估，再对最好的表调度候选做优先级采样，最后由保底池与跨问题互投得到最终方案。



图 6-1 问题二求解流程图

6.3. 场景 B 的执行模型与搬运量公式

核心 k 的完成时间受四条流水线最大占用的限制，跨核同步则沿层次累加，Makespan 近似满足

$$T_k = \max(M_k, V_k, \frac{B_k^{\text{in}}}{bw}, \frac{B_k^{\text{out}}}{bw}), MK_B \gtrsim \max(\max_k T_k, \frac{B_{\text{DDR}}}{bw}) + \delta R \quad (6-1)$$

记 $\mathcal{P}_t = \{c(u): u \in P_t\}$ 、 $\mathcal{C}_t = \{c(u): u \in C_t\}$ 为张量 t 的生产核集合与消费核集合。评估器对每一对（生产核，消费核）插入一对 COPY，于是核心 k 的搬入、搬出字节为

$$B_k^{\text{in}} = \sum_{t: \mathcal{P}_t = \emptyset, k \in \mathcal{C}_t} s_t + \sum_{t: k \in \mathcal{C}_t} |\mathcal{P}_t \setminus \{k\}| s_t \quad (6-2)$$

$$B_k^{\text{out}} = \sum_{t: (t \in T_{\text{out}} \vee \mathcal{C}_t = \emptyset), k \in \mathcal{P}_t} s_t + \sum_{t: k \in \mathcal{P}_t} |\mathcal{C}_t \setminus \{k\}| s_t \quad (6-3)$$

由此，被 λ_t 个核心触及的单生产核中间张量产生 $2s_t(\lambda_t - 1)$ 字节额外搬运，图输入张量产生 $s_t(\lambda_t - 1)$ 字节，正是式 (4-6) 的 $(\lambda - 1)$ 超图割。把强连接张量的读者放在同一核心就是在最小化这一割值，S1 的亲 and 原子化以此为目标。式 (5-4) 与式 (6-2)、式 (6-3) 一起构成搬运量解析模型，第 5.3 节已验证它与官方输出逐字节一致。

6.4. 算法改进

6.4.1. 粒度两端覆盖与同步权重

同核边界零搬运，粒度可以细到上千个子图而不增加 DDR 流量。候选网格因此同时覆盖“每核约一个子图”（ $\beta = 0.6$ ）到“几乎一块一个子图”（ $\beta = 0.005$ 、以块为子图）的两端。代理 F 的同步项权重取 $\delta = 500$ ；由于事实 3，这个权重只是真实代价的下限，处理跨核等待的主要手段仍是同步层次分层。

6.4.2. 单算子子图与容量感知表调度

层次分层只决定“哪些算子在同一层”，层内顺序沿用线性扩展 σ ，没有考虑片上驻留：按 σ 的顺序往往让一个大张量的全部消费者执行完才释放它，驻留峰值高。当某核某层的驻留量超过 $L1/UB$ 容量时，评估器 Step2 会插入换入换出。为此增加一类候选：分核仍由 $S1 \sim S2$ 给出，但每个算子单独成为一个子图，核内顺序由一个容量感知的表调度决定（算法 6-1）。

优先级取 HEFT^[20] 式的向上秩 $\text{rank}_u(v) = c_v + \max_s(\text{comm}(v, s) + \text{rank}_u(s))$ ，跨核边的 comm 取 δ 加字节数除以带宽，并归一化到 $[0, 1]$ 。每个核从就绪列表中秩最高的 64 个算子里挑选，评分为

$$\text{score}(v) = \text{rank}_u(v) - \mu \frac{\Delta \text{live}(v)}{C} + v \mathbb{1}[p_v \neq p_k^{\text{busy}}] + \eta g_v \quad (6-4)$$

其中 $\Delta \text{live}(v)$ 是执行 v 后片上驻留量的净变化（新申请的输出与未驻留的输入，减去最后一次使用后释放的张量）， C 取对应缓冲区的容量； p_k^{busy} 是核 k 上最晚空闲的流水线，第三项鼓励交替使用不同流水线，让 Cube 与 Vector 重叠； g_v 为 Gumbel 噪声，只在采样时开启（ $\eta = 0.05$ ）。执行 v 会让 $L1$ 或 UB 驻留量超过 αC 的候选不参与评分，全部超限时退而取 Δlive 最小者。

算法 6-1 容量感知表调度（单算子子图候选）

输入：算子级 DAG，分核 $c(v)$ ，层指派 $l(v)$ ，参数 α, μ, ν, η ；输出：各核的算子序 π_k

1. 计算归一化向上秩 rank_u ；各核就绪列表按秩降序初始化为无前驱的算子；
 2. **while** 仍有未调度的算子 **do**
 3. 对每个核 k ：在就绪列表前 64 个算子中剔除会使 $L1/UB$ 驻留超过 αC 者，按式（6-4）取最高分；若全部超限，取 Δlive 最小者；
 4. 在各核候选中取可开始时刻最早者 (k, v) ，提交到 π_k ，更新流水线空闲时刻与驻留量；
 5. 对 v 的每个后继 s 更新就绪时刻（跨核再加 δ 与传输时间），入度归零时加入所在核的就绪列表；
 6. **end while**
 7. 各 π_k 按层 l 做稳定排序，子图编号按（层，核，核内位置）递增分配。
-

第 7 步把每核序列按层次做稳定排序，同层同核的算子只是把（核心，层次）等

价类进一步细分，其顺序是表调度给出的拓扑序，所以引理 4-1 的结论保持不变，方案仍自动可行。网格中的 $c_{14} \sim c_{17}$ 取 $(\alpha, \mu, \nu) = (0.8, 1, 0.5)$ 、 $(1.0, 1, 0.5)$ 、 $(0.8, 0, 0)$ ，以及 $\beta = 0.15$ 下的 $(0.8, 1, 0.5)$ 。

表调度共产生 3200/3200 个可行的单算子子图候选（问题二、三各 100 个用例 $\times 4$ 个核数 $\times 4$ 组参数），在主实验中成为 229 个问题二、三配置的冠军。加入它之后，问题二 $N = 4$ 冠军方案的换入换出总量由 9.63×10^8 字节降到 3.43×10^8 字节，减少 64%。式 (6-4) 的两个附加项都有贡献：以完整表调度为基准做配对消融（ $N = 2 \sim 5$ ），去掉驻留项（ $\mu = 0$ ）后问题二、三的平均加速比分别变化 -1.1%、-1.1%，去掉流水线交错项（ $\nu = 0$ ）后变化 -2.5%、-2.6%，Holm 校正后均显著；把容量余量放宽到 $\alpha = 1$ 只变化 -0.7%、-0.6%，影响很小。网格中两项都开启的 c_{14} 与两项都关闭的 c_{16} 相比，在 800 组配对中平均高 +3.2%（胜/平/负 526/169/105，Wilcoxon $p = 6.0 \times 10^{-69}$ ）。

6.4.3. 优先级采样与跨问题互投

表调度对优先级的细小差别很敏感。对每个配置中最好的单算子子图候选，用 16 个随机种子开启 Gumbel 扰动，得到 16 个方案，用算子级事件模拟代理（只用于排序）给它们排序，把前 3 个交官方评估，共新增 2,400 次官方评估。与不采样相比，问题二 $N = 4$ 的主实验平均加速比由 3.54 升到 3.58，问题三由 3.63 升到 3.65，采样方案在 165 个配置上成为主实验冠军。

问题二、三的约束完全相同，只差 L2。保底池把问题三的冠军方案也放到问题二的评估器下比较（反之亦然），最终 400 个问题二方案中有 82 个来自问题三的候选。

6.5. 粒度、驻留与通信的耦合

场景 B 把问题一中“边界搬运对粒度”的权衡换成了“驻留对顺序”的权衡。子图切得再细也不增加 DDR 流量，但核内顺序决定了驻留峰值：按 σ 顺序执行往往驻留峰值高，按表调度交错执行则峰值低、换入换出少。另一方面，跨核割值只取决于分核，与粒度和顺序无关。因此问题二的求解可以分成两层：分核决定跨核搬运和同步深度，层次与表调度决定驻留和流水线重叠。粒度扫描（第 8.4 节）显示问题二的最优粒度参数为 $\beta = 0.5$ ，在 $\beta \in [0.12, 0.5]$ 内平均加速比的波动为 3.3%，比问题一的 6.9% 平缓，印证了粒度在场景 B 中不再是主要矛盾。

6.6. 求解结果与分析

6.6.1. 平均加速比

问题二在 1~5 核下的平均加速比如图 6-2 所示，统计量见表 6-1，逐用例结果见附录 A。

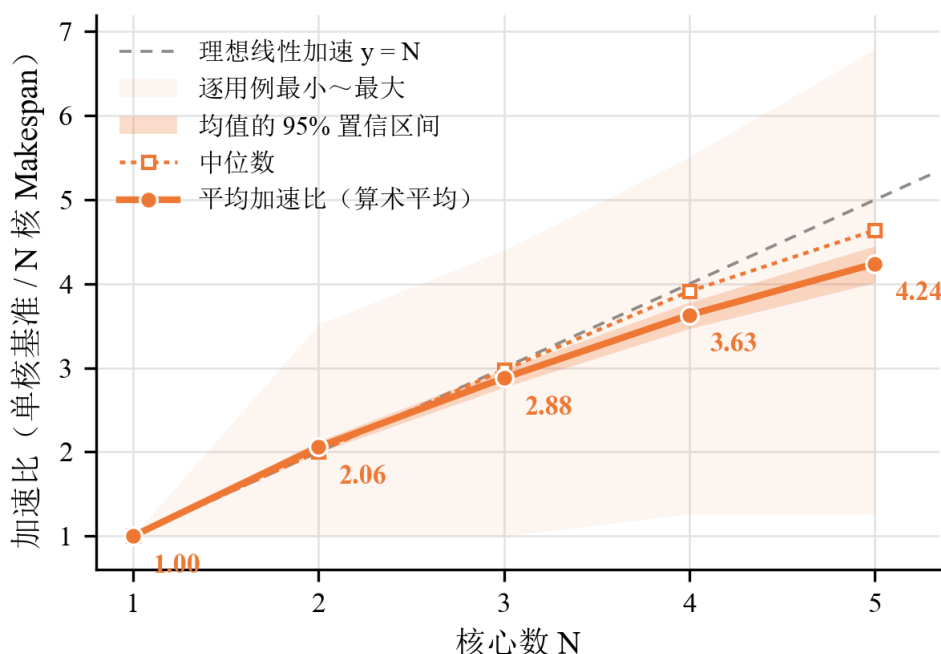


图 6-2 问题二在 1~5 核下的平均加速比（阴影为 bootstrap 95% 置信区间与逐用例范围）

表 6-1 问题二的加速比统计（100 个用例）

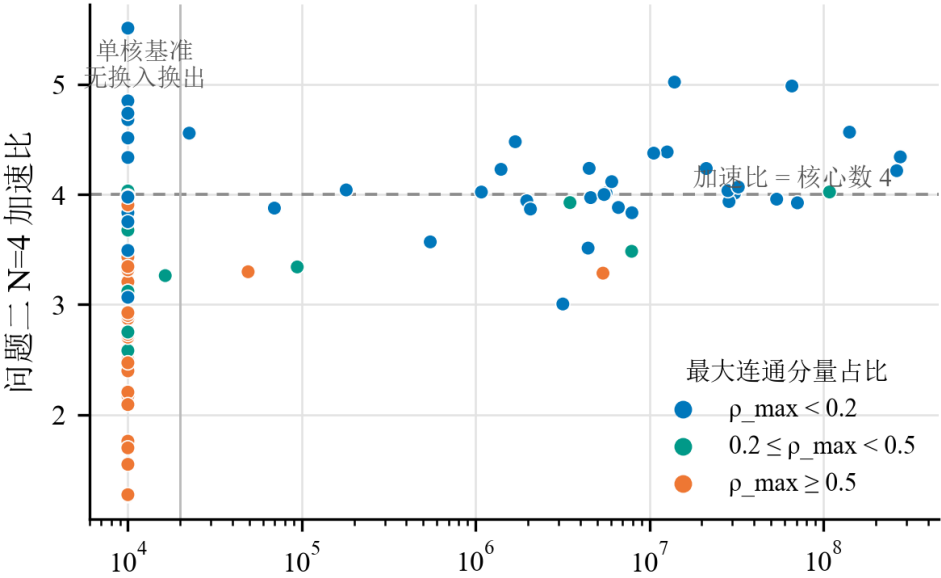
核心数	$N = 2$	$N = 3$	$N = 4$	$N = 5$
平均加速比（算术平均）	2.06	2.88	3.63	4.24
95% 置信区间	[1.99, 2.13]	[2.77, 2.98]	[3.47, 3.78]	[4.01, 4.45]
几何平均（补充）	2.03	2.82	3.52	4.06
中位数	2.01	2.98	3.91	4.64
最小值 / 最大值	1.00 / 3.51	1.00 / 4.40	1.26 / 5.51	1.26 / 6.79
超线性用例数（加速比 > N ）	56	38	29	20
并行效率（均值 / N ）	103%	96%	91%	85%

问题二在每个核数下都高于问题一（ $N = 4$ 时 3.63 对 3.19），且差距随核数增大。原因在于场景 A 的每条子图边界都要经 DDR 往返，同核 Task 完全串行，核数越多，跨核 Task 依赖越多，1000 cycle 的等待与边界搬运的代价越大；场景 B 则把这两项代价都压缩到了跨核边上。

6.6.2. 超线性加速的来源

$N = 4$ 时有 29 个用例的加速比超过 4， $N = 2$ 时有 56 个超过 2， $N = 2$ 的平均并行效率 103% 已超过 100%。按式 (4-8)，超线性的前提是单核基准存在可被多核切分消除的损失。100 个用例中有 40 个的单核基准存在换入换出，合计 1.24×10^9 字节，单个用例最多 2.77×10^8 字节。图 6-3 把 $N = 4$ 的加速比与单核基准的换入换出量对照：超线性用例集中在右侧（单核基准有大量换入换出）且分量碎片化

的区域, 29 个超线性用例中有 21 个属于这一类。其余超线性用例的单核基准没有换入换出, 我们推断其来源是单核启发式调度本身的损失: 单核基准相对单核下界的比值中位数为 1.15, 这部分空隙在多核切分、核内序列变短后可以被部分收回。



单核基准自身的换入换出字节数（左侧竖线左边为无换入换出的用
图 6-3 问题二 $N = 4$ 的加速比与单核基准自身换入换出量的关系

6.6.3. 额外搬运的构成

图 6-4 给出三个问题额外搬运量的构成。问题二 $N = 4$ 时 100 个用例合计新增 7.40×10^8 字节, 其中切分边界占 53.9%, 其余为核内换入换出; 这一总量只有单核基准自身换入换出总量的 60%。另有 10 个用例的额外搬运为 0: 它们的各个连通分量整体落在单个核上, 既无跨核通信也无换入换出。问题一的额外搬运明显更多, 且以切分边界为主, 这正是场景 A“边界经 DDR 往返”的代价。

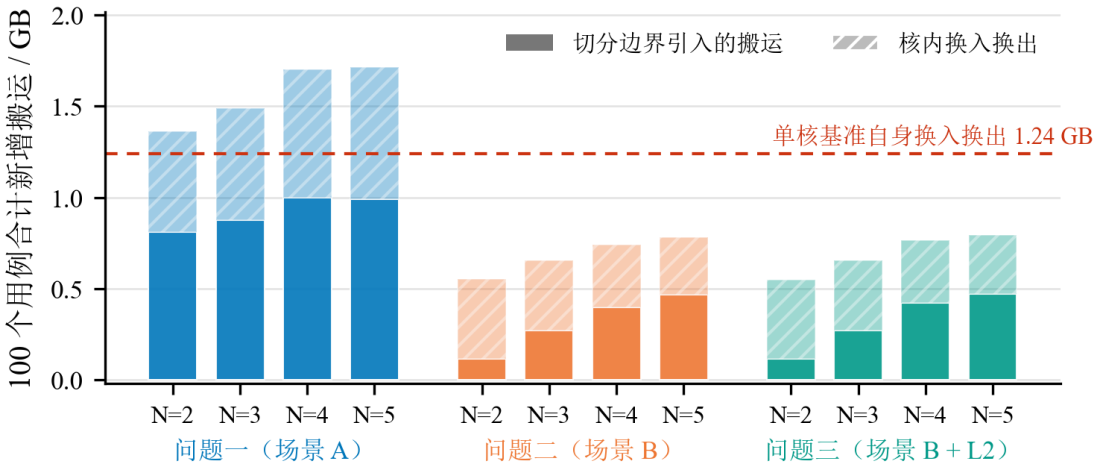


图 6-4 三个问题在 $N = 2 \sim 5$ 下的额外搬运量构成 (100 个用例合计)

6.6.4. 结构分层与难例

按最大连通分量占比 $\rho_{\max}$ 把用例分为三层（表 6-2），可以看出加速比的来源与上限。

表 6-2 按最大连通分量占比分层的加速比（问题二， $N = 4$ ）

分层	用例数	平均加速比	中位数	最小值
$\rho_{\max} < 0.2$ （分量碎片化）	59	4.07	3.99	3.00
$0.2 \leq \rho_{\max} < 0.5$	12	3.48	3.58	2.58
$\rho_{\max} \geq 0.5$ （单一主分量）	29	2.81	2.90	1.26

分量碎片化的用例可以按连通分量装箱，几乎没有跨核依赖，平均加速比超过核数；单一主分量的用例必须横切主分量，跨核同步不可避免。 $N = 4$ 时最难的用例为 case_016（1.26）、case_024（1.27）、case_048（1.55）、case_064（1.70）、case_051（1.76）、case_071（2.09）。以 case_016 为例，它有 17995 个算子，深度 2440，平均宽度只有 7.4，总计算量与关键路径之比为 11.8，即使不计任何通信，可用的并行度上限也只有 11.8。它的结构接近一条很长的同步链，每一段只有少量并行，要在多核上加速必须横切每一段，每切一段就多一次跨核同步；它在问题二 $N = 4$ 的加速比为 1.26（问题一为 1.12）。对这类结构，子图级切分的空间有限，需要算子内部的切分（第 9.2 节）。

6.7. 本章小结

针对问题二，本章证明场景 B 的跨核搬运量是以张量字节为权的 $(\lambda-1)$ 超图割，并指出问题二是“分核决定通信、顺序决定驻留”的两层问题。在 CAP-LS 的基础上，新增单算子子图与容量感知表调度候选，使冠军方案的换入换出减少 64%，并以优先级采样和跨问题互投进一步提升。 $N = 2 \sim 5$ 的平均加速比为 2.06、2.88、3.63、4.24，全面高于问题一；超线性加速主要来自单核基准自身换入换出的消失，难例集中在单一主分量、长同步链的结构上。

第7章 问题三模型的建立与求解

7.1. 问题分析

问题三在场景 B 的基础上加入所有核心共享的只读 L2 Cache（1 MB，250 B/cycle，带宽独立于 DDR），要求研究其容量与带宽对切图与调度的影响。回答这个问题需要先弄清三件事：命中从哪里来、命中能省下多少时间、命中能否兑现。只读 Cache 只服务 COPY_IN，因此它不改变跨核依赖的结构，只改变“重复读取”的代价——在问题二中，把一个被多核读取的张量分别读到多个核心是纯粹的额外搬运；有了 L2，第二次起的读取可能以约四倍的速度完成且不占 DDR 带宽。这一代价变化应当反映到分核决策中。另一方面，L2 是 FIFO 淘汰、发射时判定命中的，命中能否兑现取决于各核读取同一张量的时间间隔，这又与切图和调度相关。

7.2. 求解思路

问题三的求解流程如图 7-1 所示：先由评估器的命中规则推出命中来源、命中上界与时序条件；再把预测命中字节按 250 B/cycle 折算进分配代价，在 CAP-LS 中求解并由官方问题三评估器择优；通过问题二、三冠军方案的互投保证只读 Cache 配置不劣于无 L2；最后给出两种配置的对比，并用命中来源拆分与灵敏度实验分析容量与带宽的作用。

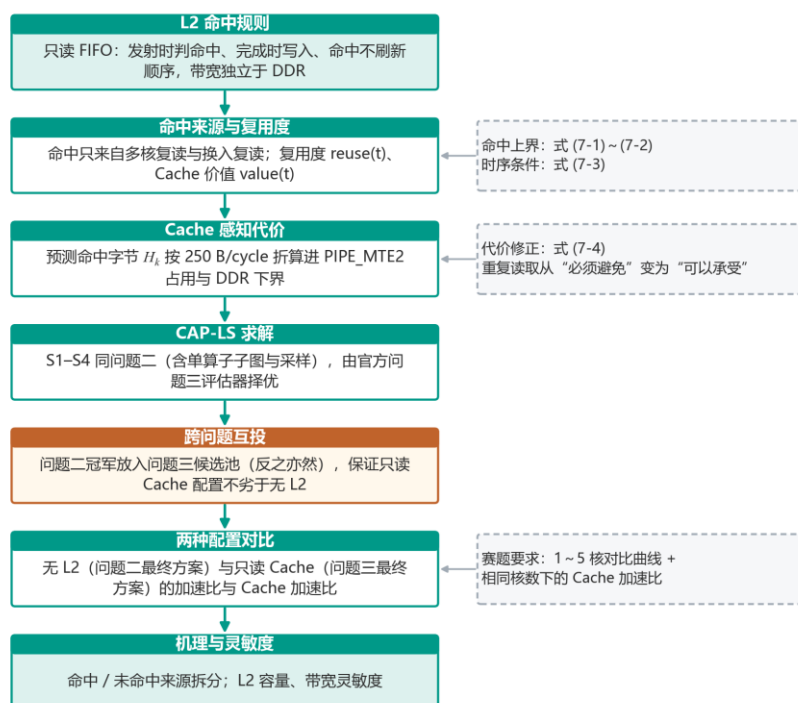


图 7-1 问题三求解流程图

7.3. 只读 L2 的命中模型

官方问题三评估器在问题二的执行模型上增加一个共享只读 Cache，规则可以归纳为四条：（1）只有 COPY_IN 查询 Cache，命中键是写入片上张量的逻辑编号；

（2）是否命中在 COPY_IN 发射时判定，命中时以 $bw_{L2} = 250 \text{ B/cycle}$ 读取，且不进入 DDR 带宽池；（3）未命中的 COPY_IN 走 DDR，完成时才写入 Cache；（4）容量 $C_{L2} = 1 \text{ MB}$ ，按 FIFO 淘汰，命中不改变条目顺序，大于容量的张量不写入。

由此，一个张量能命中的前提是它此前已经被某个 COPY_IN 从 DDR 读入过。满足这一前提的读取只有两类：一是同一张量被多个核心各自读入（图输入张量、跨核中间张量），第一次未命中并写入 Cache，之后的读取可能命中；二是同一核心对同一逻辑张量的再次读入（释放后重读，或换出后换入）。定义张量 t 的复用度与 Cache 价值为

$$\text{reuse}(t) = |\{\text{对 } t \text{ 的 COPY_IN}\}|, \text{value}(t) = s_t(\text{reuse}(t) - 1)\mathbb{1}[s_t \leq C_{L2}] \quad (7-1)$$

$\sum_t \text{value}(t)$ 是命中字节的上界。一次命中节省的搬运时间为 $s_t(1/bw - 1/bw_{L2})$ ，于是

$$\Delta T_{L2} \leq \sum_t \text{value}(t) \left(\frac{1}{bw} - \frac{1}{bw_{L2}} \right) = 0.0127 \sum_t \text{value}(t) \quad (7-2)$$

命中还不占 DDR 带宽池，相当于把 DDR 争用降低了同样的字节数，这是 L2 的第二重收益，在带宽受限的用例上更明显。

上界能否达到取决于时序。把张量 t 写入 Cache 之后、最后一次读 t 之前写入 Cache 的字节数记为复用距离 D_t ，在 FIFO 且命中不刷新的规则下， t 的后续读取全部命中的条件是

$$D_t + s_t \leq C_{L2} \text{ 且后续读取的发射时刻晚于首次读入的完成时刻} \quad (7-3)$$

两个条件的方向相反：读 t 的各个核心相隔太远， D_t 变大， t 在被复用前就被淘汰；相隔太近，后面的读取在第一次读入完成前就已发射，同样未命中。这就是 L2 与切图、调度的耦合点。

7.4. Cache 感知的分配代价

问题三把命中折算进 S2 的代理 F ：核心 k 的 PIPE_MTE2 占用由 B_k^{in}/bw 改为

$$\frac{B_k^{\text{in}} - H_k}{bw} + \frac{H_k}{bw_{L2}} \quad (7-4)$$

全局 DDR 下界由 B_{DDR}/bw 变为 $(B_{\text{DDR}} - \sum_k H_k)/bw$ 。其中 H_k 为核心 k 的预测命中字节：对每个要从 DDR 读入多个核心的张量（图输入张量或跨核中间张量），第一个读入核按未命中计，其余读入核按命中计，大于 C_{L2} 的张量不计。这一改动带来一个定性变化：在问题二中，把一个被多核读取的张量复制到多个核心是纯粹的损失；在问题三中，第二份起只花约四分之一的时间且不占 DDR 带宽。重复读取的代价下降后，分核可以更多地追求负载均衡与并行。

式（7-4）没有模拟 FIFO 淘汰与式（7-3）的时序条件，因此它对命中是乐观估计。为弥补这一点，S4 由官方问题三评估器择优，并把问题二的冠军方案放进问题三的候选池（跨问题互投）。最终 400 个问题三方案中有 80 个来自问题二的候选，其余来自问题三自身的网格、单算子子图、采样与其他实验阶段的方案。

7.5. L2 容量与带宽的影响

模型推断。 由式（7-2），L2 带宽决定单次命中的收益系数 $1/bw - 1/bw_{L2}$ 。 $bw_{L2} \rightarrow \infty$ 时系数趋于 $1/bw$ ，题设的 250 B/cycle 已取得这一上限的 76%；把带宽降到 125 B/cycle 只剩 52%；降到与 DDR 相同的 60 B/cycle 时单次命中不再更快，只剩“不占 DDR 带宽池”的收益。带宽越高，边际收益越小。容量则通过式（7-3）的第一个条件起作用：它决定一个被多核复读的张量能否在全部读者读完前留在 Cache 中。容量越小，要求各读者在时间上越集中，调度对命中率的影响越大；容量足够大时第一个条件自动满足，命中只受第二个条件和 $value(t)$ 的限制。

实测。 表 7-1 在 18 个用例、 $N = 4$ 上把 L2 容量在 64 KB~16 MB、带宽在 60~1000 B/cycle 之间变动，在修改后的参数下重新求解并评估（该实验改动了硬件参数，属于研究性实验，其评估缓存与正式结果物理隔离）。表中数值为题设参数下的 Makespan 与该参数下 Makespan 之比的几何平均，大于 1 表示变快。

表 7-1 L2 容量与带宽的灵敏度（18 个用例， $N = 4$ ，相对题设参数）

L2 容量	64 KB	256 KB	1 MB（题设）	4 MB	16 MB
Makespan 比	0.998	0.999	1.000	1.000	1.000
L2 带宽 / (B/cycle)	60	125	250（题设）	500	1000
Makespan 比	0.999	1.000	1.000	1.000	1.000

在这 18 个用例上，容量和带宽在所测范围内的影响都不超过 0.2%。结合模型推断，我们的解释是：对这批用例，L2 收益的上限由图中可共享的复读量 $\sum_t value(t)$ 决定，题设的 1 MB、250 B/cycle 已接近饱和。这一结论有明确的适用范围：灵敏度样本并非按 L2 收益抽取， $N = 4$ 时 Cache 加速比最高的 10 个用例只有 2 个在样本内，样本的平均命中率为 19.9%，低于全体的 26.1%。第 7.6 节的未命中拆分显示， $N = 4$ 时 50.4% 的未命中字节是“写入后被 FIFO 淘汰再读”，说明对高复用用例，容量减小时命中率可能明显下降。

7.6. 求解结果与分析

7.6.1. 无 L2 与只读 Cache 两种配置的对比

赛题要求报告 1~5 核下无 L2 与只读 Cache 两种配置的对比，以及相同核数下只读 Cache 相对无 L2 的加速比 $MK_{\text{无 L2}}/MK_{\text{只读}}$ 。本文的口径是：无 L2 配置取问题二的最终方案（场景 B 无 Cache），只读 Cache 配置取问题三的最终方案，两者各自在本配置下择优； $N = 1$ 时两种配置都取整图单核方案。两条加速比曲线都以

官方单核基准为分母。作为补充，还给出同一方案分别在两种配置下评估的结果，用来分离 L2 本身的作用。结果见图 7-2 与表 7-2。

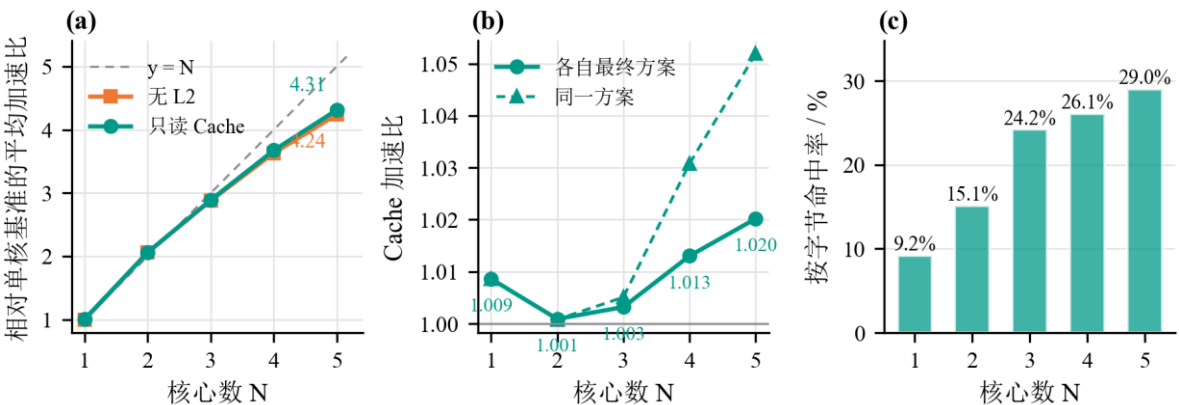


图 7-2 问题三：(a) 两种配置相对单核基准的平均加速比；(b) 只读 Cache 相对无 L2 的 Cache 加速比；(c) 只读 Cache 的按字节命中率

表 7-2 无 L2 与只读 Cache 两种配置的对比（100 个用例，算术平均）

核心数	$N = 1$	$N = 2$	$N = 3$	$N = 4$	$N = 5$
无 L2 平均加速比	1.00	2.06	2.88	3.63	4.24
只读 Cache 平均加速比	1.01	2.06	2.89	3.67	4.31
Cache 加速比（各自最终方案）	1.009	1.001	1.003	1.013	1.020
其中 > 1.01 的用例数	6	2	9	26	39
最大值（用例编号）	1.43 (044)	1.02 (071)	1.04 (057)	1.20 (046)	1.20 (046)
按字节命中率	9.2%	15.1%	24.2%	26.1%	29.0%
Cache 加速比（同一方案，补充）	1.009	1.001	1.005	1.031	1.052

表 7-2 说明三点。（1）**L2 的收益随核数增大**。Cache 加速比从 $N = 2$ 的 1.001 升到 $N = 5$ 的 1.020，命中率同步从 15.1% 升到 29.0%：核数越多，同一张量被越多核心各自读入，可命中的字节越多。同一方案在两种配置下的对比（最后一行）收益更大， $N = 5$ 时为 1.052，原因是问题二的最终方案本身针对无 L2 优化过，在无 L2 配置下比问题三的方案更好，二者之差吃掉了一部分 L2 收益。（2）**平均值小，个案差异大**。 $N = 5$ 时有 39 个用例的收益超过 1%，最高 1.20 倍（case_046）；按连通分量就能完美切分的用例几乎没有复读，命中率接近 0（图 7-3）。这与第 7.3 节的结构分析一致：收益取决于用例的输入共享结构。（3）**单核也有收益**。 $N = 1$ 时有 31 个用例的命中率非零，最高收益 1.43 倍（case_044）。单核没有跨核复读，这些命中来自同一核对同一逻辑张量的再次读入，说明只读 L2 也能缓解核内容量不足带来的重复读取。

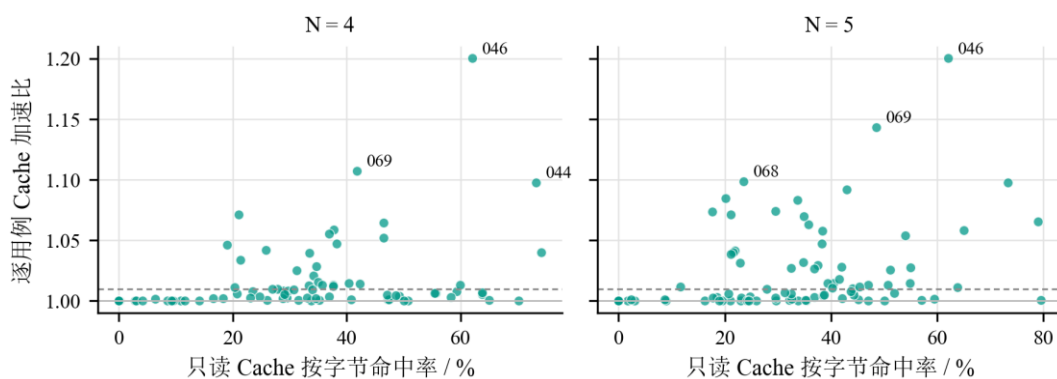


图 7-3 逐用例的 Cache 加速比与命中率（虚线为 1.01）

7.6.2. 只读 Cache 配置的加速比

问题三（只读 Cache 配置）在 1~5 核下的平均加速比如图 7-4 所示，统计量见表 7-3。

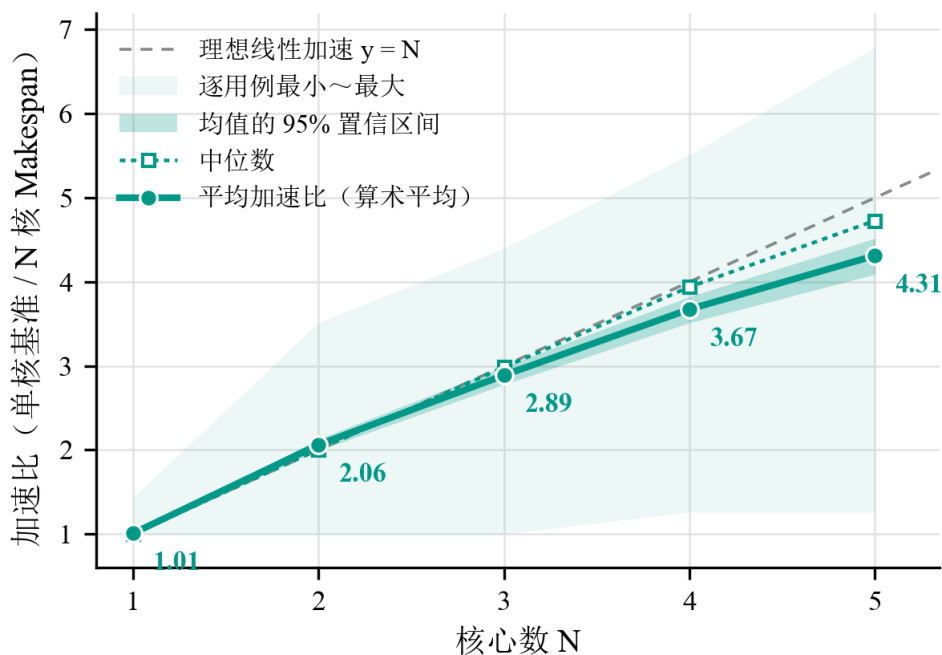


图 7-4 问题三（只读 Cache 配置）在 1~5 核下的平均加速比

表 7-3 问题三（只读 Cache 配置）的加速比统计（100 个用例）

核心数	$N = 2$	$N = 3$	$N = 4$	$N = 5$
平均加速比（算术平均）	2.06	2.89	3.67	4.31
95% 置信区间	[2.00, 2.13]	[2.78, 2.99]	[3.51, 3.82]	[4.09, 4.52]
几何平均（补充）	2.03	2.83	3.56	4.14
中位数	2.01	2.98	3.94	4.73
最小值 / 最大值	1.00 / 3.52	1.00 / 4.40	1.26 / 5.51	1.26 / 6.79
超线性用例数（加速比 > N ）	56	40	32	25
并行效率（均值 / N ）	103%	96%	92%	86%

7.6.3. 命中与未命中的来源

用官方评估器重跑问题三的全部冠军方案，读取 Cache 事件流并按张量分类（图 7-5），逐项求和与官方的命中、未命中字节完全一致。 $N = 4$ 时，96.0% 的命中字节来自图输入张量的重复读入，跨核中间张量占 4.0%。未命中字节中，40.3% 是每个张量的首次读入（冷启动，不可避免），9.2% 是多个核心在首次读入完成前并发读同一张量，50.4% 是写入后被 FIFO 淘汰、之后再读。后两类正对应式（7-3）的两个条件，合计约占未命中的 60%，是 Cache 感知调度可以继续争取的空间；本文的代价式（7-4）没有建模它们（第 9.1.2 节）。

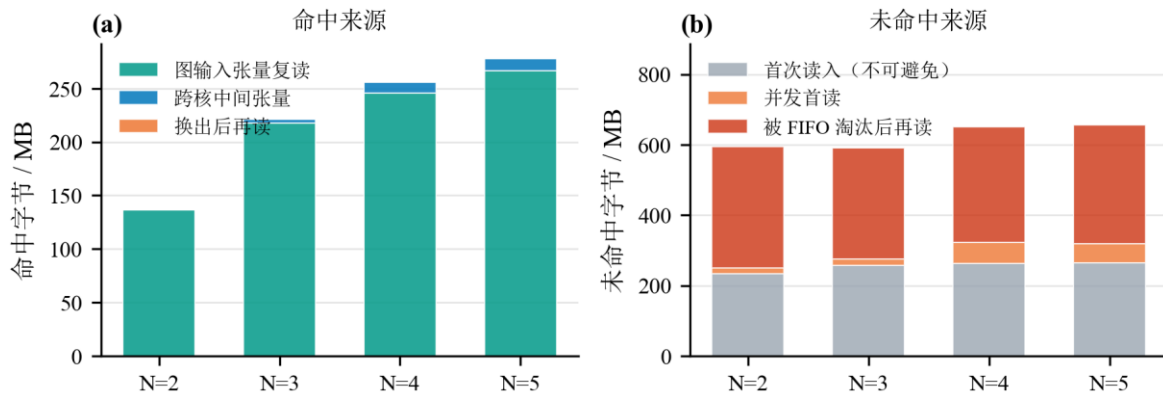


图 7-5 问题三冠军方案的 L2 命中与未命中来源（100 个用例合计）

7.6.4. 问题三与问题二的比较

任何问题二的方案都是问题三的可行方案，保底池又做了跨问题互投，所以问题三的冠军理应不慢于问题二。 $N = 4$ 时，问题三冠军在 68 个用例上快于问题二冠军，32 个相等，0 个更慢（图 7-6）。在全部 400 个（用例， N ）中有 2 个例外（括号内为问题三与问题二冠军的 Makespan，单位 cycle）：case_030（ $N = 3$ ，931340 对 931329）、case_048（ $N = 3$ ，149071 对 149061），最大相对差 0.0067%。我们用两个评估器直接复核了同一方案，确认这不是互投或本项目代码的问题，而是 FIFO Cache 本身的时序效应（事实 5）：命中让部分 COPY_IN 提前完成，改变了后续搬

运的完成顺序与 DDR 带宽的分配时序，关键路径上的个别操作因此略晚完成。这与列表调度中“资源变多反而变慢”的时序异常同类^[18]。

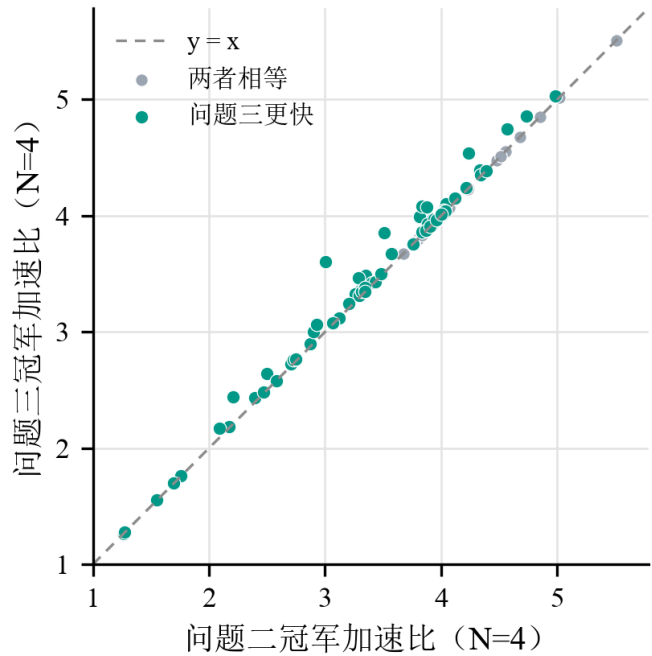


图 7-6 $N = 4$ 时问题二与问题三冠军方案的逐用例加速比对比

7.7. 本章小结

针对问题三，本章由评估器的 FIFO 命中规则推出命中只来自多核复读与换入复读，给出了命中上界式 (7-2) 与时序条件式 (7-3)，并把预测命中字节折算进分配代价式 (7-4)，使重复读取从“必须避免”变为“可以承受”。只读 Cache 配置在 $N = 2 \sim 5$ 的平均加速比为 2.06、2.89、3.67、4.31，相对无 L2 的 Cache 加速比在 $N = 1 \sim 5$ 时为 1.009、1.001、1.003、1.013、1.020，随核数增大。L2 容量与带宽在题设值附近已接近饱和，收益的上限由计算图可共享的复读量决定；约 60% 的未命中来自并发首读与 FIFO 淘汰，是后续改进的主要方向。

第8章 模型检验与灵敏度分析

8.1. 实验设置与统计口径

全部实验在赛题给出的 100 个用例、 $N = 1 \sim 5$ 、三个问题上进行，硬件参数固定为 data/config.txt。上报的 Makespan、额外搬运量与命中率全部来自官方评估程序；解析代价与事件模拟代理只在搜索内部使用。最终的 1500 个方案（三个问题 $\times$ 100 个用例 $\times$ 5 个核数）另用官方命令行脚本逐一复核，与项目内评估接口的结果不一致之处为 0。

加速比按赛题定义取 MK_1/MK_N ，100 个用例取算术平均。统计量的约定如下：均值的 95% 置信区间用 10000 次 bootstrap 重采样的百分位法^[21]；方法之间的比较用配对 Wilcoxon 符号秩检验^[22]，多重比较用 Holm 法校正^[23]，同时报告逐用例的胜/平/负数；几何平均只作补充。随机过程均固定种子，同一方案的评估结果按哈希缓存，重复运行得到相同结果。除主实验外，以下实验使用了子集：问题一的消融排除了官方评估最慢的 4 个大图（96 个用例）；粒度扫描用 29 个用例、 $N = 4$ ；硬件参数灵敏度用 18 个用例、 $N = 4$ 。子集实验的结论只在其样本上成立。

8.2. 与基线算法的对比

我们实现了四类基线（表 8-1），它们与 CAP-LS 使用同一个评估器、同一套方案格式，但分别缺少通信感知、负载均衡或层次分层中的若干项；其中 B1 直接调用赛题提供的随机切图示例程序。所有基线在全部配置上都得到了可行方案。

表 8-1 与基线算法的对比（ $N = 4$ ，100 个用例的平均加速比）

方法	设计要点	问题一	问题二	问题三
B1 随机	赛题示例程序：随机合法切图 + 随机分核	0.92	1.10	1.11
B2 拓扑等分	σ 区间等分 + 轮转，无通信、负载与分层	1.22	1.87	1.90
B3 负载均衡	σ 区间 + LPT，无通信与分层	1.23	1.67	1.69
B4 通信聚类	亲和原子化 + 轮转，无负载均衡与分层	2.35	2.78	2.83
CAP-LS 单一配置 c2	不做多起点与保底池	2.91*	3.07	3.17
CAP-LS 最终方案	本文	3.19	3.63	3.67
最终方案相对 B4	提升 / 胜平负	+36% / 99/1/0	+30% / 81/19/0	+30% / 82/18/0
最终方案相对 B4	Holm 校正 p	5.1×10^{-17}	3.8×10^{-14}	2.6×10^{-14}

注：* 问题一的单一配置取自消融实验，为 96 个用例的平均，同一批用例上 B4 为 2.29。

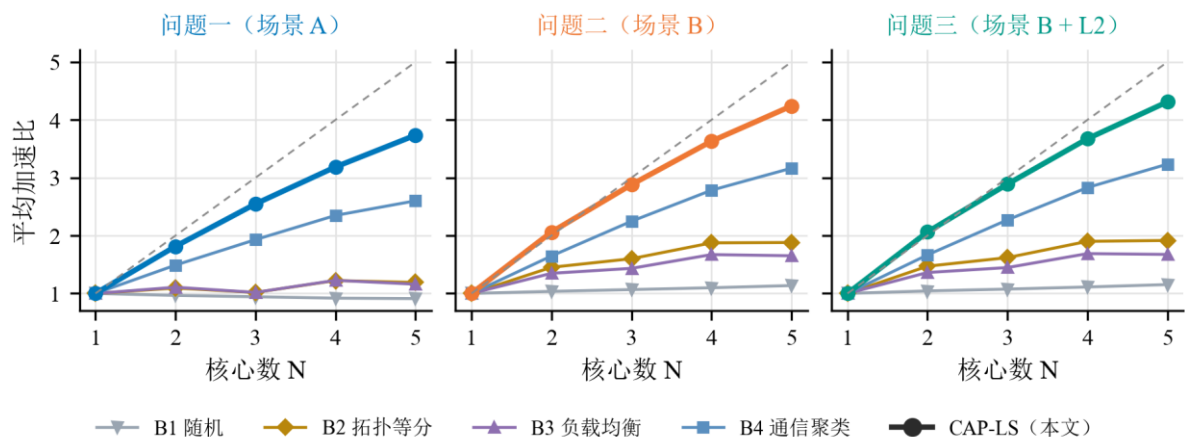


图 8-1 四类基线与 CAP-LS 在 1~5 核下的平均加速比

最强的基线是 B4 通信聚类。最终方案相对它在三个问题上分别提高 +36%、+30%、+30%，在任何用例上都没有输给它。考虑到最终方案经过了多起点与保底池，比较对象并不对等，我们也比较了不做任何择优的单一配置 c2：它在同一批用例上相对 B4 仍高 +26.8%（问题一）、+10.4%（问题二）和 +12.2%（问题三），多起点与保底池在此基础上又把问题二的平均加速比从 3.07 到 3.63。

基线之间的差别也有信息量。B1 在问题一中甚至慢于单核（ $N = 4$ 时 0.92），原因是随机切出的 Task 边界不考虑数据依赖，场景 A 中每条边界都要经 DDR 往返，同核 Task 又完全串行。B3 只看负载、不看通信，在问题一上与 B2 几乎相同，说明负载均衡本身不是瓶颈。B4 考虑了通信，但缺少负载均衡与层次分层。额外搬运方面，问题二 $N = 4$ 时 B4 的合计为 1.42×10^9 字节，CAP-LS 为 7.40×10^8 字节，约少一半。CAP-LS 的优势主要来自对跨核依赖的安排，少搬数据只起次要作用，这一点在消融实验中看得更清楚。

8.3. 消融实验

消融以单一配置 c2（ $\beta = 0.35$ 、轮转初始分配、按层次成子图）为基准，每次去掉或替换一个机制，在 $N = 2 \sim 5$ 的全部配置上与基准配对比较，结果见表 8-2 与图 8-2。

表 8-2 消融实验（相对基准 c2 的平均加速比变化， $N = 2 \sim 5$ 配对）

变体	含义	问题一	问题二	问题三
基准 c2	平均加速比	2.554	2.696	2.759
no_level	以块为子图，不分层	-19.8%	-8.3%	-9.2%
no_comm	不做亲和原子化	-6.1%	-6.6%	-6.7%
no_level_no_comm	以上两项都去掉	-58.0%	-39.7%	-39.9%
no_sync	代价中去掉同步深度项	-0.5%	-0.2%	-0.2%
no_localsearch	不做局部搜索	-0.1%	+0.3%	-0.1%
max_ops500	子图至多 500 个算子	-0.2%	+0.2%	+0.2%
lvl_alap	ALAP 填充层指派	+0.6%	+2.5%	+2.4%
op_full	单算子子图 + 表调度	—	+1.8%	+1.0%

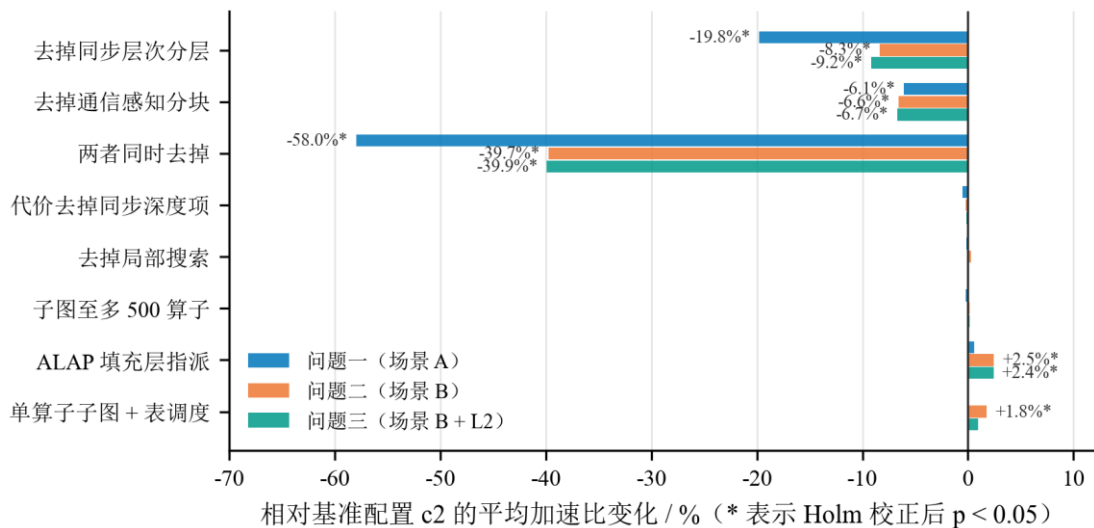


图 8-2 消融实验：各变体相对基准配置 c2 的平均加速比变化

结论有四条。（1）同步层次分层是贡献最大的单一机制。去掉后三个问题分别变化 -19.8%、-8.3%、-9.2%，问题一最敏感，因为场景 A 中跨核等待直接串行地累加在 Task 链上。（2）通信感知分块次之，且二者互补。去掉亲和原子化后变化 -6.1%、-6.6%、-6.7%；两者同时去掉时变化 -58.0%、-39.7%、-39.9%，远大于两项单独下降之和：通信感知分块减少跨核边的数量，层次分层处理剩下的跨核边。（3）局部搜索与同步深度项的单独作用很小，它们的价值主要体现在多起点组合中提供差异化的候选。（4）ALAP 层指派在问题二、三上有 +2.5%、+2.4% 的稳定提升，单算子子图与表调度在问题二上提高 +1.8%。

8.4. 灵敏度分析

8.4.1. 切图粒度

图 8-3 在 29 个用例、 $N = 4$ 上扫描粒度参数 β （单块计算量上限为 $\beta W/N$ ）。问题一的最优值在 $\beta = 2$ ，平均加速比 2.87， $\beta = 0.03$ 时降到 2.46，损失 14.2%；

问题二、三的最优值都在 $\beta = 0.5$ ，最细粒度下的损失分别只有 5.7% 与 4.1%。在 $\beta \in [0.12, 0.5]$ 内，三个问题的波动分别为 6.9%、3.3%、0.4%。这与第 5.4 节的分析一致：场景 A 偏好粗粒度，场景 B 对粒度不敏感。默认值 $\beta = 0.35$ 处在平台区内，最优粒度随用例变化，所以 S4 同时保留了粗细两端的候选。二折交叉验证中，在一半用例上选出的 β 用到另一半用例上，相对默认值的变化在 $-3.1\% \sim +0.3\%$ 之间，没有稳定优势，说明默认参数没有对这 100 个用例过拟合。

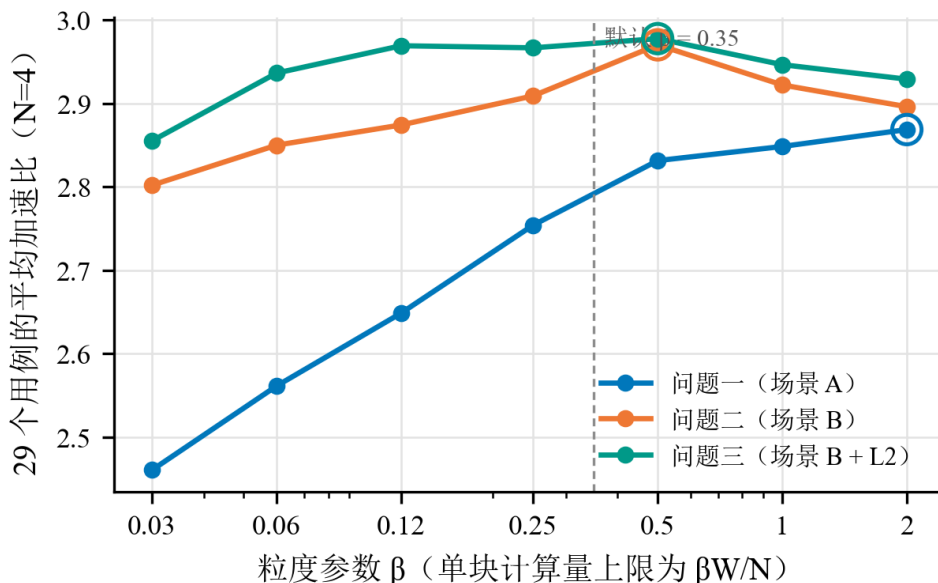


图 8-3 粒度参数 β 对平均加速比的影响（圆圈为各问题的最优值）

8.4.2. 硬件参数

为理解方法在题设配置之外的表现，在 18 个用例、 $N = 4$ 上逐一变动七个硬件参数，在修改后的参数下重新求解并评估（图 8-4）。这组实验改动了 `config.txt` 中的参数，只作研究用途，其结果与正式成绩的评估缓存物理隔离。

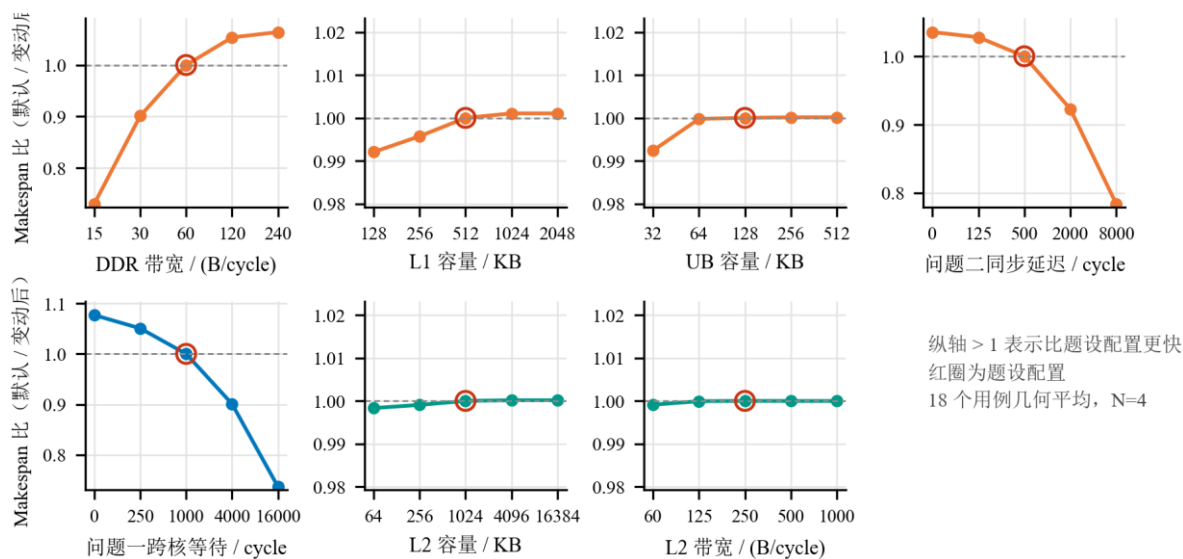


图 8-4 硬件参数灵敏度（纵轴为题设配置 Makespan 与变动后 Makespan 之比的几何平均，红圈为题设值）

DDR 带宽是最敏感的参数：减半到 30 B/cycle 时 Makespan 比为 0.901，降到 15 B/cycle 时为 0.731；翻倍到 120 B/cycle 只提高到 1.054，说明在题设带宽下多数用例已由计算或同步主导。跨核延迟的影响次之：问题二的同步延迟从 500 增加到 8000 cycle 时为 0.784，问题一的跨核等待从 1000 增加到 16000 cycle 时为 0.737；反之把它们降为 0 只提高到 1.035 与 1.077。L1 与 UB 容量在所测范围内的影响不超过 1% (0.992~1.001)，原因是切分后单核承担的工作量变小，容量在题设值附近已不是主要约束。L2 两个参数的影响见表 7-1。总体上，方法对题设配置附近的参数扰动是稳定的。

8.5. 与理论下界的距离

式 (4-7) 给出 MK_N 的下界 LB_N 。表 8-3 列出最终方案与下界之比，以及加速比达到可达上界 MK_1/LB_N 的比例（逐用例求比后取算术平均），图 8-5 给出 $N = 4$ 的逐用例分布。

表 8-3 最终方案与理论下界的距离（100 个用例）

指标	$N = 2$	$N = 3$	$N = 4$	$N = 5$
问题一 MK/LB (均值 / 中位数)	1.49 / 1.28	1.64 / 1.43	1.80 / 1.57	1.97 / 1.67
问题二 MK/LB (均值 / 中位数)	1.23 / 1.09	1.35 / 1.18	1.46 / 1.27	1.61 / 1.35
问题三 MK/LB (均值 / 中位数)	1.23 / 1.09	1.35 / 1.17	1.44 / 1.25	1.57 / 1.34
可达加速比上界 MK_1/LB_N 的均值	2.48	3.71	4.93	6.11
问题一 / 二 / 三达到上界的比例	75% / 85% / 85%	70% / 80% / 80%	67% / 76% / 77%	64% / 72% / 73%

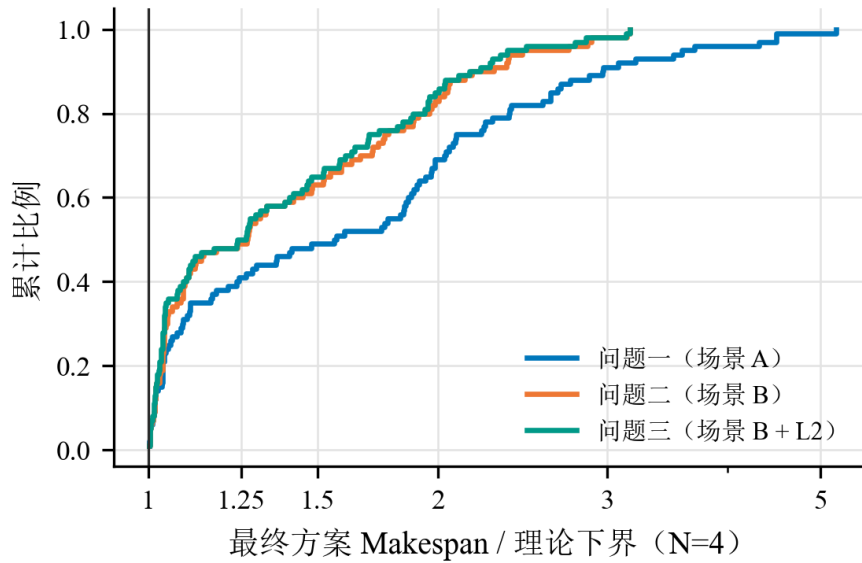


图 8-5 $N = 4$ 时最终方案 Makespan 与理论下界之比的累计分布

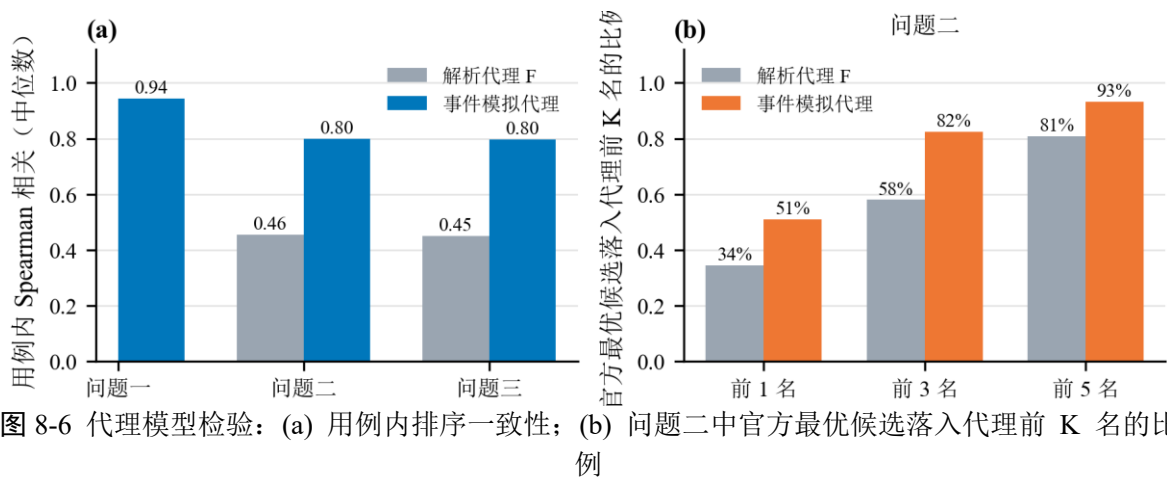
问题二 $N = 4$ 时，半数用例的 Makespan 在下界的 1.27 倍以内，平均达到可达上界的 76%。下界本身并不紧：它忽略了核内调度损失，而官方单核基准相对单核下界的比值中位数就有 1.15。可达上界的均值在 $N = 4$ 时为 4.93，高于核数，从另一个角度说明了超线性加速的来源（式（4-8））。问题一与下界的距离更大，也随核数增长更快，这是场景 A 边界搬运与 Task 串行的结构性代价。

8.6. 代理模型的可靠性与候选组合

代理与官方评估的一致性。 对每个（用例， N ）的全部网格候选同时计算代理值与官方 Makespan，检验代理的排序能力（表 8-4、图 8-6）。问题一的 Task 级代理与官方结果高度一致；问题二、三的解析代理 F 在用例内的排序一致性只有中等水平，改用算子级事件模拟代理后明显改善，但代理的第一名仍常常不是官方的第一名。这支持了 S4 的做法：代理只用于缩小范围，最终由官方评估器择优。

表 8-4 代理模型的排序能力（每个问题 400 组，每组为同一（用例， N ）的全部网格候选）

问题	代理	用例内 Spearman 中 位数	Kendall 中 位数	官方最优落入代 理前 1 / 前 3	取代理前 3 名 的损失（90% 分位）
问题一	Task 级事件模拟	0.94	0.86	67% / 92%	0.0%
问题二	解析代理 F	0.46	0.33	34% / 58%	19.8%
问题二	算子级事件模拟	0.80	0.64	51% / 82%	1.6%
问题三	解析代理 F	0.45	0.33	33% / 58%	19.8%
问题三	算子级事件模拟	0.80	0.67	51% / 84%	0.8%



在 $N = 4$ 上，若完全不调用官方评估、只凭代理在主实验候选（网格与补充搜索）中选一个，三个问题的平均加速比分别为 3.07、3.34、3.42，比官方择优的 3.14、3.58、3.65 低 -2.4%、-6.5%、-6.4%。这是官方择优的价值，也是问题一 4 个大图代理兜底的代价量级。

候选组合的边际收益。 图 8-7 在 $N = 4$ 上从网格与补充搜索的候选中逐个贪心加入使平均加速比增加最多的候选。问题一的第一个候选（退火方案）已达 3.005，加到 4 个时为 3.129，全部 12 个时为 3.141；问题二从 3.153 升到 3.510（4 个）和 3.577（26 个）。前几个候选贡献了大部分收益，但没有任何一个候选能单独代表整个组合。为检验候选选择是否对这 100 个用例过拟合，我们做了二折交叉验证：在一半用例上贪心选 4 个候选，用到另一半用例上，与全部候选的冠军相比，折外损失最大为 4.0%。

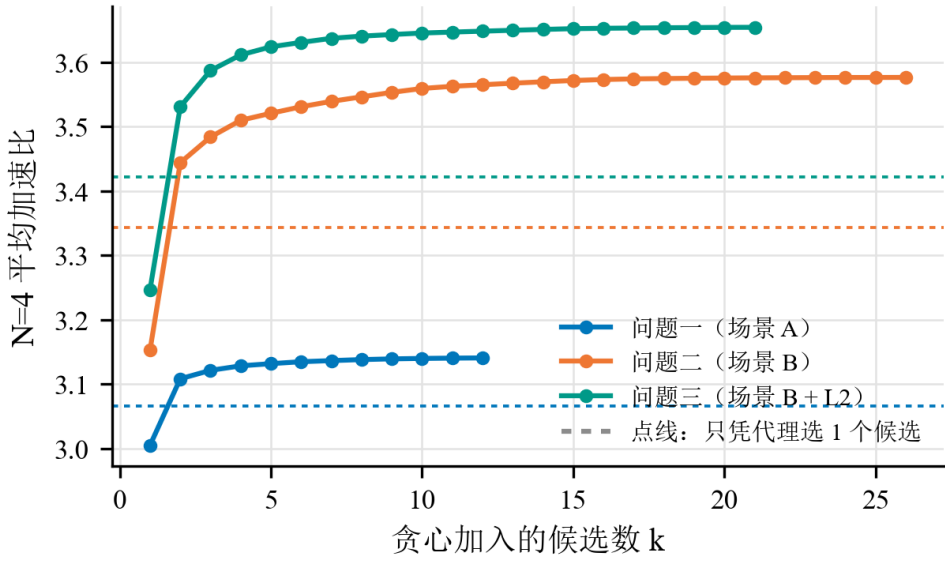


图 8-7 候选组合的贪心前向选择 ($N = 4$; 点线为只凭代理选择的平均加速比)

最终方案的来源。 表 8-5 按来源统计 1200 个最终方案。问题一中消融阶段的 CAP-LS 变体、网格候选与退火贡献最多；问题二、三中单算子子图候选贡献最多，

跨问题互投次之。整图单核兜底一次也没有用到。

表 8-5 最终方案的来源（每个问题 400 个配置）

来源	问题一	问题二	问题三
网格 c0~c11	89	54	49
网格 c12~c13（ALAP 层指派）	44	20	22
网格 c14~c17（单算子子图）	—	110	119
优先级采样	—	28	28
模拟退火	81	—	—
消融实验中的 CAP-LS 变体	151	93	93
粒度实验中的 CAP-LS 变体	7	3	5
基线算法	6	3	2
$N - 1$ 冠军嵌入	22	7	2
跨问题互投	0	82	80
整图单核兜底	0	0	0

8.7. 图结构特征与难例

图 8-8 把问题二 $N = 4$ 的逐用例加速比与四个结构特征对照。与加速比相关性最强的是最大连通分量占比 $\rho_{\max}$ （Pearson -0.69 ，Spearman -0.65 ）和关键路径占总计算量的比例（ -0.61 、 -0.60 ），平均宽度正相关（ $+0.51$ 、 $+0.55$ ），图的规模本身几乎无关（ $+0.04$ 、 $+0.08$ ）。能加速多少主要取决于图的形状，与规模关系不大。按 $\rho_{\max}$ 分层后，问题一 $N = 4$ 三层的平均加速比为 3.67、2.99、2.28，与问题二（表 6-2）的规律相同。

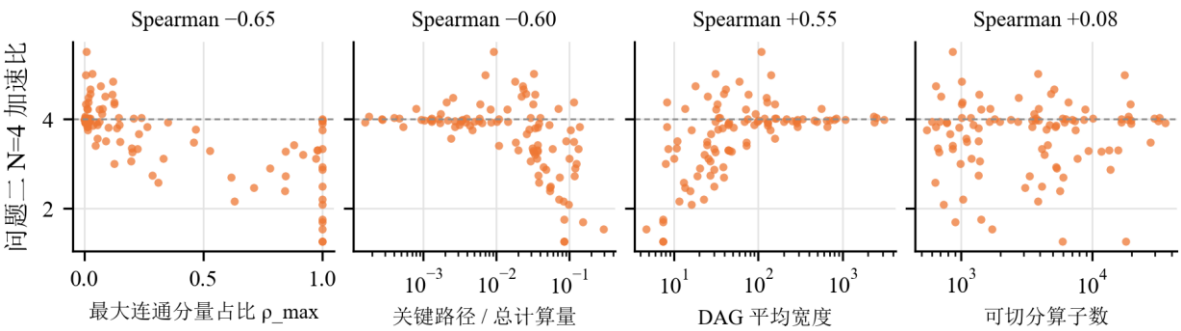


图 8-8 问题二 $N = 4$ 的逐用例加速比与图结构特征（标题为 Spearman 相关系数）

图 8-9 把三个问题全部（用例， N ）的并行效率排成热力图，用例按 $\rho_{\max}$ 升序排列。左侧分量碎片化的用例在三个问题中都接近甚至超过理想效率，右侧单一主分量的用例效率明显下降，且问题一下降最多：它们必须横切主分量，而每切一刀都会引入跨核同步，在场景 A 中这些同步还要叠加 1000 cycle 的等待和 DDR 往返。

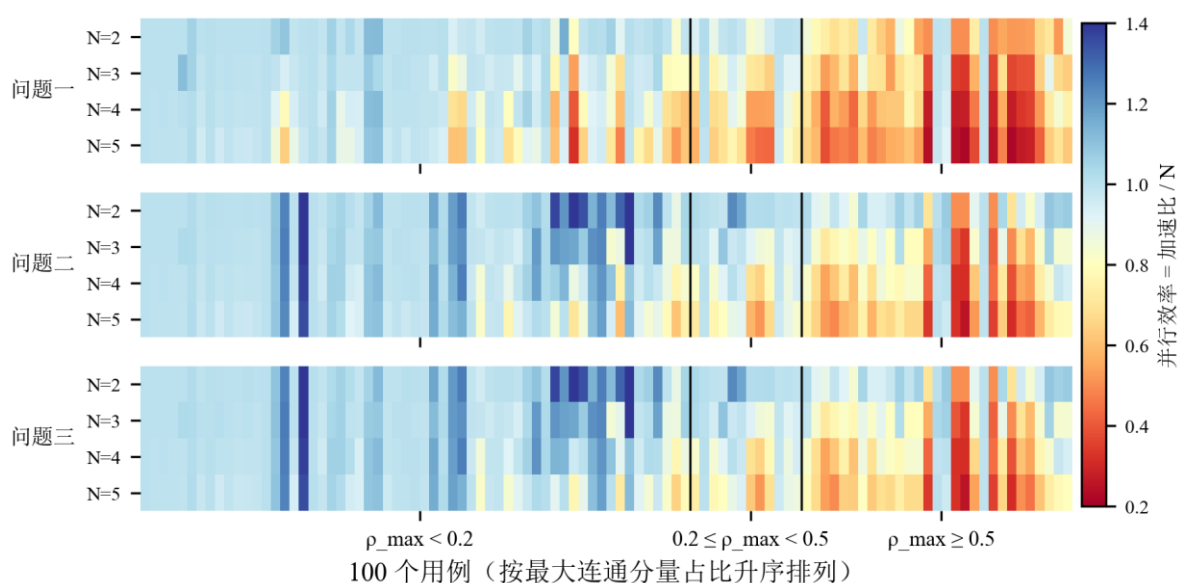


图 8-9 三个问题逐用例的并行效率（加速比 / N），用例按最大连通分量占比升序排列

8.8. 赛题优化方向的对应

赛题列出了六个优化方向。表 8-6 汇总本文在每个方向上的建模或算法手段，以及实验中直接测得的效果。

表 8-6 赛题六个优化方向与本文的对应

优化方向	本文的模型或算法	实测效果
子图间通信成本	$(\lambda-1)$ 超图割式 (4-6)；S1 亲和原子化；通信加权 LPT (c6)	去掉亲和原子化：-6.1% / -6.6% / -6.7%；搬运量模型与官方最大相对误差 0.0000%
多核并行度	同步层次分层（引理 4-1、4-2）；Cube/Vector 分开计负载的 LPT；粒度与降核候选	去掉分层：-19.8% / -8.3% / -9.2%； $N = 4$ 达到可达上界的 67% / 76% / 77%
数据重复读取代价	扇出阈值区分强连接与广播张量；问题三把命中折算进代价式 (7-4) 并跨问题互投	只读 Cache 相对无 L2： $N = 5$ 为 1.020，命中率 29.0%， $N = 4$ 时 96.0% 的命中来自输入张量复读
Cube 与 Vector 并行	代价中 Cube、Vector 分开计负载；表调度的流水线交错项 ν	相对完整表调度去掉 ν ：-2.5%（问题二） / -2.6%（问题三）
核内启发式调度对大图的效率	子图规模控制（ β 、max_ops、层等价类）；单算子子图 + 容量感知表调度	冠军方案的换入换出减少 64%；问题二 $N = 4$ 的额外搬运只有单核基准换入换出的 60%
单核内通信对缓存资源的占用	表调度的驻留项 $\mu \Delta \text{live} / C$ 与容量余量 α	相对完整表调度去掉 μ ：-1.1% / -1.1%

8.9. 本章小结

本章从五个角度检验了模型与算法。与四类基线相比，CAP-LS 在三个问题上都显著更优且没有一个用例劣于最强基线；消融实验确认同步层次分层与通信感知分块是两项核心机制，且二者互补；粒度与硬件参数的灵敏度分析说明方法在题设配置附近稳定，并刻画了带宽、跨核延迟等参数的影响边界；与理论下界的比较给出了改进空间的量级；代理检验与交叉验证说明官方择优是必要的，候选组合没有明显过拟合。

第9章 模型的评价与推广

9.1. 模型的优缺点

9.1.1. 模型的优点

(1) **以评估器为准，模型与评分口径一致。** 本文先逐行读懂官方评估程序，再确定模型结构。搬运量解析模型逐条对应评估器的插入规则，在 90 组对照中与官方输出的最大相对误差为 0.0000%；“全局申请序导致整核停摆”这一机制也是从评估器中读出、并由官方 trace 证实的，它决定了同步层次分层这一核心设计。

(2) **可行性由构造保证，搜索高效。** 引理 4-1 与推论 4-3 使所有按层成形的方案自动满足全部硬约束，搜索只在分核一维上进行且不需要判环，单候选的算法时间中位数为 0.23 s。全部 1500 个最终方案经官方命令行脚本逐一复核，不一致数为 0。

(3) **一套框架覆盖三个场景，效果显著。** 三个问题共用 CAP-LS，差异只落在代价权重、候选集合与 L2 折算上。与最强基线相比， $N = 4$ 时三个问题的平均加速比分别高 +36%、+30%、+30%，且没有一个用例劣于该基线。

(4) **结果稳定、可复现、可解释。** 最终方案的加速比在每个核数下都不低于 1，逐用例随核数单调不降；随机过程固定种子，评估结果按方案哈希缓存，附录 B 给出从零复现与逐方案复核的步骤；加速比、超线性、L2 收益等现象都有对应的机理解释（式（4-8）、式（7-2）、式（7-3））。

9.1.2. 模型的缺点

(1) **大图的求解时间。** 问题一中官方评估最慢的 4 个用例改用代理兜底，候选之间凭解析代价选择，可能错过官方意义上更好的候选；问题一共有 14 个用例的部分配置超过 10 分钟，耗时主要花在官方评估上。

(2) **最终结果依赖保底池。** 上报的最终方案是全部实验阶段中经官方评估的方案的最优者；单独运行求解入口 `solve.py`（只含网格候选）得到的结果略低，例如 $N = 4$ 时只用网格候选并由官方择优，三个问题的平均加速比为 3.11、3.54、3.63，最终方案为 3.19、3.63、3.67。二折交叉验证的折外损失不超过 4.0%，说明组合选择没有明显过拟合。

(3) **代理精度与 Cache 感知调度。** 问题二、三的解析代理在用例内的排序一致性只有中等水平，因此必须依赖官方择优；问题三的代价不模拟 FIFO 淘汰与发射时刻的命中判定，而 $N = 4$ 时约 60% 的未命中字节恰好来自这两种情形，这部分收益本文没有争取。

(4) **结构性难例与子集实验。** 对 `case_016` 这类长同步链结构，子图级切分的空间有限， $N = 4$ 时加速比只有 1.26（问题二）；本文没有改动核内调度，也没有做算子内部的切分。此外，粒度与硬件灵敏度实验只在子集上进行，高复用用例的 L2 容量-带宽曲线尚未完成。

9.2. 模型的改进与推广

(1) **Cache 感知的核内顺序。** 由式 (7-3)，让读取同一共享张量的各核在时间上“错开但不过远”，可以同时减少并发首读与 FIFO 淘汰两类未命中。把复用距离加入表调度的评分式 (6-4)，是问题三最直接的改进方向。

(2) **算子内部切分。** 对长同步链结构，把单个大算子沿数据维切开，可以在不增加同步层数的前提下提高每层的并行度。这需要改写计算图，超出了赛题允许的方案形式，但在实际编译器中是可行的。

(3) **更快、更准的评估。** 事件模拟代理已使问题二、三的排序一致性明显提高；若能实现与官方评估器逐位一致的快速评估，大图的代理兜底即可取消，求解时间也能全部控制在 10 分钟内。

(4) **向更一般的硬件与更大的图推广。** 同步层次分层只依赖“跨核依赖的真实代价远大于名义延迟”这一性质，对多芯片、异构核同样适用；把 S1 换成成熟的超图或有向无环划分器^[5,7]，可以处理规模更大的图；候选选择也可以用图结构特征（第 8.7 节）预测，进一步减少官方评估次数。

9.3. 总结

本文针对通用 NPU 的多核切图与调度问题，以官方评估程序为唯一事实基准，完成了“机理分析—统一建模—算法设计—检验评价”的完整链条。机理上，识别出全局申请序导致整核停摆这一决定性机制；模型上，以跨核同步层次构造子图，把全部硬约束化为构造性定理；算法上，设计三个问题共用的 CAP-LS 框架，并针对三个场景分别加入 Task 级时序模型与代理退火、容量感知表调度、Cache 感知代价。三个问题在 1~5 核下的平均加速比汇总于图 9-1：问题二、三的曲线在 $N \leq 3$ 时贴近甚至略高于理想线性加速，问题一因场景 A 的边界往返与 Task 串行而低一档，三者都随核数稳定上升。

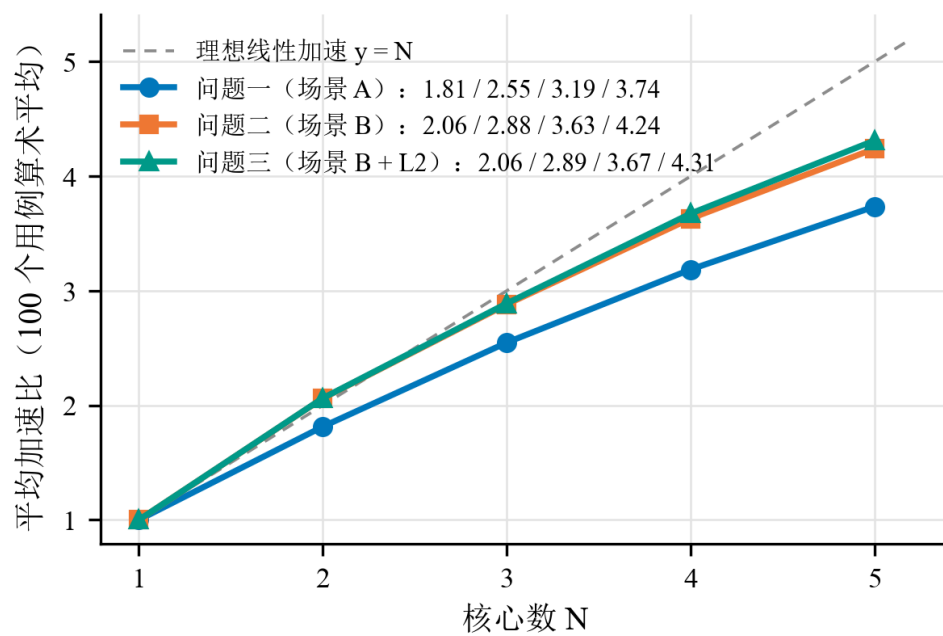


图 9-1 三个问题在 1~5 核下的平均加速比汇总（100 个用例算术平均）

参考文献

- [1] Liao H, Tu J, Xia J, et al. Ascend: a scalable and unified architecture for ubiquitous deep neural network computing[C]//2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE, 2021: 789-801.
- [2] Ullman J D. NP-complete scheduling problems[J]. Journal of Computer and System Sciences, 1975, 10(3): 384-393.
- [3] Kwok Y K, Ahmad I. Static scheduling algorithms for allocating directed task graphs to multiprocessors[J]. ACM Computing Surveys, 1999, 31(4): 406-471.
- [4] Karp R M. Reducibility among combinatorial problems[M]//Miller R E, Thatcher J W. Complexity of Computer Computations. New York: Plenum Press, 1972: 85-103.
- [5] Karypis G, Kumar V. A fast and high quality multilevel scheme for partitioning irregular graphs[J]. SIAM Journal on Scientific Computing, 1998, 20(1): 359-392.
- [6] Çatalyürek Ü V, Aykanat C. Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication[J]. IEEE Transactions on Parallel and Distributed Systems, 1999, 10(7): 673-693.
- [7] Herrmann J, Özkaya M Y, Uçar B, et al. Multilevel algorithms for acyclic partitioning of directed acyclic graphs[J]. SIAM Journal on Scientific Computing, 2019, 41(4): A2117-A2145.
- [8] Mirhoseini A, Pham H, Le Q V, et al. Device placement optimization with reinforcement learning[C]//Proceedings of the 34th International Conference on Machine Learning. PMLR 70, 2017: 2430-2439.
- [9] Jia Z, Zaharia M, Aiken A. Beyond data and model parallelism for deep neural networks[C]//Proceedings of Machine Learning and Systems (MLSys). 2019.
- [10] Zheng L, Li Z, Zhang H, et al. Alpa: automating inter- and intra-operator parallelism for distributed deep learning[C]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022: 559-578.
- [11] Gao M, Yang X, Pu J, et al. TANGRAM: optimized coarse-grained dataflow for scalable NN accelerators[C]//Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2019: 807-820.
- [12] Belady L A. A study of replacement algorithms for a virtual-storage computer[J]. IBM Systems Journal, 1966, 5(2): 78-101.
- [13] Valiant L G. A bridging model for parallel computation[J]. Communications of the ACM, 1990, 33(8): 103-111.
- [14] Papp P A, Anegg G, Yzelman A N. DAG scheduling in the BSP model[C]//SOFSEM 2025: Theory and Practice of Computer Science. Cham: Springer, 2025.

- [15] Papp P A, Anegg G, Karanasiou A, et al. Efficient multi-processor scheduling in increasingly realistic models[C]//Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA). 2024: 463-474.
- [16] Papp P A, Böhnlein T, Yzelman A N. Multiprocessor scheduling with memory constraints: fundamental properties and finding optimal solutions[EB/OL]. arXiv:2507.17411, 2025.
- [17] Tarjan R. Depth-first search and linear graph algorithms[J]. SIAM Journal on Computing, 1972, 1(2): 146-160.
- [18] Graham R L. Bounds on multiprocessing timing anomalies[J]. SIAM Journal on Applied Mathematics, 1969, 17(2): 416-429.
- [19] Kirkpatrick S, Gelatt C D, Vecchi M P. Optimization by simulated annealing[J]. Science, 1983, 220(4598): 671-680.
- [20] Topcuoglu H, Hariri S, Wu M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2002, 13(3): 260-274.
- [21] Efron B. Bootstrap methods: another look at the jackknife[J]. The Annals of Statistics, 1979, 7(1): 1-26.
- [22] Wilcoxon F. Individual comparisons by ranking methods[J]. Biometrics Bulletin, 1945, 1(6): 80-83.
- [23] Holm S. A simple sequentially rejective multiple test procedure[J]. Scandinavian Journal of Statistics, 1979, 6(2): 65-70.

附录

附录 A 逐用例结果

本附录的全部数值由官方评估脚本在固定配置 `data/config.txt` 下给出： $N = 2 \sim 5$ 取自 `results/final.csv`（每个（用例，问题，核数）的最终方案）， $N = 1$ 取自 `results/n1.csv`（整图一个子图、放在 0 号核，由对应问题的评估器评估）。Makespan 单位为 cycle，额外搬运量单位为字节。对应的方案文件为 `results/final_plans/p<问题>/n<核数>/<用例>_multicore_res.json`，可按附录 B 的命令逐个复核。问题三的无 L2 配置即场景 B 不带 Cache，其逐用例 Makespan 与额外搬运量就是表 A-3、表 A-4。

表 A-1 问题一各用例的 Makespan (cycle)

用例	N=1	N=2	N=3	N=4	N=5
case_001	233110	116868	78545	58984	47502
case_002	261945	132841	90573	69677	56971
case_003	602212	548523	280031	225366	224860
case_004	392357	199074	134043	104188	88341
case_005	95618	65153	55517	51248	51248
case_006	82837	44485	36031	33209	28796
case_007	1012562	507494	338094	254398	203834
case_008	487605	244163	163256	122957	100500
case_009	153261	111035	88247	88247	68190
case_010	86969	44202	35952	35660	27970
case_011	184697	92646	68437	46880	46777
case_012	59088	29841	21365	18774	18241
case_013	263949	132198	88351	66982	54363
case_014	17698626	8838781	5979998	4543341	3722693
case_015	177362	88439	61168	53003	53003
case_016	7715523	7715523	7715523	6919582	6919582
case_017	162810	80139	57095	44241	35157
case_018	1004367	503406	338456	254677	204554
case_019	81144	41518	30948	24099	23617
case_020	532746	266580	177950	133704	107574
case_021	6567937	3102580	2110025	1550324	1249504
case_022	56780	28674	19800	14940	14940
case_023	122614	60389	41575	30864	24927
case_024	2555343	2555343	2473380	2359811	2053364
case_025	4863026	2431840	1622164	1217444	975282
case_026	99203	57246	47309	46327	45344
case_027	2363952	1119508	793160	559196	520248
case_028	43836040	20808437	14792144	12041229	10194624
case_029	37555	19740	12743	11028	9383
case_030	2808034	1396147	931329	701262	561417
case_031	1414369	706617	475188	362001	293945
case_032	41443	21082	16407	14775	11904

用例	N=1	N=2	N=3	N=4	N=5
case_033	1774196	893844	589568	446802	357201
case_034	842996	418733	278567	218066	180411
case_035	169571	115014	106705	105203	89476
case_036	931510	466068	311345	233584	187182
case_037	210001	105374	74038	53320	43013
case_038	1364555	679097	459298	342469	275584
case_039	1435262	692660	484854	379863	326287
case_040	253454	185290	143249	110202	110202
case_041	6003957	2986534	1994174	1496957	1192316
case_042	1686740	842623	575665	434592	351352
case_043	677656	395771	301895	242816	208601
case_044	154407	96401	96401	96401	96401
case_045	125614	63059	42601	31878	26088
case_046	276455	151992	133707	115384	115384
case_047	326508	297834	160313	144597	134932
case_048	222220	212543	202773	202773	202773
case_049	196276	171171	122860	78438	69579
case_050	149690	117878	71842	61002	57813
case_051	607628	607628	549836	533409	491857
case_052	180301	88782	60932	48113	37926
case_053	1472686	758976	570063	556564	494868
case_054	815051	510285	364389	264500	264500
case_055	118835	57684	38093	31823	26800
case_056	277285	158608	136182	110892	96683
case_057	59726	33614	26147	22025	18596
case_058	4539717	2266429	1519779	1154519	948106
case_059	829473	369318	266217	185216	150782
case_060	888789	452002	298981	232939	179018
case_061	77191	38873	32703	28050	23868
case_062	2854809	1429373	955282	718373	575863
case_063	1070397	537888	359105	271192	218990
case_064	19116	19071	17646	17646	16068
case_065	53068	22981	17966	13745	11421
case_066	359070	230062	168908	158070	121632
case_067	63834834	32441828	21673797	17872457	14414488
case_068	362095	249377	172181	171012	151139
case_069	25144	23844	21838	18723	18723
case_070	117300	58469	41458	32371	30646
case_071	18919	17677	16201	13512	13512
case_072	23897828	11672534	7914084	5935363	4822195
case_073	10081626	5394231	4084492	3636147	3315822
case_074	1682239	826937	544838	408750	341070
case_075	1441809	848258	756355	483591	462313
case_076	6185113	3067395	2043989	1533331	1233514
case_077	495320	250735	180572	161888	135117
case_078	2296143	1080972	902667	679519	645756
case_079	31052251	14723450	9817862	7635624	6075976
case_080	292138	145926	100325	91350	86030

用例	N=1	N=2	N=3	N=4	N=5
case_081	151795	76860	51943	39588	30943
case_082	721041	509781	268232	249866	227315
case_083	1057993	542602	404888	395632	344254
case_084	2507412	1252724	836168	629028	503472
case_085	3187899	3048236	1589859	948232	909962
case_086	118403	88853	61208	56811	56811
case_087	3974071	1993185	1459150	1152694	965586
case_088	243623	168809	115533	115533	101323
case_089	693092	346763	242914	175600	144382
case_090	1178363	670035	506027	506027	388325
case_091	9186769	4592279	3078955	2321866	1898042
case_092	4815554	2200120	1712160	1527185	1510160
case_093	74205	37827	25103	19779	15876
case_094	94892	47453	34069	24860	21487
case_095	2084853	1042787	624024	518918	420852
case_096	84194	44237	43628	38923	38923
case_097	11491509	5156692	3554934	2583836	2102348
case_098	177430	106182	82843	82843	75917
case_099	1622999	810196	542088	407859	337370
case_100	151223	90879	63340	58085	58085

表 A-2 问题一各用例的总额外数据搬运量（字节）

用例	N=1	N=2	N=3	N=4	N=5
case_001	0	1152	2304	3456	4608
case_002	0	1575936	55296	86016	112128
case_003	0	1659238	3040688	3751658	4539968
case_004	0	110592	221184	331776	442368
case_005	0	929524	1143334	1263492	1263492
case_006	16400	686320	363168	1036744	377182
case_007	0	27648	55296	82944	110592
case_008	0	1990656	1327104	995328	811008
case_009	0	2289206	699372	699372	825050
case_010	0	27648	1076548	676806	455990
case_011	0	180224	1241088	540672	1216512
case_012	0	0	87136	0	101380
case_013	0	8192	16384	24576	32768
case_014	71010720	86154144	81345584	90386328	70838688
case_015	180256	216336	290056	461872	461872
case_016	0	0	0	154476142	154476142
case_017	22528	251908	163584	245376	327168
case_018	1970176	3347456	3903872	6238464	3661824
case_019	0	0	251444	0	248220
case_020	0	0	0	0	0
case_021	4501504	6656000	7229440	4608000	8908800
case_022	0	0	0	0	0
case_023	0	301576	147456	147456	221184
case_024	0	0	61410808	67376912	67770326

用例	N=1	N=2	N=3	N=4	N=5
case_025	0	27648	55296	82944	110592
case_026	0	453364	1212616	445288	419892
case_027	1400832	3317760	6346752	2211840	12343296
case_028	277412256	326568960	326568960	326568960	326568960
case_029	0	0	0	0	0
case_030	5519360	7126016	9363456	10560000	11588608
case_031	3491328	4402838	5121536	5401088	4911428
case_032	0	260264	462204	397156	127130
case_033	4592640	4953600	7086144	7344224	8111232
case_034	1079832	1864192	2647040	3101184	3532800
case_035	0	153406	205126	1731896	1821886
case_036	0	1152	2304	3456	4608
case_037	0	204800	737280	614400	1310720
case_038	5627904	5607936	6934020	7945216	9409536
case_039	7929856	9805824	10346496	11517952	12304384
case_040	0	2193632	750764	0	0
case_041	30898176	33595392	35017728	38879232	37172224
case_042	6629376	7265568	7684096	8503584	9736192
case_043	49160	5579966	1710506	4154484	5033556
case_044	4438112	1283040	1283040	1283040	1283040
case_045	0	0	0	0	0
case_046	3176288	1819232	3761600	4554048	4554048
case_047	0	3389130	1496464	2517116	2310282
case_048	0	615242	620994	620994	620994
case_049	0	391226	1896282	978962	891320
case_050	0	581308	583052	654496	735830
case_051	0	0	19138090	16713772	16648208
case_052	0	342066	147456	221184	73728
case_053	5405536	5121886	10926778	11776336	7148056
case_054	0	1292256	2171082	2551950	2551950
case_055	69632	220350	32768	115712	198656
case_056	0	1641806	2188924	1490826	1377620
case_057	94216	0	118852	180060	168018
case_058	28696576	33689600	32636928	42622976	31719424
case_059	1691648	360448	2998272	1081344	3063808
case_060	2068520	2724108	3164416	3217544	4039168
case_061	0	25344	117032	164032	110634
case_062	0	21361152	86016	119808	153600
case_063	0	76800	5495808	104448	2113536
case_064	0	21060	117382	117382	165088
case_065	0	110228	27648	41472	55296
case_066	0	2000906	1979482	2396358	2393192
case_067	264449408	267900160	250816128	256028288	258091392
case_068	0	3519460	2668770	3075204	3449990
case_069	0	803334	788514	173960	173960
case_070	0	88064	176128	264192	352256
case_071	0	118178	479260	220566	220566
case_072	108613680	88018992	88018992	126956448	130109488

用例	N=1	N=2	N=3	N=4	N=5
case_073	141570432	137954176	136747904	136563328	139611648
case_074	6070280	6590472	7585792	7819264	8425472
case_075	0	7472044	5103460	5406232	5667772
case_076	28280832	28306440	30927876	32051200	35630276
case_077	0	57344	425870	1359978	1522830
case_078	12644568	15039360	11951544	11965008	11594736
case_079	32448512	36765696	36765696	30445568	47677440
case_080	548864	2191360	3047424	1667072	1904640
case_081	0	0	0	0	0
case_082	0	3014040	1738500	2698382	2671100
case_083	13934688	21362560	11005760	12578112	10720768
case_084	0	4967424	3308544	0	0
case_085	0	15641984	14939962	8025052	11301760
case_086	0	1837042	1346240	1738222	1738222
case_087	7902720	8248320	6727680	22261304	23677874
case_088	0	2064350	1848762	1848762	2383818
case_089	0	556160	683776	1033600	1208832
case_090	21233592	4295328	21721008	21721008	10869504
case_091	53947008	61042308	68036740	69020832	73166848
case_092	66000096	16871008	36590944	44909472	58281984
case_093	0	8192	16384	24576	32768
case_094	0	106272	212544	318816	425088
case_095	0	8515584	17805312	18284544	0
case_096	0	0	177552	1295704	1295704
case_097	10584064	11927552	42156032	14483456	21311488
case_098	0	1847330	2377992	2377992	1614026
case_099	5484544	7315456	6660096	9818112	6799360
case_100	0	1364804	1239434	0	0

表 A-3 问题二（无 L2）各用例的 Makespan（cycle）

用例	N=1	N=2	N=3	N=4	N=5
case_001	233110	116868	78545	58984	47502
case_002	261945	121768	82329	65392	54581
case_003	602212	285420	252886	182260	182260
case_004	392357	195960	130725	100126	83319
case_005	95618	52975	38285	38285	36630
case_006	82837	40056	27488	25390	22940
case_007	1012562	507494	338094	254398	203834
case_008	487605	155082	110895	88500	71833
case_009	153261	91267	66004	63921	49750
case_010	86969	41582	27958	25971	23284
case_011	184697	92646	65543	46880	40551
case_012	59088	21950	18325	15493	13832
case_013	263949	132198	88351	66982	54363
case_014	17698626	8904271	5933771	4511152	3643563
case_015	177362	83564	58469	43898	42014
case_016	7715523	7715523	7715523	6104382	6104382

用例	N=1	N=2	N=3	N=4	N=5
case_017	162810	71293	44420	35734	29386
case_018	1004367	503406	338456	254778	204554
case_019	81144	34656	27380	21169	17971
case_020	532746	266580	177950	133704	107574
case_021	6567937	3194570	2104376	1550324	1250168
case_022	56780	27481	19193	14940	12461
case_023	122614	41726	28284	28284	21898
case_024	2555343	2555343	2159403	2006540	1579198
case_025	4863026	2431840	1622164	1217444	975282
case_026	99203	48612	38424	38424	38424
case_027	2363952	1115722	764163	559196	487450
case_028	43836040	20028806	13289382	10099681	8055128
case_029	37555	19737	12743	11028	9164
case_030	2808034	1399993	931329	699599	561417
case_031	1414369	711625	478206	360302	293958
case_032	41443	21098	12670	11276	10350
case_033	1774196	893844	589568	446802	357201
case_034	842996	412272	272649	209623	165143
case_035	169571	95406	78824	78073	64436
case_036	931510	466068	311345	233584	187182
case_037	210001	105374	72874	53320	43013
case_038	1364555	679097	458108	339997	275584
case_039	1435262	708561	482923	374421	311816
case_040	253454	128421	110105	102570	102570
case_041	6003957	2986534	1994174	1496957	1193573
case_042	1686740	842623	571713	434592	343502
case_043	677656	327858	230016	205499	180949
case_044	154407	43959	43959	43959	43959
case_045	125614	63059	42601	31878	26088
case_046	276455	105718	105663	92005	92005
case_047	326508	156922	122587	113671	102458
case_048	222220	159142	149061	143466	143466
case_049	196276	100467	81125	61217	59412
case_050	149690	80347	55518	43625	43625
case_051	607628	607628	484407	345402	345402
case_052	180301	71775	48446	37183	30397
case_053	1472686	703950	553159	448335	392998
case_054	815051	411504	342634	245854	245854
case_055	118835	48250	33559	30663	26653
case_056	277285	130574	90771	88923	76464
case_057	59726	30053	21820	17875	15599
case_058	4539717	2263428	1510985	1153590	948106
case_059	829473	369318	255358	185216	150736
case_060	888789	452002	298981	229829	179018
case_061	77191	38873	29597	22111	19552
case_062	2854809	1429017	954231	716795	574337
case_063	1070397	534284	346909	270124	213041
case_064	19116	13698	11449	11254	11254

用例	N=1	N=2	N=3	N=4	N=5
case_065	53068	21905	14937	11760	10622
case_066	359070	197561	142974	132815	109925
case_067	63834834	30338059	20153440	15143945	12531847
case_068	362095	197044	147939	124804	124172
case_069	25144	13266	12194	11407	11323
case_070	117300	47580	40551	29097	26399
case_071	18919	11459	10712	9045	9045
case_072	23897828	11846123	8000696	5941844	4804197
case_073	10081626	4236675	2831560	2208323	1816764
case_074	1682239	826937	544838	408750	336523
case_075	1441809	670177	572934	383971	365235
case_076	6185113	3067902	2054012	1533331	1235977
case_077	495320	250735	176359	131477	118802
case_078	2296143	1022234	917340	523604	486337
case_079	31052251	15159240	10072944	7635624	6075976
case_080	292138	145926	100165	81859	76491
case_081	151795	76860	51943	39588	30943
case_082	721041	331837	227528	215554	198621
case_083	1057993	419453	288815	210763	184990
case_084	2507412	1255052	836220	629028	503472
case_085	3187899	1503897	1272448	816364	816364
case_086	118403	63004	47791	40468	40468
case_087	3974071	1993185	1459150	1140911	894365
case_088	243623	130634	99762	89270	84089
case_089	693092	295144	206778	148144	119458
case_090	1178363	430364	326383	278236	278236
case_091	9186769	4571759	3074692	2321866	1898042
case_092	4815554	1926278	1275324	966196	780940
case_093	74205	37827	25103	19779	15876
case_094	94892	43154	26395	20034	18550
case_095	2084853	1042890	673706	524422	420852
case_096	84194	40994	33260	26984	25104
case_097	11491509	5478142	3568148	2626908	2102348
case_098	177430	87157	64508	64508	60773
case_099	1622999	810196	541232	405864	336006
case_100	151223	85674	57948	49328	49328

表 A-4 问题二（无 L2）各用例的总额外数据搬运量（字节）

用例	N=1	N=2	N=3	N=4	N=5
case_001	0	1152	2304	3456	4608
case_002	0	90624	138240	181248	207360
case_003	0	1550994	2358296	3927966	3927966
case_004	0	1382400	221184	331776	442368
case_005	0	477222	801420	801420	837720
case_006	16400	501324	618368	493156	336400
case_007	0	27648	55296	82944	110592
case_008	0	0	0	0	0

用例	N=1	N=2	N=3	N=4	N=5
case_009	0	299150	574108	747930	929592
case_010	0	496332	548196	635524	471140
case_011	0	180224	1200128	540672	1650688
case_012	0	0	163322	74960	0
case_013	0	8192	16384	24576	32768
case_014	71010720	76033440	80019536	87805200	60562848
case_015	180256	240928	1464888	832948	317952
case_016	0	0	0	35524768	35524768
case_017	22528	81792	163584	240768	327168
case_018	1970176	3347456	3903872	5118976	3661824
case_019	0	510500	685380	582148	503844
case_020	0	0	0	0	0
case_021	4501504	3174400	7186432	4608000	11431936
case_022	0	0	112516	0	157360
case_023	0	73728	147456	147456	544408
case_024	0	0	22415452	29037088	30282428
case_025	0	27648	55296	82944	110592
case_026	0	218548	923392	923392	923392
case_027	1400832	1105920	6107136	2211840	12023808
case_028	277412256	20677824	11546880	11508096	12550656
case_029	0	16400	0	0	67656
case_030	5519360	7834624	9363456	8380416	11588608
case_031	3491328	4581888	4969984	5298688	4816896
case_032	0	168116	205008	0	0
case_033	4592640	4953600	7086144	7344224	8111232
case_034	1079832	1238528	2328576	3019264	3508224
case_035	0	0	765572	1128234	1160968
case_036	0	1152	2304	3456	4608
case_037	0	204800	1003520	614400	1310720
case_038	5627904	5607936	7012352	7758336	9409536
case_039	7929856	11067392	10113024	10510336	11489280
case_040	0	0	0	1931186	1931186
case_041	30898176	33595392	35017728	38879232	40452096
case_042	6629376	7265568	7746560	8503584	9689088
case_043	49160	1884268	3085000	4132756	5217408
case_044	4438112	930400	930400	930400	930400
case_045	0	0	0	0	0
case_046	3176288	930400	2064576	2213920	2213920
case_047	0	794950	1239692	1947090	2584600
case_048	0	233030	576140	1756370	1756370
case_049	0	319282	497688	751934	872740
case_050	0	272582	481660	571290	571290
case_051	0	0	10748968	2228604	2228604
case_052	0	73728	147456	221184	294912
case_053	5405536	2184118	2848940	12542018	14846008
case_054	0	933062	1613448	2545234	2545234
case_055	69632	98816	99328	677058	427118
case_056	0	496950	767084	1107930	1454488

用例	N=1	N=2	N=3	N=4	N=5
case_057	94216	269820	397250	382150	250392
case_058	28696576	33492992	31211520	31985664	31719424
case_059	1691648	360448	2162688	1081344	4325376
case_060	2068520	2724108	3164416	3510784	4039168
case_061	0	25344	179618	50688	328440
case_062	0	53760	72192	89088	86016
case_063	0	9019392	4116480	64512	367104
case_064	0	61542	99404	122098	122098
case_065	0	13824	27648	41472	55296
case_066	0	952044	1398840	2060756	2476400
case_067	264449408	8325504	10769152	14048384	17327616
case_068	0	1553566	2085676	3208234	3538800
case_069	0	371046	335116	432114	351476
case_070	0	88064	176128	264192	690024
case_071	0	326886	229324	267856	267856
case_072	108613680	113201200	119709046	125858856	130011184
case_073	141570432	3721600	7443200	11164800	14886400
case_074	6070280	6590472	7585792	7819264	8302592
case_075	0	2310790	3422476	5734290	6600472
case_076	28280832	31201280	34470912	32051200	34179072
case_077	0	57344	932152	147094	1592816
case_078	12644568	2148480	6471168	8694144	11371008
case_079	32448512	32612352	32722944	30445568	47677440
case_080	548864	2191360	3932160	2543616	2031616
case_081	0	0	0	0	0
case_082	0	832902	1428236	2106514	2932248
case_083	13934688	930400	1860800	2791200	3721600
case_084	0	0	0	0	0
case_085	0	3911858	5532684	8300178	8300178
case_086	0	518694	919116	1363698	1363698
case_087	7902720	8248320	6727680	21730796	6981120
case_088	0	1049798	1449996	2268882	2451224
case_089	0	238720	477440	716160	954880
case_090	21233592	2917248	4296960	7285632	7285632
case_091	53947008	57904096	60897408	69020832	73166848
case_092	66000096	5437728	3859648	3905312	3950976
case_093	0	8192	16384	24576	32768
case_094	0	114464	212544	318528	425088
case_095	0	0	16330752	0	0
case_096	0	124748	125704	0	597692
case_097	10584064	14655488	18454528	14221312	21311488
case_098	0	960674	1341690	1341690	1254586
case_099	5484544	7315456	6590464	9782400	7528448
case_100	0	306188	0	1560842	1560842

表 A-5 问题三（只读 Cache）各用例的 Makespan（cycle）

用例	N=1	N=2	N=3	N=4	N=5
case_001	233110	116868	78545	58984	47502
case_002	261945	121768	82329	65392	54581
case_003	602212	285418	251261	179646	179646
case_004	392357	195960	130725	100126	83319
case_005	95618	52945	38024	36155	35497
case_006	82837	39976	26930	24877	22533
case_007	1012562	507494	338094	254398	203834
case_008	487605	155082	110895	88500	71833
case_009	153261	91263	65976	63026	48320
case_010	86969	41582	27337	24926	22007
case_011	184697	92646	65543	46880	40551
case_012	59088	21950	18324	14808	13281
case_013	263949	132198	88351	66982	54363
case_014	17689602	8896369	5881266	4453421	3632901
case_015	177362	83405	58467	43221	42014
case_016	7715523	7715523	7715523	6103967	6103967
case_017	162810	71293	44420	35734	29386
case_018	1003110	502856	337587	252790	204343
case_019	81144	34656	27321	21005	17497
case_020	532746	266580	177950	133704	107574
case_021	6558837	3193634	2103364	1550324	1249380
case_022	56780	27481	19192	14940	12434
case_023	122614	41726	28284	27929	21898
case_024	2555343	2555343	2159126	2003351	1577373
case_025	4863026	2431840	1622164	1217444	975282
case_026	99203	48612	38423	38423	38423
case_027	2359515	1115722	763896	559196	487450
case_028	43836040	20009296	13270473	10080305	7991196
case_029	37555	19737	12743	10970	9164
case_030	2807891	1399744	931340	699009	560550
case_031	1414159	710284	478086	360132	293738
case_032	41443	20940	12612	11276	10350
case_033	1774080	893649	588351	445198	356085
case_034	842733	412214	272476	208260	164310
case_035	169571	95406	78754	77573	63503
case_036	931510	466068	311345	233584	187182
case_037	210001	105374	72874	53320	43013
case_038	1363312	678831	456400	339825	271662
case_039	1435262	705615	481303	351769	302347
case_040	253454	128421	110105	101948	101948
case_041	6001575	2981380	1990580	1493320	1191841
case_042	1675633	840543	569525	428924	342039
case_043	677656	327792	229480	204676	171690
case_044	107814	43910	43910	40054	40054
case_045	125614	63059	42601	31878	26088
case_046	244994	105718	102704	76653	76653
case_047	326508	156919	122464	112730	99665

用例	N=1	N=2	N=3	N=4	N=5
case_048	222220	159138	149071	142969	142969
case_049	196276	100433	81123	60474	58817
case_050	149690	80346	55445	43605	43605
case_051	607628	607628	483306	344572	344572
case_052	180301	71775	48446	37183	30397
case_053	1428109	703878	553159	424896	365951
case_054	815051	411477	342219	243498	243498
case_055	118835	48250	33559	29142	24606
case_056	277285	130574	90592	88843	75480
case_057	59726	30053	21007	17645	15428
case_058	4539717	2262108	1496185	1142629	947187
case_059	815938	369318	254656	185216	150665
case_060	887801	451802	298831	229535	179018
case_061	77191	38873	29381	22111	18802
case_062	2854809	1428976	954215	716754	574337
case_063	1070397	533800	344374	270081	213041
case_064	19116	13674	11407	11227	11227
case_065	53068	21905	14937	11760	10499
case_066	359070	197561	142969	131603	107074
case_067	63834834	30303605	20084166	15040197	12497325
case_068	362095	197002	147814	120739	113021
case_069	25144	13112	12005	10301	9903
case_070	117300	47580	40551	29097	26399
case_071	18919	11236	10636	8702	8425
case_072	23890756	11841570	7989837	5928167	4776264
case_073	10081626	4197779	2796345	2123315	1705058
case_074	1678120	824879	544114	405309	332111
case_075	1441809	670175	572669	383543	362317
case_076	6183098	3065275	2049651	1529902	1230166
case_077	495320	250735	175206	131477	114398
case_078	2296143	1022234	900985	523593	457569
case_079	31052251	15159240	10072944	7635624	6075472
case_080	291826	145674	99832	79590	70042
case_081	151795	76860	51943	39588	30943
case_082	721041	331832	227521	215536	197346
case_083	946145	419453	288815	210763	184990
case_084	2507412	1255052	836220	629028	503472
case_085	3187899	1503897	1272443	816273	816273
case_086	118403	62938	47288	38643	38643
case_087	3969779	1991129	1456602	1135476	894365
case_088	243623	130606	98649	88285	77530
case_089	693092	295144	206778	148144	119458
case_090	1178363	430364	324313	259720	259720
case_091	9168820	4563996	3069257	2317037	1884651
case_092	4333631	1926278	1274060	957454	771848
case_093	74205	37827	25103	19779	15876
case_094	94892	43154	26386	19544	18350
case_095	2084853	1042890	673706	524422	420852

用例	N=1	N=2	N=3	N=4	N=5
case_096	84194	40994	33260	26984	24477
case_097	11491509	5477830	3563647	2626908	2102250
case_098	177430	87088	64155	64155	56819
case_099	1618902	805721	540032	404548	327016
case_100	151223	85674	57948	49109	46608

表 A-6 问题三（只读 Cache）各用例的总额外数据搬运量（字节）

用例	N=1	N=2	N=3	N=4	N=5
case_001	0	1152	2304	3456	4608
case_002	0	90624	138240	181248	207360
case_003	0	1550994	2358296	3927966	3927966
case_004	0	1382400	221184	331776	442368
case_005	0	477222	804684	1236978	837720
case_006	16400	501324	618368	493156	916100
case_007	0	27648	55296	82944	110592
case_008	0	0	0	0	0
case_009	0	299150	574108	747930	929592
case_010	0	496332	548196	635524	670752
case_011	0	180224	1200128	540672	1650688
case_012	0	0	163322	185214	182852
case_013	0	8192	16384	24576	32768
case_014	71010720	76033440	80019536	87805200	60562848
case_015	180256	191760	1464888	832948	317952
case_016	0	0	0	35524768	35524768
case_017	22528	81792	163584	240768	327168
case_018	1970176	3347456	3903872	5118976	3661824
case_019	0	510500	685380	582148	503844
case_020	0	0	0	0	0
case_021	4501504	3174400	7186432	4608000	11431936
case_022	0	0	112516	0	157360
case_023	0	73728	147456	1197318	544408
case_024	0	0	22415452	29037088	30282428
case_025	0	27648	55296	82944	110592
case_026	0	218548	923392	923392	923392
case_027	1400832	1105920	6107136	2211840	12023808
case_028	277412256	20677824	11546880	11508096	12550656
case_029	0	16400	0	490010	67656
case_030	5519360	7834624	9363456	8838144	11588608
case_031	3491328	4032000	4969984	5298688	4816896
case_032	0	168116	205008	0	0
case_033	4592640	6224928	7086144	7344224	8111232
case_034	1079832	1238528	2328576	3019264	4801550
case_035	0	0	765572	1128234	1160968
case_036	0	1152	2304	3456	4608
case_037	0	204800	1003520	614400	1310720
case_038	5627904	5607936	7012352	7758336	9409536
case_039	7929856	11067392	10113024	17899520	11489280

用例	N=1	N=2	N=3	N=4	N=5
case_040	0	0	0	1931186	1931186
case_041	30898176	33595392	35017728	38879232	40452096
case_042	6629376	7265568	7746560	8925184	9689088
case_043	49160	1884268	3085000	4132756	5217408
case_044	4438112	930400	930400	3517344	3517344
case_045	0	0	0	0	0
case_046	3176288	930400	2064576	2213920	2213920
case_047	0	794950	1239692	1947090	2584600
case_048	0	233030	576140	1756370	1756370
case_049	0	319282	497688	751934	872740
case_050	0	272582	481660	571290	571290
case_051	0	0	10748968	2228604	2228604
case_052	0	73728	147456	221184	294912
case_053	5405536	2184118	2848940	12542018	14846008
case_054	0	933062	1613448	2545234	2545234
case_055	69632	98816	99328	677058	427118
case_056	0	496950	767084	1107930	1454488
case_057	94216	269820	526964	610884	709300
case_058	28696576	33492992	31211520	42622976	31719424
case_059	1691648	360448	3397632	1081344	4325376
case_060	2068520	2724108	3164416	3510784	4039168
case_061	0	25344	179618	50688	328440
case_062	0	53760	41472	89088	86016
case_063	0	9019392	4119552	64512	367104
case_064	0	61542	99404	122098	122098
case_065	0	13824	27648	41472	55296
case_066	0	952044	1398840	2060756	2476400
case_067	264449408	8325504	10769152	14048384	17327616
case_068	0	1553566	2085676	3208234	3538800
case_069	0	371046	335116	432114	351476
case_070	0	88064	176128	264192	690024
case_071	0	326886	229324	267856	239056
case_072	108613680	113201200	119709046	125858856	130011184
case_073	141570432	3721600	7443200	11164800	14886400
case_074	6070280	6590472	7585792	7819264	8302592
case_075	0	2310790	3422476	5734290	6600472
case_076	28280832	31201280	34470912	32051200	34179072
case_077	0	57344	932152	147094	1592816
case_078	12644568	2148480	6471168	8694144	11371008
case_079	32448512	32612352	32722944	30445568	47677440
case_080	548864	2191360	3932160	2789376	3538944
case_081	0	0	0	0	0
case_082	0	832902	1428236	2106514	2932248
case_083	13934688	930400	1860800	2791200	3721600
case_084	0	0	0	0	0
case_085	0	3911858	5532684	8300178	8300178
case_086	0	518694	919116	1363698	1363698
case_087	7902720	8248320	6727680	21730796	6981120

用例	N=1	N=2	N=3	N=4	N=5
case_088	0	1049798	1449996	2268882	2451224
case_089	0	238720	477440	716160	954880
case_090	21233592	2917248	4296960	6445440	6445440
case_091	53947008	52114560	60897408	69020832	73166848
case_092	66000096	5437728	3859648	4167456	3950976
case_093	0	8192	16384	24576	32768
case_094	0	114464	212544	318528	425088
case_095	0	0	16330752	0	0
case_096	0	124748	125704	0	965588
case_097	10584064	14655488	18454528	14221312	21311488
case_098	0	960674	1341690	1341690	2001274
case_099	5484544	7315456	6590464	9818112	10776576
case_100	0	306188	0	1560842	2045040

表 A-7 问题三（只读 Cache）各用例的 Cache 命中率（按字节，%）

用例	N=1	N=2	N=3	N=4	N=5
case_001	0.0	0.0	0.0	0.0	0.0
case_002	0.0	1.1	1.4	2.8	1.7
case_003	0.0	16.1	24.3	40.4	40.4
case_004	0.0	16.7	0.0	14.1	0.0
case_005	0.0	17.8	27.7	37.8	34.8
case_006	2.9	35.1	43.4	34.2	41.6
case_007	0.0	0.0	0.0	0.0	0.0
case_008	0.0	0.0	0.0	0.0	0.0
case_009	0.0	22.4	29.5	42.3	37.6
case_010	0.0	24.3	20.0	25.8	38.3
case_011	0.0	0.0	39.5	0.0	35.3
case_012	0.0	0.0	21.9	19.0	21.9
case_013	0.0	0.0	0.0	0.0	0.0
case_014	12.3	19.0	33.9	37.7	18.5
case_015	16.9	1.1	49.2	35.0	21.0
case_016	0.0	0.0	0.0	3.0	3.0
case_017	4.0	5.3	9.3	10.8	8.9
case_018	34.4	57.0	59.1	59.3	45.1
case_019	0.0	12.5	22.8	23.4	32.5
case_020	0.0	0.0	0.0	0.0	0.0
case_021	50.1	21.8	48.9	0.0	57.0
case_022	0.0	0.0	34.5	0.0	42.1
case_023	0.0	9.7	8.5	33.4	23.0
case_024	0.0	0.0	2.8	6.4	8.6
case_025	0.0	0.0	0.0	0.0	0.0
case_026	0.0	26.9	33.8	33.8	33.8
case_027	26.9	0.0	31.6	0.0	29.6
case_028	0.0	13.9	35.9	16.5	44.1
case_029	0.0	4.5	0.0	63.8	19.5
case_030	26.0	44.4	54.4	65.1	59.4
case_031	26.1	33.9	30.5	31.5	32.1

用例	N=1	N=2	N=3	N=4	N=5
case_032	0.0	33.4	41.9	0.0	0.0
case_033	9.6	31.2	35.0	36.9	36.8
case_034	30.9	35.0	50.3	55.7	44.3
case_035	0.0	0.0	42.7	55.4	54.8
case_036	0.0	0.0	0.0	0.0	0.0
case_037	0.0	0.0	52.6	0.0	0.0
case_038	17.4	19.1	28.9	26.0	39.4
case_039	0.0	37.0	20.1	46.5	22.9
case_040	0.0	0.0	0.0	20.7	20.7
case_041	10.2	20.0	26.7	33.0	32.6
case_042	31.9	35.6	33.9	35.7	38.5
case_043	3.3	31.8	45.7	49.3	53.9
case_044	82.0	0.2	0.2	73.3	73.3
case_045	0.0	0.0	0.0	0.0	0.0
case_046	75.3	0.0	38.3	62.1	62.1
case_047	0.0	16.4	28.0	28.9	42.0
case_048	0.0	0.1	7.5	24.7	24.7
case_049	0.0	8.5	26.7	37.6	44.1
case_050	0.0	25.7	34.6	35.2	35.2
case_051	0.0	0.0	10.6	23.1	23.1
case_052	0.0	5.2	8.2	11.6	16.3
case_053	62.5	27.6	42.2	37.0	29.5
case_054	0.0	7.4	18.4	27.9	27.9
case_055	10.0	8.2	13.7	46.5	33.7
case_056	0.0	17.9	35.2	40.7	50.7
case_057	21.9	40.6	57.6	60.0	63.8
case_058	0.0	11.5	42.6	26.9	2.4
case_059	61.0	0.0	61.3	0.0	79.5
case_060	42.3	40.4	46.9	47.2	47.0
case_061	0.0	4.5	22.5	8.5	21.6
case_062	0.0	0.0	0.0	0.1	0.1
case_063	0.0	4.7	11.7	0.2	0.1
case_064	0.0	16.2	24.4	29.5	29.5
case_065	0.0	1.5	0.0	9.5	11.7
case_066	0.0	16.9	21.8	34.0	36.8
case_067	0.0	33.8	53.2	63.8	17.7
case_068	0.0	24.3	15.8	21.3	23.6
case_069	0.0	16.3	32.8	41.8	48.6
case_070	0.0	7.7	8.9	9.6	26.0
case_071	0.0	17.7	20.5	33.5	17.6
case_072	2.6	13.4	14.4	18.3	32.5
case_073	0.0	48.9	42.2	74.2	78.9
case_074	16.7	29.3	30.7	29.7	47.0
case_075	0.0	14.4	30.4	47.5	43.7
case_076	13.4	27.6	35.7	34.5	38.7
case_077	0.0	2.6	17.8	0.0	21.2
case_078	0.0	0.0	51.8	9.2	35.7
case_079	0.0	0.0	0.0	0.0	24.4

用例	N=1	N=2	N=3	N=4	N=5
case_080	16.9	43.6	55.4	34.7	43.0
case_081	0.0	0.0	0.0	0.0	0.0
case_082	0.0	20.2	28.4	50.8	51.9
case_083	79.6	44.0	30.5	70.2	19.0
case_084	0.0	0.0	0.0	0.0	0.0
case_085	0.0	15.9	30.2	50.0	50.0
case_086	0.0	19.5	21.9	38.2	38.2
case_087	18.3	19.0	23.3	47.1	24.2
case_088	0.0	24.7	14.7	20.4	20.2
case_089	0.0	0.0	27.1	0.0	0.0
case_090	0.0	18.3	30.0	21.1	21.1
case_091	10.5	33.9	43.5	28.8	31.3
case_092	79.4	0.0	14.3	30.7	45.3
case_093	0.0	0.0	0.0	0.0	0.0
case_094	0.0	15.1	25.7	31.2	40.2
case_095	0.0	0.0	0.0	0.0	0.0
case_096	0.0	31.7	32.0	0.0	51.1
case_097	0.0	5.0	30.0	4.2	24.5
case_098	0.0	17.7	29.1	29.1	34.9
case_099	25.2	45.4	36.3	58.3	55.0
case_100	0.0	40.8	0.0	48.6	65.0

表 A-8 只读 Cache 相对无 L2 的加速比 (MK(无 L2) / MK(只读 Cache))

用例	N=1	N=2	N=3	N=4	N=5
case_001	1.000	1.000	1.000	1.000	1.000
case_002	1.000	1.000	1.000	1.000	1.000
case_003	1.000	1.000	1.006	1.015	1.015
case_004	1.000	1.000	1.000	1.000	1.000
case_005	1.000	1.001	1.007	1.059	1.032
case_006	1.000	1.002	1.021	1.021	1.018
case_007	1.000	1.000	1.000	1.000	1.000
case_008	1.000	1.000	1.000	1.000	1.000
case_009	1.000	1.000	1.000	1.014	1.030
case_010	1.000	1.000	1.023	1.042	1.058
case_011	1.000	1.000	1.000	1.000	1.000
case_012	1.000	1.000	1.000	1.046	1.041
case_013	1.000	1.000	1.000	1.000	1.000
case_014	1.001	1.001	1.009	1.013	1.003
case_015	1.000	1.002	1.000	1.016	1.000
case_016	1.000	1.000	1.000	1.000	1.000
case_017	1.000	1.000	1.000	1.000	1.000
case_018	1.001	1.001	1.003	1.008	1.001
case_019	1.000	1.000	1.002	1.008	1.027
case_020	1.000	1.000	1.000	1.000	1.000
case_021	1.001	1.000	1.000	1.000	1.001
case_022	1.000	1.000	1.000	1.000	1.002
case_023	1.000	1.000	1.000	1.013	1.000

用例	N=1	N=2	N=3	N=4	N=5
case_024	1.000	1.000	1.000	1.002	1.001
case_025	1.000	1.000	1.000	1.000	1.000
case_026	1.000	1.000	1.000	1.000	1.000
case_027	1.002	1.000	1.000	1.000	1.000
case_028	1.000	1.001	1.001	1.002	1.008
case_029	1.000	1.000	1.000	1.005	1.000
case_030	1.000	1.000	1.000	1.001	1.002
case_031	1.000	1.002	1.000	1.000	1.001
case_032	1.000	1.008	1.005	1.000	1.000
case_033	1.000	1.000	1.002	1.004	1.003
case_034	1.000	1.000	1.001	1.007	1.005
case_035	1.000	1.000	1.001	1.006	1.015
case_036	1.000	1.000	1.000	1.000	1.000
case_037	1.000	1.000	1.000	1.000	1.000
case_038	1.001	1.000	1.004	1.001	1.014
case_039	1.000	1.004	1.003	1.064	1.031
case_040	1.000	1.000	1.000	1.006	1.006
case_041	1.000	1.002	1.002	1.002	1.001
case_042	1.007	1.002	1.004	1.013	1.004
case_043	1.000	1.000	1.002	1.004	1.054
case_044	1.432	1.001	1.001	1.097	1.097
case_045	1.000	1.000	1.000	1.000	1.000
case_046	1.128	1.000	1.029	1.200	1.200
case_047	1.000	1.000	1.001	1.008	1.028
case_048	1.000	1.000	1.000	1.003	1.003
case_049	1.000	1.000	1.000	1.012	1.010
case_050	1.000	1.000	1.001	1.000	1.000
case_051	1.000	1.000	1.002	1.002	1.002
case_052	1.000	1.000	1.000	1.000	1.000
case_053	1.031	1.000	1.000	1.055	1.074
case_054	1.000	1.000	1.001	1.010	1.010
case_055	1.000	1.000	1.000	1.052	1.083
case_056	1.000	1.000	1.002	1.001	1.013
case_057	1.000	1.000	1.039	1.013	1.011
case_058	1.000	1.001	1.010	1.010	1.001
case_059	1.017	1.000	1.003	1.000	1.000
case_060	1.001	1.000	1.001	1.001	1.000
case_061	1.000	1.000	1.007	1.000	1.040
case_062	1.000	1.000	1.000	1.000	1.000
case_063	1.000	1.001	1.007	1.000	1.000
case_064	1.000	1.002	1.004	1.002	1.002
case_065	1.000	1.000	1.000	1.000	1.012
case_066	1.000	1.000	1.000	1.009	1.027
case_067	1.000	1.001	1.003	1.007	1.003
case_068	1.000	1.000	1.001	1.034	1.099
case_069	1.000	1.012	1.016	1.107	1.143
case_070	1.000	1.000	1.000	1.000	1.000
case_071	1.000	1.020	1.007	1.039	1.074

用例	N=1	N=2	N=3	N=4	N=5
case_072	1.000	1.000	1.001	1.002	1.006
case_073	1.000	1.009	1.013	1.040	1.066
case_074	1.002	1.002	1.001	1.008	1.013
case_075	1.000	1.000	1.000	1.001	1.008
case_076	1.000	1.001	1.002	1.002	1.005
case_077	1.000	1.000	1.007	1.000	1.038
case_078	1.000	1.000	1.018	1.000	1.063
case_079	1.000	1.000	1.000	1.000	1.000
case_080	1.001	1.002	1.003	1.029	1.092
case_081	1.000	1.000	1.000	1.000	1.000
case_082	1.000	1.000	1.000	1.000	1.006
case_083	1.118	1.000	1.000	1.000	1.000
case_084	1.000	1.000	1.000	1.000	1.000
case_085	1.000	1.000	1.000	1.000	1.000
case_086	1.000	1.001	1.011	1.047	1.047
case_087	1.001	1.001	1.002	1.005	1.000
case_088	1.000	1.000	1.011	1.011	1.085
case_089	1.000	1.000	1.000	1.000	1.000
case_090	1.000	1.000	1.006	1.071	1.071
case_091	1.002	1.002	1.002	1.002	1.007
case_092	1.111	1.000	1.001	1.009	1.012
case_093	1.000	1.000	1.000	1.000	1.000
case_094	1.000	1.000	1.000	1.025	1.011
case_095	1.000	1.000	1.000	1.000	1.000
case_096	1.000	1.000	1.000	1.000	1.026
case_097	1.000	1.000	1.001	1.000	1.000
case_098	1.000	1.001	1.006	1.006	1.070
case_099	1.003	1.006	1.002	1.003	1.027
case_100	1.000	1.000	1.000	1.004	1.058

注：表 A-8 中大于 1 表示只读 Cache 更快。case_030 与 case_048 在 $N = 3$ 时的值略小于 1（相对差分别为 0.0012% 与 0.0067%），保留三位小数后显示为 1.000，即第 7.6 节所述的 FIFO 时序例外。

附录 B 程序说明与复现步骤

运行环境。 程序为纯 Python 实现，在赛题材料包根目录下运行，要求 Python 3.10 及以上；依赖见仓库根目录的 requirements.txt（numpy、scipy、matplotlib、python-docx）。CAP-LS 的求解部分只用标准库，scipy 只用于统计检验，matplotlib 与 python-docx 只用于作图和生成 Word。官方评估脚本 code/ 与数据 data/ 保持赛题原样，data/config.txt 未做任何修改。

表 B-1 程序与数据的目录结构

路径	内容
code/、data/	赛题提供的评估程序、示例生成器、100 个用例与固定配置（只读）
solution/solve.py	单用例求解入口，输出官方格式方案
solution/npu/	算法实现：读图与 COPY 收缩、S1 切块、S2 分配与局部搜索、S3 分层、表调度、问题一退火、事件模拟代理、官方评估封装与哈希缓存
solution/run_experiments.py	实验驱动：主实验、基线、消融、粒度、硬件灵敏度等阶段
solution/build_pool.py、 export_plans.py、 verify_final.py	保底池汇总；导出官方格式最终方案；用官方命令行脚本逐个复核
solution/paper_v2/	本文的数字提取（facts.py）、作图（figs_*.py、 trace_fig.py）与 Word 排版（build_docx.py）
results/	全部实验记录（csv、json）、评估缓存与最终方案

输入与输出格式。 输入为赛题附录 B.3 格式的计算图 data/<case>.json；输出严格按赛题附录 B.5，文件名为 <case>_multicore_res.json，顶层只有 node_to_subgraph 与 core_schedules 两个字段。CAP-LS 生成的子图编号按（层，核）递增分配，每个核的内层数组就是其子图编号的升序，这一性质保证了引理 4-1 的可行性。

单用例求解与官方评估。

```
# 生成问题 2、4 核的方案（候选网格逐个交官方评估器择优）
python solution/solve.py data/case_001.json --problem 2 --cores 4 -o out.json
# 用官方脚本评估（问题 1/2/3 分别换成对应脚本）
python code/multicore_cut_evaluate_problem_2.py data/case_001.json out.json \n      --
config data/config.txt
```

solve.py 只包含候选网格，不含退火、采样和保底池，因此其结果可能略低于正文报告的最终方案（第 9.1.2 节）；所有网格候选都失败时，它退回整图单核方案，保证总有可行输出。

复现全部实验与本文。 评估结果按方案哈希缓存在 results/cache/p{1,2,3}/，删除该目录即从零重算；随机过程全部固定种子，重复运行得到相同的 final.csv。

```
bash solution/run_all.sh 16
python solution/build_pool.py --jobs 16
python solution/export_plans.py
python solution/verify_final.py --problems 1 2 3
python solution/paper_v2/facts.py
python solution/paper_v2/figs_diagrams.py
python solution/paper_v2/figs_data.py
python solution/paper_v2/trace_fig.py
python solution/paper_v2/build_docx.py

# 主实验、基线、消融与灵敏度
# 保底池汇总 -> final.csv
# 导出官方格式最终方案
# 官方命令行逐方案复核
# 提取正文数字
# 示意图与流程图
# 数据图
# 官方 trace 甘特图
# 生成本文的 Word 与 PDF
```

正文与附录中的实验数字均不手工填写：facts.py 从 results/ 中的 csv 与 json 计算全部数字，Word 源文件中以占位符引用，build_docx.py 遇到未定义的占

位符合直接报错。

附录 C 核心程序（节选）

（1）跨核同步层次与子图成形（solution/npu/stratify.py，对应定义 4-1 与引理 4-1）

```
def cross_core_levels(g, core_of_op):
    """r(v): 沿依赖路径累计的换核次数（拓扑序一次扫描）。"""
    level = {}
    for v in g.topo:
        cv = core_of_op[v]
        best = 0
        for p in g.preds[v]:
            cand = level[p] + (1 if core_of_op[p] != cv else 0)
            best = max(best, cand)
        level[v] = best
    return level

def build_subgraphs(g, core_of_op, num_cores, rank, max_ops=None, levels=None):
    """子图 = (核心, 层次) 等价类, 按 (层次, 核心, 切片序) 递增编号。"""
    levels = levels if levels is not None else cross_core_levels(g, core_of_op)
    check_monotone(g, core_of_op, levels) # 显式层指派须满足推论 4-3 的单调性
    buckets = defaultdict(list)
    for v in g.nodes:
        buckets[(levels[v], core_of_op[v])].append(v)
    mapping, schedules, sid = {}, [], [[] for _ in range(num_cores)], 0
    for (r, k) in sorted(buckets): # 编号沿 (r, k) 递增 -> 商图无环
        members = sorted(buckets[(r, k)], key=lambda v: rank[v])
        step = max_ops or len(members)
        for i in range(0, len(members), step): # 可选: 按规模沿 sigma 切片
            for v in members[i:i + step]:
                mapping[v] = sid
                schedules[k].append(sid)
                sid += 1
    return mapping, schedules, levels
```

（2）容量感知表调度的评分（solution/npu/listsched.py，对应式（6-5）与算法 6-1）

```
def upward_rank(g, core_of_op, delay, bandwidth):
    """rank_u(v) = c_v + max_s (comm(v,s) + rank_u(s)), 归一化到 [0, 1]。"""
    pb, rank = pair_bytes(g), {}
    for v in reversed(g.topo):
        best = 0.0
        for s in g.succs[v]:
            same = core_of_op[s] == core_of_op[v]
            comm = 0.0 if same else delay + pb[v].get(s, 0) / bandwidth
            best = max(best, comm + rank[s])
        rank[v] = max(1, g.cycles(v)) + best
    top = max(rank.values(), default=1.0) or 1.0
    return {v: r / top for v, r in rank.items()}

def pick(k):
    # 为核 k 选下一个算子
    pool = ready[k][:TOP_CANDIDATES] # 秩最高的 64 个就绪算子
    latest = max(range(4), key=lambda p: (pipe_free[k][p], -p))
    best_feasible, best_fallback = None, None
    for _, v in pool:
        alloc, dl, c = delta_live(v, k) # 驻留净变化 Δlive 与对应容量 C
```

```

if best_fallback is None or (dl, v) < best_fallback[:2]:
    best_fallback = (dl, v)
if any(used[k][pos] + alloc[pos] > limit[pos] for pos in ("L1", "UB")):
    continue # 超过 alpha*C 的候选不参与评分
p = PIPE_INDEX[g.pipe(v)]
score = (rank[v] - mu * dl / c + (nu if p != latest else 0.0)
        + noise * gumbel.get(v, 0.0)) # 式 (6-5)
if best_feasible is None or (score, -v) > best_feasible[:2]:
    best_feasible = (score, -v, v)
return best_feasible[2] if best_feasible else best_fallback[1]

```